

QuartzHub-Handbuch

Aus einem Ordner voller Notizen eine Website

Stand 14. November 2026

Inhalt

QuartzHub 4

1 – Einstieg 6
- 1.1 – Was QuartzHub ist 7
- 1.2 – Installieren und erster Start 9
- 1.3 – Das Fenster lesen 12
- 1.4 – Die erste Website 15

2 – Projekte 18
- 2.1 – Das Fenster „Projekte“ 19
- 2.2 – Ein neues Projekt anlegen 23
- 2.3 – Ein vorhandenes Projekt öffnen 27
- 2.4 – Ein Projekt duplizieren 28
- 2.5 – Die Einstellungen 30

3 – Übersicht 35
- 3.1 – Die Übersicht 36

4 – Einrichtung 40
- 4.1 – Website 41
- 4.2 – Der Content-Ordner 45
- 4.3 – Das Projektbild 47
- 4.4 – Übersetzungen 48
- 4.5 – Plugins 50
- 4.6 – Navigationen 53
- 4.7 – Zusätzliche Inhalte 56
- 4.8 – Mehrsprachigkeit 59
- 4.9 – Formulare 61
- 4.10 – Marktplatz 65

5 – Gestaltung 67
- 5.1 – Die Ebenen 68
- 5.2 – Layout-Übersicht 70
- 5.3 – Komponenten 73
- 5.4 – Frames 76
- 5.5 – Stile ▸ Basis 79
- 5.6 – Community-Themes 82
- 5.7 – Variablen 84
- 5.8 – Eigenes CSS 86

5.9 – Vorlagen 89

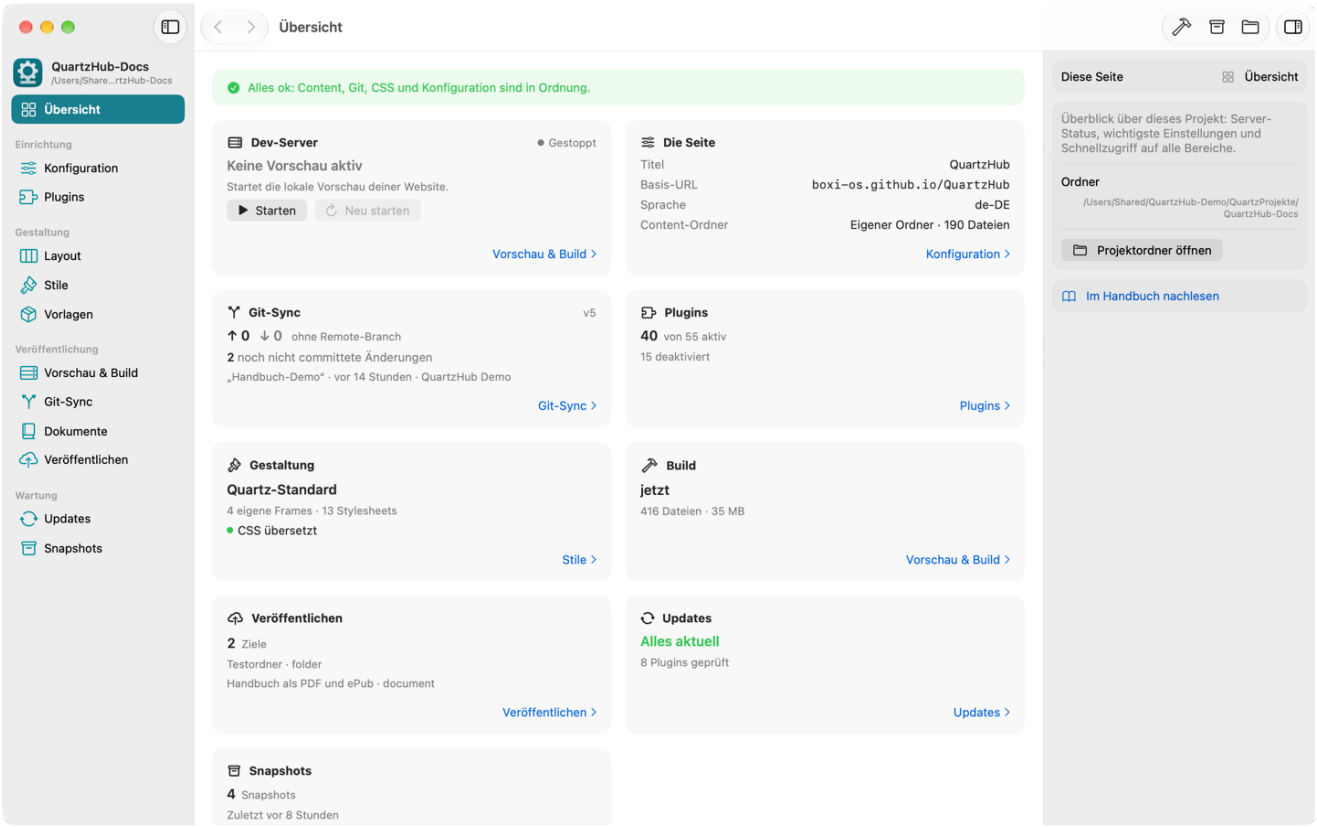
6 – Veröffentlichung 92
6.1 – Die Vorschau 93
6.2 – Der Build 95
6.3 – Die Ausgabe lesen 97
6.4 – Fremde Server 99
6.5 – Git-Sync 100
6.6 – Dokumente 103
6.7 – Zugänge und Ziele 105
6.8 – GitHub Pages 108
6.9 – Auf einen Webspace 109
6.10 – In einen Ordner 111
6.11 – Webhook 112
6.12 – Was hochgeladen wird 113

7 – Wartung 115
7.1 – Updates 116
7.2 – Snapshots 119
7.3 – Zurückholen 122
7.4 – Aufräumen 124

8 – Nachschlagen 125
8.1 – Menüs und Kürzel 126
8.2 – Wo die Daten liegen 129
8.3 – Verbindungen ins Netz 132
8.4 – Bekannte Grenzen 134
8.5 – Glossar 135

QuartzHub

QuartzHub ist eine Mac-App für **Quartz**, einen Generator für statische Websites. Ohne sie bedient man Quartz über die Kommandozeile und bearbeitet seine Konfiguration im Texteditor; mit ihr klickt man. Dieses Handbuch erklärt jedes Fenster und jede Seite von QuartzHub: was sie tut, wann man sie braucht und was man dort besser nicht tut.



Die Übersicht eines Projekts: Vorschau, Build, Veröffentlichen, Snapshots und Updates auf einen Blick.

Was QuartzHub kann

Aus Notizen wird eine Website

Ein Obsidian-Vault oder irgendein Ordner wird zum Content-Ordner, **verknüpft**, wenn man in Obsidian weiterschreiben will, kopiert, wenn nicht. Umstellen geht später, ohne eine Notiz zu verlieren. → 4.2

Gestalten, ohne CSS zu schreiben

Farben und Schriften, Community-Themes, die CSS-Variablen der Website für hell und dunkel getrennt, eigene Stylesheets. Das Seitenraster entsteht in einem Editor mit der Maus, je Bildschirmbreite. → 5.1, 5.4

Plugins aus einem Marktplatz

Durchsuchen, installieren, einstellen, mit einem Formular statt YAML. Navigationen, zusätzliche Inhaltsboxen, Mehrsprachigkeit und Formulare haben eigene Editoren mit einer Vorschau daneben. → 4.5

Ansehen, bevor es jemand anders sieht

Die Vorschau läuft im Fenster und baut bei jeder Änderung neu, in der Breite eines Desktops, Tablets oder Telefons; die Ausgabe von Quartz steht darunter. → 6.1

Bücher aus der Website

Eine Auswahl von Seiten, in der gewünschten Reihenfolge, als PDF oder ePub, mit Inhaltsverzeichnis und Formeln. Auch dieses Handbuch gibt es so. → 6.6

Veröffentlichen und zurückkommen

GitHub Pages, ein Webspace über SFTP, rsync oder FTP, ein Ordner, ein Webhook, mit einer Vorschau dessen, was übertragen wird. Und vor größeren Änderungen ein Snapshot, zu dem man zurückkehren kann. → 6.7, 7.2

Wo anfangen

- **Neu hier:** 1.1 – Was QuartzHub ist, dann 1.4 – Die erste Website.
- **Etwas nachschlagen:** Menüs und Kürzel, Glossar.

Wie man *Notizen schreibt*, damit daraus gute Seiten werden, steht im **Example-Handbuch**.

1 – Einstieg

Dieser Teil setzt nichts voraus. Wer Notizen schreibt und daraus eine Website machen will, ist hier richtig, auch ohne je etwas von Quartz, git oder einem Static-Site-Generator gehört zu haben.

1. **Was QuartzHub ist**: was es einem abnimmt und was es bewusst nicht tut.
2. **Installieren und erster Start**: was der Mac braucht und was beim ersten Öffnen passiert.
3. **Das Fenster lesen**: Seitenleiste, Inspektor, Verlauf, Speichern. Die Bedienung ist auf jeder Seite dieselbe; wer diese Seite kennt, kennt sie überall.
4. **Die erste Website**: vom leeren Projekt bis zur Vorschau.

1.1 – Was QuartzHub ist

QuartzHub ist eine Mac-App für **Quartz**. Ohne sie bedient man Quartz über die Kommandozeile und bearbeitet seine Konfigurationsdatei im Texteditor; mit ihr klickt man.

Was es einem abnimmt

Ein Quartz-Projekt besteht aus einer Konfigurationsdatei, einem Ordner voller Notizen, einigen Stylesheets und einer Liste installierter **Plugins**. Alles davon ist eine Textdatei, und alles davon ließe sich von Hand bearbeiten. QuartzHub gibt jedem dieser Dinge eine Seite:

- **Ein neues Projekt anlegen**, ohne git und npm selbst zu bedienen.
- **Die Konfiguration bearbeiten**, ohne YAML zu schreiben, mit einer Erklärung bei jedem Feld.
- **Den Content-Ordner verknüpfen**, auch direkt mit einem Obsidian-**Vault**, und zwischen **Kopieren** und **Verknüpfen** wechseln, ohne Notizen zu verlieren.
- **Plugins installieren und einstellen**, aus einem durchsuchbaren **Marktplatz**, mit einem Formular statt YAML.
- **Farben, Schriften und Abstände ändern**, in Ebenen, die in der Reihenfolge nebeneinander liegen, in der sie sich überschreiben.
- **Das Seitenraster bauen**, in einem Editor mit der Maus statt in CSS.
- **Die Vorschau starten und die Website bauen**, mit der Ausgabe von Quartz im selben Fenster.
- **Bücher machen**: eine Auswahl von Seiten als PDF oder ePub.
- **Veröffentlichen**, auf mehreren Wegen, mit einer Vorschau dessen, was übertragen wird.
- **Zurückkommen**, wenn etwas schiefgegangen ist: QuartzHub legt vor jeder größeren Änderung von selbst einen **Snapshot** an.

Was es nicht tut

Es ist kein Editor. Notizen schreibt man woanders, in Obsidian, in einem Texteditor, irgendwo. QuartzHub fasst den Inhalt des Content-Ordners nicht an; es baut daraus eine Website.

Es ersetzt Quartz nicht. Was auf der fertigen Seite erscheint, entscheiden Quartz und seine Plugins. Wo dieses Handbuch bei „das macht Quartz so“ aufhört, geht die **Quartz-Dokumentation** weiter.

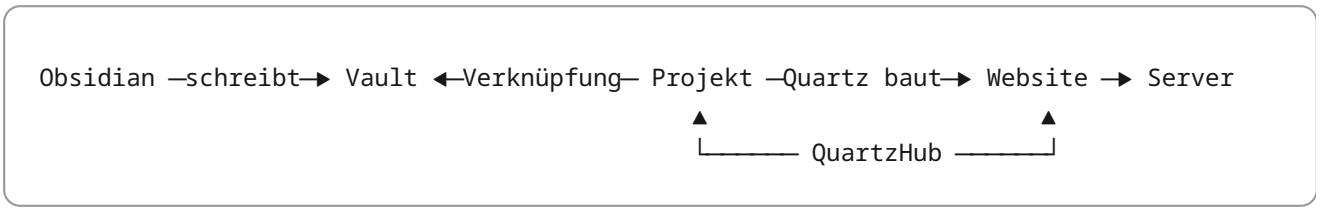
Es versteckt die Dateien nicht. Jede Änderung landet in genau der Datei, in der sie auch gelandet wäre, wenn man sie von Hand gemacht hätte. Ein Projekt, das mit QuartzHub eingerichtet wurde, lässt sich jederzeit von Hand weiterführen, und umgekehrt.

Die Werkzeuge dahinter

Wer mit QuartzHub arbeitet, hat mit vier Programmen zu tun. Bedienen muss man nur eines davon.

Programm	Rolle	Muss man es bedienen?
Obsidian oder ein anderer Editor	Hier werden die Notizen geschrieben, als Markdown-Dateien in einem Ordner.	Ja, zum Schreiben. Obsidian ist keine Voraussetzung, aber Quartz versteht seine [[8-nachschlagen/05-glossar#Wikilink
Quartz	Macht aus den Notizen eine Website. Diesen Vorgang nennt man [[8-nachschlagen/05-glossar#Build	Build]].
Node und npm	Die Laufzeit, in der Quartz arbeitet, und ihr Paketverwalter.	Nein, QuartzHub bringt beide mit.
git	Führt die Geschichte von Dateien.	Nur unter **Git-Sync** und beim Veröffentlichen über einen [[8-nachschlagen/05-glossar#Branch

So hängen sie zusammen:



Der Vault muss nicht außerhalb liegen. Wer keinen hat, lässt QuartzHub einen leeren Content-Ordner im Projekt anlegen und schreibt darin.

Für wen

Für alle, die Notizen schreiben und daraus eine Website machen wollen. Vorkenntnisse in Quartz sind nicht nötig, in git auch nicht. Wer Quartz schon benutzt, findet dieselben Dateien wieder, aber mit einer Vorschau neben jedem Schalter und einem Weg zurück, wenn ein Update etwas kaputt macht.

1.2 – Installieren und erster Start

Systemvoraussetzungen

- **macOS 26 oder neuer.**
- **Ein Mac mit Apple-Chip** (M1 oder neuer). Für Macs mit Intel-Prozessor gibt es keine Fassung.
- **Internet** beim Anlegen eines Projekts (Quartz kommt von GitHub), beim Installieren von Plugins und beim Veröffentlichen. Bearbeiten, Vorschau und Build gehen ohne.

Quartz, Node und npm muss man **nicht** installieren: Node und npm bringt QuartzHub mit, und Quartz liegt in jedem Projekt selbst. git nimmt QuartzHub vom System, wenn dort eines antwortet. Dann gelten die eigene `.gitconfig` und die eigenen Zugangshelfer weiter. Sonst nimmt es das mitgebrachte.

Herunterladen und installieren

QuartzHub gibt es als DMG auf der **Releases-Seite**. Die DMG öffnen und QuartzHub in den Ordner „Programme“ ziehen.

macOS lässt QuartzHub beim ersten Mal nicht starten

QuartzHub ist nur ad hoc signiert und nicht bei Apple notarisiert; dafür bräuchte es ein Developer-ID-Zertifikat. Gatekeeper, die Startkontrolle von macOS, verweigert deshalb den ersten Start mit dem Hinweis, Apple könne die App nicht überprüfen. Einmal starten und die Meldung schließen, dann **Systemeinstellungen ▸ Datenschutz & Sicherheit**, im Abschnitt **Sicherheit** auf **Dennoch öffnen** klicken und mit dem Anmeldepasswort bestätigen. Über das Kontextmenü (rechte Maustaste ▸ Öffnen) geht es seit macOS 15 nicht mehr.

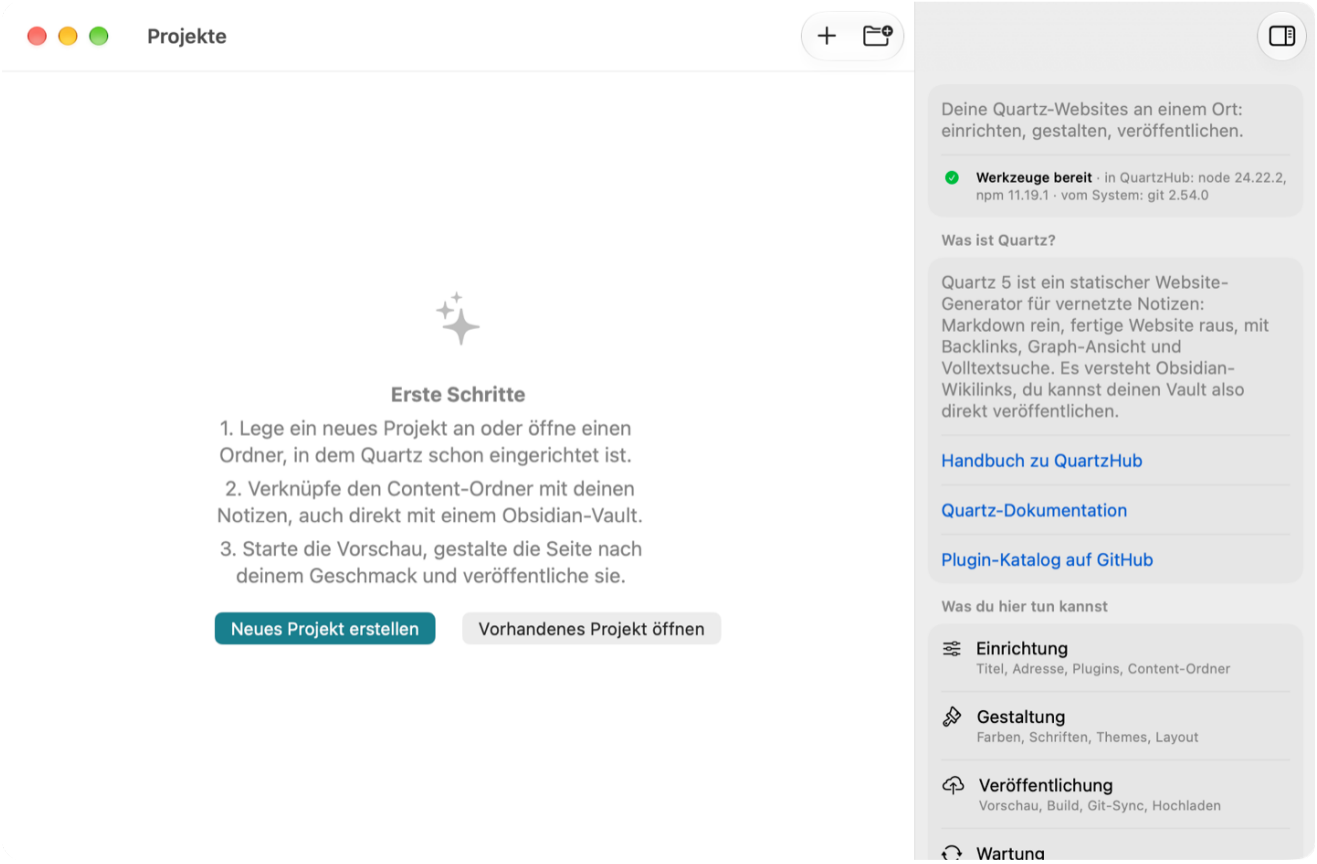
Wer das Terminal mag, entfernt stattdessen die Quarantäne-Markierung, die macOS an heruntergeladene Dateien hängt:

```
xattr -dr com.apple.quarantine /Applications/QuartzHub.app
```

Aktualisieren heißt: die neue DMG laden und die App im Ordner „Programme“ ersetzen. Projekte, Einstellungen und Zugänge liegen außerhalb der App und bleiben dabei, wie sie sind (siehe 8.2).

Der erste Start

Beim ersten Start öffnet sich das Fenster **Projekte**.



Das Fenster „Projekte“ beim ersten Start: links die Projektliste, noch leer, mit den ersten Schritten; rechts der Inspektor.

Im Inspektor rechts steht, was QuartzHub über den Mac weiß:

- **Werkzeuge bereit** heißt: `node`, `npm` und `git` haben auf die Frage nach ihrer Version geantwortet. Dahinter steht, woher jedes kommt, „in QuartzHub“ oder „vom System“.
- Antwortet eines nicht, steht über der Projektliste ein Hinweis mit dem, was ohne es nicht geht: Builds, neue Projekte, Plugins aus einer git-Quelle, Snapshots. Weil Node und npm zu QuartzHub gehören, heißt ein Hinweis über sie immer: Diese Installation ist unvollständig. Am schnellsten hilft dann, QuartzHub neu zu installieren.

Darunter erklärt der Inspektor kurz, was Quartz ist, und verweist auf die Quartz-Dokumentation und den Plugin-Katalog.

Fragen von macOS

Zwei Dinge fragt macOS nicht beim ersten Start, sondern erst, wenn es so weit ist:

- **Lokales Netzwerk.** Beim ersten Veröffentlichen auf einen Server im eigenen Netzwerk (SFTP, rsync, FTP) fragt macOS, ob QuartzHub auf das lokale Netzwerk

zugreifen darf. Ohne diese Erlaubnis scheitert die Verbindung; QuartzHub sagt dann, wo man sie nachträglich gibt: **Systemeinstellungen ▸ Datenschutz & Sicherheit ▸ Lokales Netzwerk**.

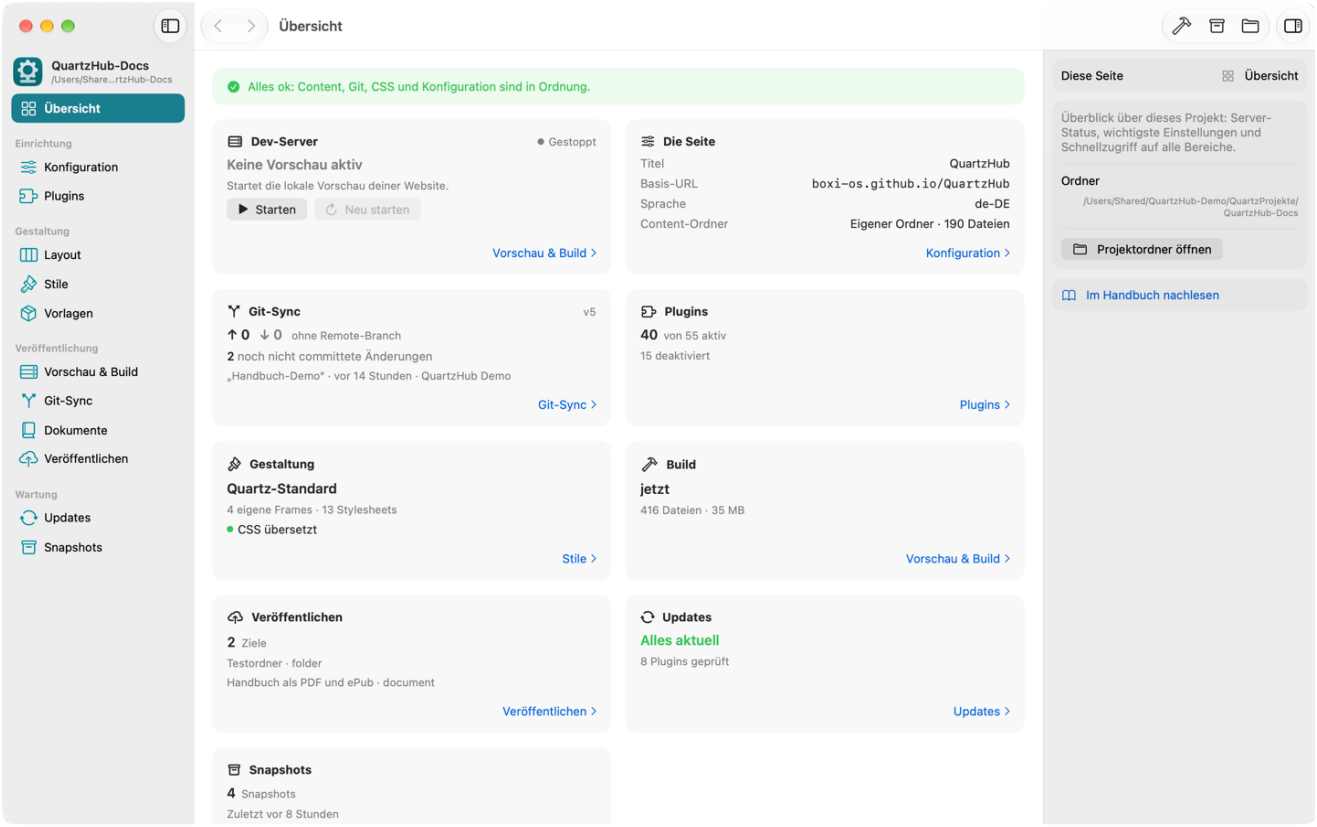
- **Schlüsselbund.** Passwörter und `Token` für das Veröffentlichen legt QuartzHub im Schlüsselbund von macOS ab, nie in einer Datei. Liest eine neue Fassung von QuartzHub einen Eintrag zum ersten Mal, fragt macOS einmal; „Immer erlauben“ genügt.

„Kann nicht prüfen“ ist nicht „alles gut“

Das gilt an vielen Stellen in QuartzHub, zum ersten Mal hier: Lässt sich etwas nicht feststellen, sagt QuartzHub das, statt Entwarnung zu geben. Ein Katalog, der nicht geladen werden konnte, ist nicht leer, und eine Version, die nicht ermittelt werden konnte, ist nicht die neueste.

1.3 – Das Fenster lesen

Jede Seite eines Projekts ist gleich aufgebaut. Wer diese Seite gelesen hat, muss die Bedienung in den Teilen 3 bis 7 nicht noch einmal lernen.



Das Projektfenster: links die Seitenleiste, in der Mitte die Seite, rechts der Inspektor.

Zwei Arten Fenster

Das Fenster **Projekte** listet alle Projekte; dort legt man sie an, öffnet, dupliziert und entfernt sie (2.1). Jedes Projekt öffnet sich in einem **eigenen Fenster**. Zwei Projekte können nebeneinander offen sein, und wer ein offenes Projekt noch einmal öffnet, holt sein Fenster nach vorn. Zurück zur Liste geht es über **Fenster ▸ Projekte** (⇧⌘0).

Die Seitenleiste

Oben stehen Bild, Name und Ordner des Projekts. Darunter die Seiten, in der Reihenfolge, in der man ein Projekt meist einrichtet:

Gruppe	Seiten	Teil
–	Übersicht	[[3-uebersicht/index
Einrichtung	Konfiguration, Plugins	[[4-einrichtung/index

Gestaltung	Layout, Stile, Vorlagen	[[5-gestaltung/index
Veröffentlichung	Vorschau & Build, Git-Sync, Dokumente, Veröffentlichen	[[6-veroeffentlichung/index
Wartung	Updates, Snapshots	[[7-wartung/index

Die Seitenleiste lässt sich mit dem Knopf links oben aus- und einblenden.

Titel und Verlauf

Der **Titel** des Fensters nennt den Ort, nicht das Projekt: oben die Seite, darunter der Reiter, wenn die Seite welche hat. Im Menü **Fenster** steht dagegen „Projekt – Seite“, damit sich zwei Projektfenster unterscheiden lassen.

Zurück und **Vorwärts** links in der Symbolleiste gehen durch den Verlauf des Fensters, Seite für Seite und Reiter für Reiter, wie in einem Browser. Die Kürzel sind ⌘[und ⌘], auf einer deutschen Tastatur ⌘Ö und ⌘Ä. Ein Verweis auf eine andere Seite, etwa „Plugins ›“ auf der Übersicht, ist ein Schritt wie jeder andere; Zurück führt wieder dorthin, woher man kam. Dieselben Befehle und alle Seiten stehen im Menü **Gehe zu**.

Reiter

Konfiguration, Plugins, Layout und Stile haben mehrere Reiter. Sie stehen als Segmente in einer eigenen Zeile über der Seite. Ist das Fenster zu schmal für alle Segmente, werden sie zu einem Menü mit dem Namen des gewählten Reiters.

Der Inspektor

Rechts steht der **Inspektor**, auf jeder Seite an derselben Stelle. Ist auf der Seite etwas gewählt, eine Zeile in einer Liste, ein Bereich in einem Raster, zeigt er dessen Einzelheiten, Einstellungen und Aktionen. Ist nichts gewählt, erklärt er die Seite, zeigt, was für sie gilt, und führt mit **Im Handbuch nachlesen** zum passenden Kapitel; mit Auswahl tut das **Hilfe ▸ Diese Seite im Handbuch**. Wo der Inspektor sofort speichert, statt auf **Speichern** zu warten, sagt er es selbst.

Ein- und ausblenden lässt er sich mit dem Knopf ganz rechts in der Symbolleiste, über **Darstellung ▸ Inspektor einblenden** oder mit ^⌘I. Das gilt für das ganze Fenster, nicht nur für eine Seite.

Die Symbolleiste

Rechts in der Symbolleiste stehen immer in derselben Reihenfolge:

1. die **Aktionen der Seite** als Symbole, etwa ➕ zum Anlegen. Höchstens eine

   Aktion je Seite ist farbig hervorgehoben, die, um die es auf der Seite meist geht, etwa **Starten** auf Vorschau & Build;
2. **Speichern**, auf Seiten, die eine Datei bearbeiten;
3. der Knopf für den **Inspektor**.

Was für das ganze Projekt gilt, steht zusätzlich im Menü **Projekt**: **Server starten** (⌘R), **Server stoppen** (⌘.), **Build ausführen** (⌘B) und **Projektordner im Finder zeigen**.

Speichern

Seiten, die eine Datei bearbeiten, etwa Konfiguration und Stile, schreiben erst mit **Speichern** (⌘S). Der Knopf ist nur aktiv, wenn es etwas zu speichern gibt. Ungespeicherte Änderungen zeigt außerdem der Punkt im roten Knopf oben links im Fenster.

Wer eine Seite mit ungespeicherten Änderungen verlässt, über die Seitenleiste, den Verlauf, einen Verweis, das Schließen des Fensters oder das Beenden, wird gefragt: **Speichern**, **Änderungen verwerfen** oder **Abbrechen**. Scheitert das Speichern, bleibt man auf der Seite. Nichts geht still verloren.

Hinweise und Erklärungen

Drei Arten Text begegnen einem neben den Bedienelementen:

- **Ein Kasten** mit Symbol sagt etwas über den Zustand einer Sache: in der Akzentfarbe eine Information, orange eine Warnung, rot ein Fehler. Sein Text lässt sich markieren und kopieren, etwa ein Pfad in einer Fehlermeldung.
- **Eine rote Zeile direkt unter einem Feld** sagt, was an der eben getippten Eingabe nicht stimmt.
- **Grauer Text** erklärt, was ein Feld tut.

Klappen und Ziehen

Klappgruppen öffnen und schließen sich mit einem Klick irgendwo in ihre Kopfzeile, nicht nur auf den Pfeil.

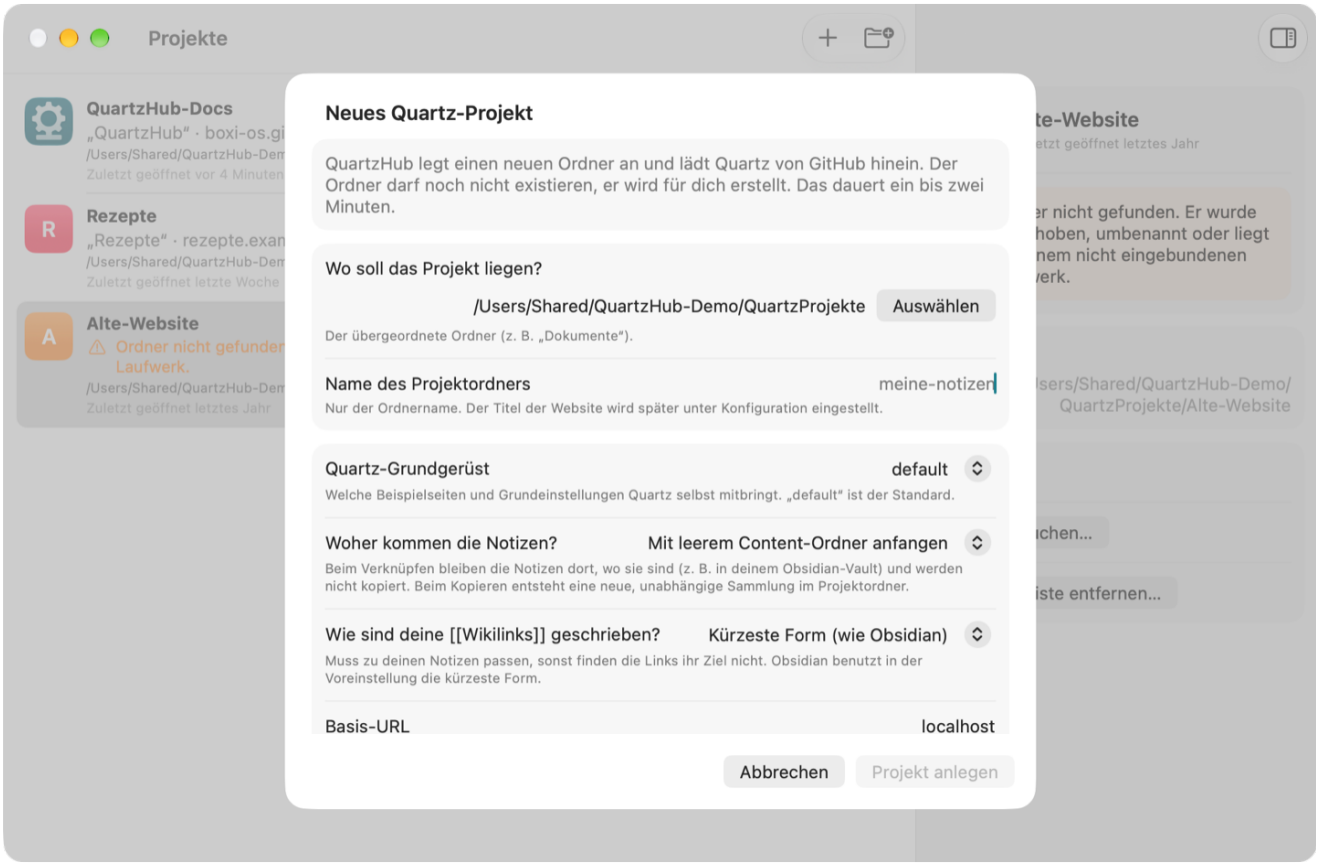
Wo eine Seite in **Bereiche übereinander** geteilt ist, etwa eine Liste und darunter die Vorschau der Website, steht zwischen ihnen ein **Griff**. Ziehen ändert die Aufteilung, ein Doppelklick stellt sie zurück, und das Fenster merkt sie sich.

1.4 – Die erste Website

Diese Seite führt einmal durch alles, was es für eine erste Website braucht. Jeder Schritt hat später ein eigenes Kapitel; hier geht es nur darum, einmal durchzukommen.

1. Ein Projekt anlegen

Im Fenster **Projekte** auf **Neues Projekt erstellen** klicken, oder **Ablage ▸ Neues Projekt…** (⌘N) aus jedem Fenster.



Der Dialog „Neues Quartz-Projekt“.

- **Wo soll das Projekt liegen?** Der Ordner, in dem der Projektordner entsteht, etwa „Dokumente“.
- **Name des Projektordners**, etwa `meine-notizen`. Das ist nur der Ordnername; den Titel der Website stellt man später ein.
- **Basis-URL**: die Adresse, unter der die Website einmal erreichbar sein soll, ohne `https://`. Wer sie noch nicht kennt, lässt `localhost` stehen.
- **Woher kommen die Notizen?** Für den ersten Versuch **Mit leerem Content-Ordner anfangen**. Wer einen Obsidian-Vault hat, wählt **Auf vorhandene Notizen verweisen**: Dann bleiben die Notizen, wo sie sind, und QuartzHub liest sie dort (4.2).
- **Basis-Template mitinstallieren** gibt der Website eine fertige Gestaltung: Farben

für hell und dunkel, eigene Seitenraster, Schriften. **Mit Beispielseiten** legt dazu einige Seiten an, die zeigen, wie das aussieht, bevor eigene Notizen da sind.

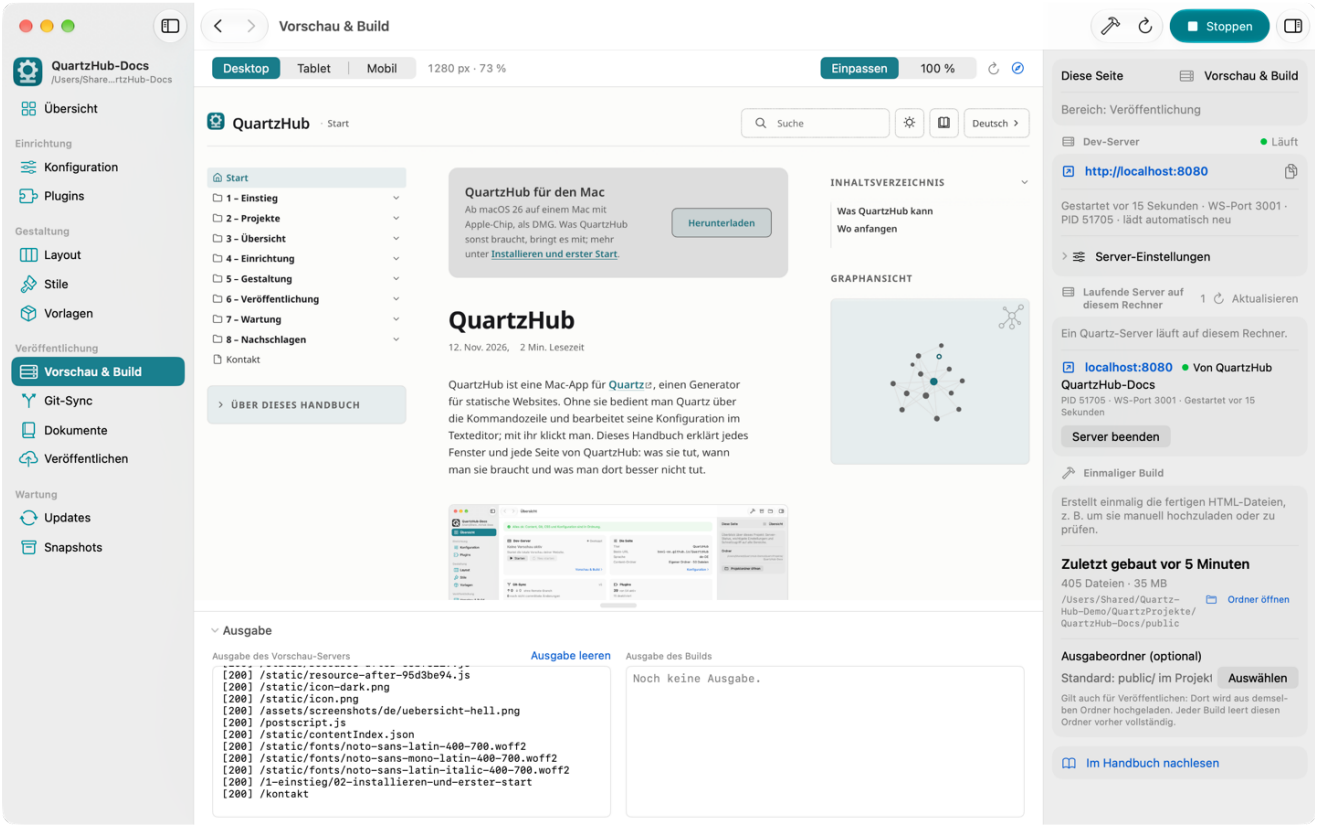
Projekt anlegen lädt Quartz von GitHub in den neuen Ordner und installiert, was es braucht. Das dauert ein bis zwei Minuten. Danach öffnet sich das Fenster des neuen Projekts.

2. Die Übersicht lesen

Das Projektfenster öffnet sich auf der **Übersicht**. Ihre Karten sagen, wie es dem Projekt geht: ob der Dev-Server läuft, wie die Website heißt, ob Plugins aktiv sind, wann zuletzt gebaut wurde. Steht oben „Alles ok“, ist nichts zu tun (3.1).

3. Die Vorschau starten

Auf **Vorschau & Build** in der Seitenleiste, dann **Starten** in der Symbolleiste (⌘R). Quartz baut die Website und hält sie als Vorschau bereit; nach einigen Sekunden erscheint sie in der Mitte des Fensters.



Die Vorschau im Fenster. Oben die Breite (Desktop, Tablet, Mobil), unten die Ausgabe von Quartz.

Die Vorschau baut bei jeder Änderung neu, an den Notizen wie an den Einstellungen. Man kann also nebenher in Obsidian schreiben und zusehen (6.1).

4. Titel und Aussehen

- Unter **Konfiguration ▸ Website** stehen **Seitentitel** und **Basis-URL**; **Speichern** (⌘S) schreibt sie (4.1).
- Unter **Stile ▸ Basis** stellt man Farben und Schriften ein (5.5), unter **Layout**, was wo auf der Seite steht (5.2).

5. Veröffentlichen

Gefällt die Vorschau, kommt die Website ins Netz. **Veröffentlichen** kennt mehrere Wege: GitHub Pages, ein Webspace über SFTP oder FTP, ein Ordner, ein Webhook. Welcher passt, beschreibt 6.7.

Vor Änderungen an Konfiguration, Plugins, Frames und Stilen und vor jedem Update legt QuartzHub von selbst einen Snapshot an, für die Konfiguration höchstens einen alle 15 Minuten. Geht beim Einrichten etwas schief, führt er zum Stand davor zurück (7.2).

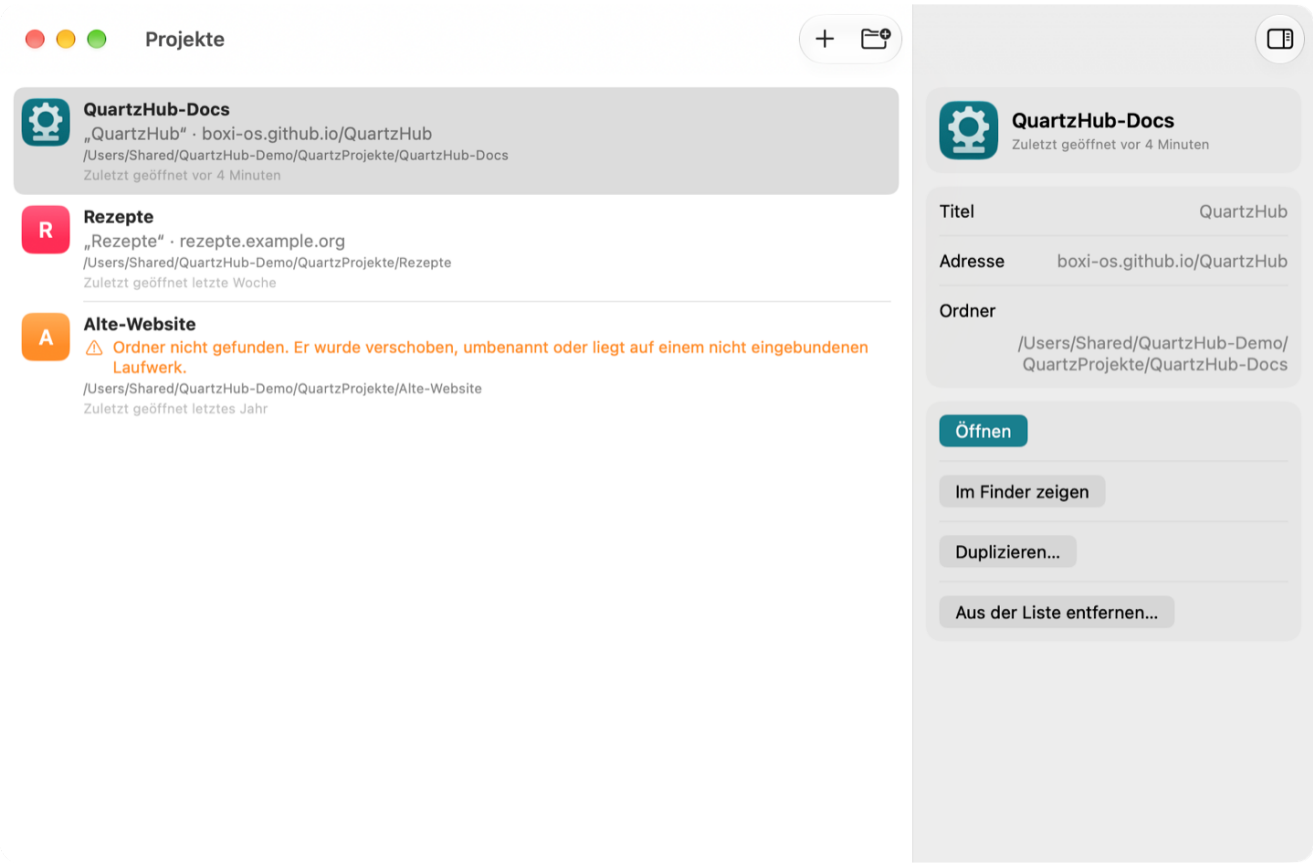
2 – Projekte

Ein Projekt ist ein Ordner mit einer Quartz-Website: Konfiguration, Stylesheets, Plugins und der Content-Ordner mit den Notizen. QuartzHub führt eine Liste dieser Ordner; die Ordner selbst liegen, wo du willst.

1. **Das Fenster „Projekte“**: die Liste, was ihre Zeilen sagen und was zu tun ist, wenn ein Ordner fehlt.
2. **Ein neues Projekt anlegen**: jede Frage des Dialogs und was danach im Ordner liegt.
3. **Ein vorhandenes Projekt öffnen**: ein geklontes Repository oder ein von Hand eingerichtetes Quartz.
4. **Ein Projekt duplizieren**: was mitkommt, was bleibt und warum.
5. **Die Einstellungen**: Aussehen, Sprache, Projektordner, laufende Server beim Beenden, Laufzeit.

2.1 – Das Fenster „Projekte“

Das Fenster **Projekte** öffnet sich beim Start. Aus jedem anderen Fenster kommt man mit **Fenster ▸ Projekte** (⇧⌘0) dorthin zurück.



Drei Projekte, das oberste ausgewählt. Rechts der Inspektor mit seinen Angaben und Aktionen.

Die Zeilen

Jede Zeile ist ein Projekt:

- **Bild und Name.** Der Name ist der Name des Projektordners. Das Bild ist das Projektbild (4.3); ohne eines steht dort der erste Buchstabe auf einer Farbe.
- **Titel und Adresse** der Website, wie sie in der Konfiguration stehen.
- **Der Ordner** auf der Platte. Ist der Pfad zu lang, zeigt die Zeile die Mitte gekürzt; der ganze steht im Tooltip und im Inspektor.
- **Wann das Projekt zuletzt geöffnet wurde.**
- **„Dev-Server auf Port …“**, solange die Vorschau dieses Projekts läuft.

Oben steht, was zuletzt geöffnet wurde, Projekte ohne dieses Datum nach Namen. Ab sechs Projekten erscheint in der Symbolleiste ein Suchfeld; es sucht in Name, Ordner und Titel.

Die Liste liest QuartzHub neu, sobald es wieder in den Vordergrund kommt. Wer

im Finder einen Projektordner umbenennt, sieht das beim Zurückwechseln.

Öffnen

Ein Doppelklick auf eine Zeile öffnet das Projekt in einem eigenen Fenster, ebenso **Öffnen** im Inspektor oder die Eingabetaste. Ist das Projekt schon offen, kommt sein Fenster nach vorn.

Der Inspektor

Ist eine Zeile ausgewählt, zeigt der Inspektor Titel, Adresse und den ganzen Ordnerpfad und darunter die Aktionen:

- **Öffnen**
- **Im Finder zeigen**
- **Duplizieren…** (2.4)
- **Aus der Liste entfernen…**

Dieselben Befehle stehen im Kontextmenü einer Zeile.

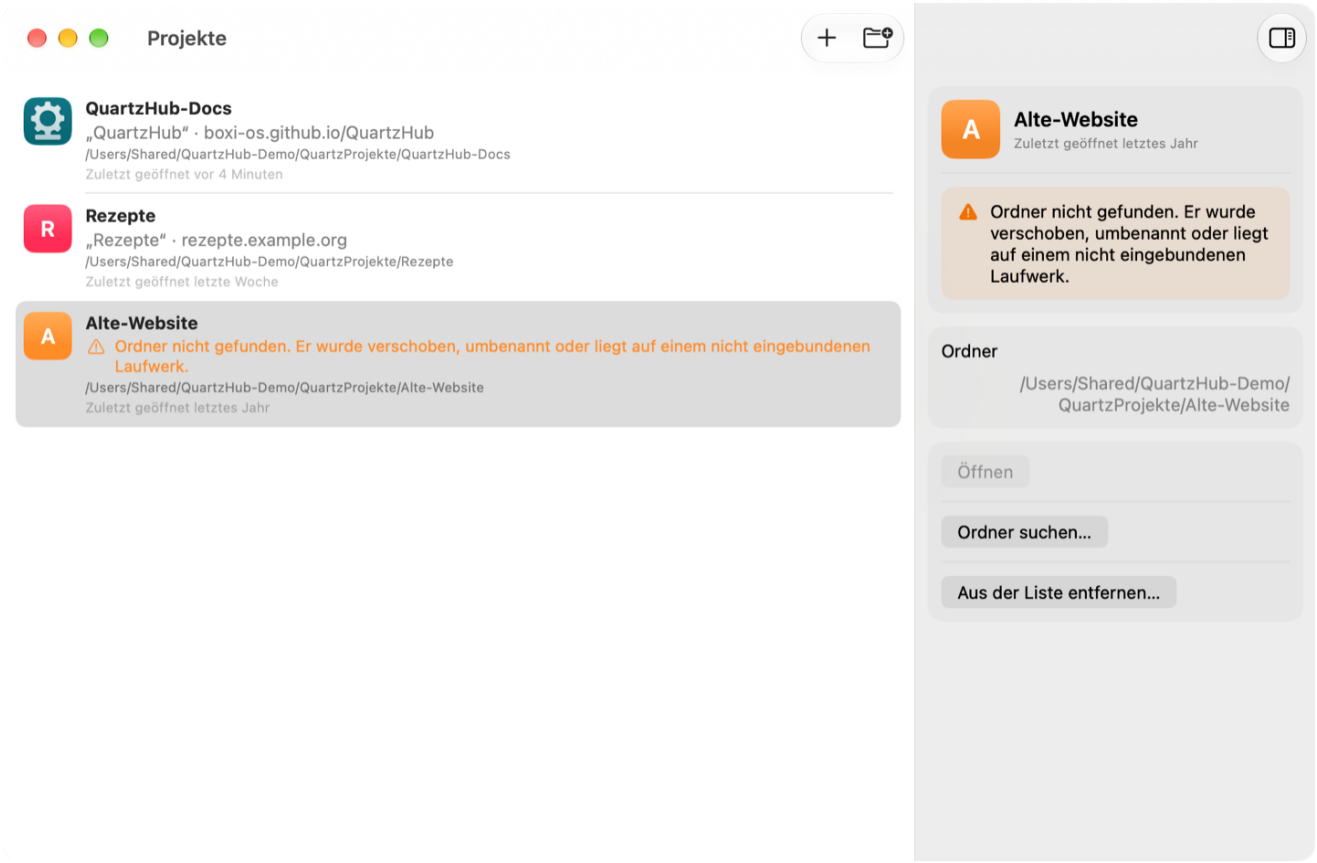
Ohne Auswahl sagt der Inspektor, ob die Werkzeuge bereit sind (1.2), erklärt kurz, was Quartz ist, verweist auf dieses Handbuch und zeigt, was man in einem Projekt tun kann.

Gibt es auf GitHub eine neuere Fassung von QuartzHub, sagt ein Band über der Liste, welche, und **Herunterladen** öffnet ihre Seite. Installiert wird nichts von selbst.

Neu und vorhanden

Die beiden Knöpfe in der Symbolleiste legen ein Projekt an (➕, 2.2) oder nehmen einen Ordner auf, in dem Quartz schon eingerichtet ist (2.3). Dasselbe geht aus jedem Fenster mit **Ablage ▸ Neues Projekt…** (⌘N) und **Ablage ▸ Vorhandenes Projekt öffnen…** (⌘O).

Wenn ein Ordner fehlt



Ein Projekt, dessen Ordner nicht mehr da ist.

Eine Zeile in Orange sagt, dass mit dem Ordner etwas nicht stimmt:

- **Ordner nicht gefunden.** Der Ordner wurde verschoben, umbenannt oder liegt auf einem Laufwerk, das gerade nicht eingebunden ist. Für ein Laufwerk reicht es, es anzuschließen. Sonst zeigt **Ordner suchen…** im Inspektor QuartzHub den neuen Ort. Der Eintrag bleibt derselbe und heißt danach wie der neue Ordner.
- **Kein Quartz-Projekt.** Im Ordner liegt keine `quartz.config.yaml`. So ein Eintrag lässt sich nicht öffnen; er entsteht, wenn man über „Vorhandenes Projekt öffnen“ einen falschen Ordner wählt.

Aus der Liste entfernen

Aus der Liste entfernen… fragt einmal nach und nimmt das Projekt dann aus der Liste. **Der Ordner auf der Platte bleibt unangetastet**; wer ihn wieder öffnet, hat das Projekt zurück.

Zwei Dinge gehen dabei verloren, und die Frage nennt sie, wenn sie zutreffen:

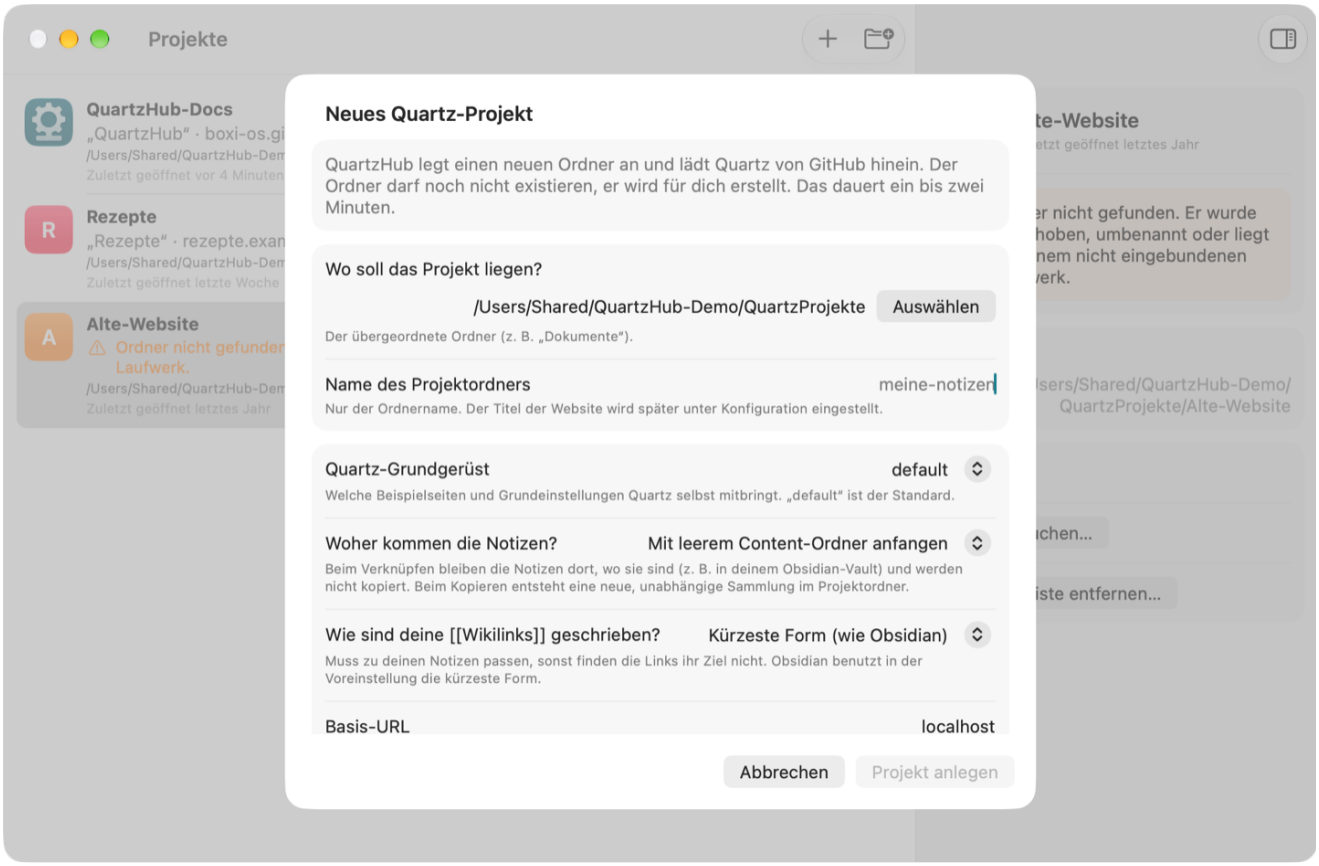
- **Ein laufender Dev-Server** dieses Projekts wird beendet.
- **Die Empfänger-Adressen und Captcha-Secrets** des Formular-Relays (4.9) löscht QuartzHub, weil sie zum Listeneintrag gehören und nicht im Projektordner liegen. Wer das Projekt wieder aufnimmt, trägt sie neu ein.

Wenn Werkzeuge fehlen

Antworten `node`, `npm` oder `git` nicht, steht über der Liste ein Hinweis in Orange: welches Werkzeug fehlt oder sich nicht ausführen lässt und was ohne es nicht geht. Node und npm gehören zu QuartzHub; fehlen sie, ist diese Installation unvollständig, und am schnellsten hilft, QuartzHub neu zu installieren. **Erneut prüfen** fragt die Werkzeuge noch einmal, etwa nachdem man git installiert hat.

2.2 – Ein neues Projekt anlegen

Neues Projekt erstellen im Fenster **Projekte**, ➕ in dessen Symbolleiste oder **Ablage ▸ Neues Projekt…** (⌘N) öffnen den Dialog „Neues Quartz-Projekt“. 1.4 geht ihn für einen ersten Versuch kurz durch; hier steht jede Frage.



Der Dialog mit den Voreinstellungen.

Ort und Name

- **Wo soll das Projekt liegen?** Der Ordner, in dem der Projektordner entsteht. Voreingestellt ist das Standard-Projektverzeichnis aus den Einstellungen (2.5).
- **Name des Projektordners.** Nur der Ordnername; er erscheint auch als Name in der Projektliste. Er darf kein „/“ enthalten und nicht „.“ oder „..“ sein. Den Titel der Website stellt man später unter **Konfiguration ▸ Website** ein.

Darunter steht, welcher Ordner angelegt wird. **Diesen Ordner darf es noch nicht geben**; QuartzHub legt ihn selbst an.

Quartz-Grundgerüst

Quartz bringt einige Grundgerüste mit: Beispielseiten und Grundeinstellungen für einen

bestimmten Zweck. **default** ist der Standard. Die anderen (**obsidian**, **ttrpg**, **blog**, **docs**) stellen Quartz für einen solchen Zweck ein; was sie genau tun, beschreibt die **Quartz-Dokumentation**. Das Grundgerüst ist etwas anderes als das Basis-Template weiter unten.

Woher kommen die Notizen?

- **Mit leerem Content-Ordner anfangen.** Der Projektordner bekommt einen eigenen, leeren Content-Ordner.
- **Vorhandene Notizen kopieren.** Die Notizen aus dem **Quellordner** werden einmal in den Content-Ordner des Projekts kopiert. Danach sind es zwei getrennte Sammlungen: Was man im Quellordner ändert, kommt nicht mehr an.
- **Auf vorhandene Notizen verweisen (symbolischer Link).** Der Content-Ordner wird ein Link auf den Quellordner, etwa einen Obsidian-Vault. Die Notizen bleiben, wo sie sind, und jede Änderung dort ist sofort in der Vorschau. QuartzHub schreibt nichts in den Vault.

Zwischen Ordner und Link lässt sich später wechseln, ohne eine Notiz zu verlieren (4.2).

Wie sind deine `[[Wikilinks]]` geschrieben?

Quartz muss wissen, wie ein Link wie `[[Seite]]` gemeint ist:

- **Kürzeste Form (wie Obsidian)**: nur der Name, solange er eindeutig ist. Das ist Obsidians Voreinstellung.
- **Absolut (vom Wurzelordner aus)**: der Pfad ab dem Content-Ordner, etwa `[[Ordner/Seite]]`.
- **Relativ (von der aktuellen Notiz aus)**.

Die Einstellung muss zu den Notizen passen, sonst finden die Links ihr Ziel nicht. In Obsidian ist das die Einstellung für das Format neuer Links.

Basis-URL

Die Adresse, unter der die Website einmal erreichbar sein soll, ohne `https://`, etwa `notizen.example.org` oder `name.github.io/projekt`. Quartz baut daraus Links, den RSS-Feed und Vorschaubilder. Wer sie noch nicht kennt, lässt `localhost` stehen und trägt sie später unter **Konfiguration ▸ Website** nach.

Basis-Template

Basis-Template mitinstallieren gibt der Website eine fertige Gestaltung: Farben für hell und dunkel, eigene Seitenraster (**Frames**), Schriften, die die Website selbst ausliefert, und jede Komponente einzeln gestaltet. Dazu kommt das Plugin quartz-forms mit einem Kontaktformular, das noch nichts sendet (4.9).

Mit Beispielseiten legt dazu einige Seiten an, die zeigen, wie die Website aussieht, bevor eigene Notizen da sind; der Dialog nennt ihre Zahl. Das geht nur mit einem leeren Content-Ordner: Beim Kopieren würden die Beispielseiten gleichnamige Notizen überschreiben, und in einen verlinkten Vault schreibt QuartzHub nie.

QuartzHub lädt das Basis-Template von GitHub, hebt es auf und fragt nach einem Tag wieder nach. Konnte es nicht geladen werden, sagt der Dialog das und bietet **Erneut versuchen**; ohne Template entsteht ein Projekt mit Quartz' eigener Grundeinstellung. Das Template lässt sich später auch noch anwenden (5.9).

Was beim Anlegen passiert

Projekt anlegen braucht ein bis zwei Minuten. Unten im Dialog steht so lange „Erstelle…". In dieser Zeit:

1. lädt QuartzHub Quartz von GitHub in den neuen Ordner, als git-Repository mit seiner ganzen Geschichte. Die Verbindung zu Quartz' Repository wird danach gelöst, damit ein späteres Hochladen nie dort landet; Updates von Quartz kommen trotzdem (7.1);
2. installiert es die npm-Pakete, die Quartz braucht;
3. richtet es Quartz mit den Antworten aus dem Dialog ein;
4. baut es die Website einmal, damit Plugins, die beim ersten Bau etwas nachinstallieren, beim nächsten Mal bereit sind;
5. wendet es das Basis-Template an, wenn es gewählt ist. Die Sprache der Website wird dann die von QuartzHub.

Scheitert einer der ersten drei Schritte, bleibt der Dialog offen und nennt, was nicht ging; in der Projektliste steht dann nichts. Danach ist das Projekt da: Es steht in der Liste, und sein Fenster öffnet sich. Ging beim Template etwas schief oder gibt es dazu etwas zu melden, sagt der Dialog das vorher und führt mit **Zum Projekt** weiter. Das Projekt ist auch dann benutzbar.

Was im Ordner liegt

`quartz.config.yaml`	die Konfiguration: Titel, Adresse, Plugins
`content/`	die Notizen, ein Ordner oder ein Link
`quartz/`	Quartz selbst, mit den Stylesheets unter `quartz/styles/`
`node_modules/`	die npm-Pakete; groß, aber jederzeit neu installierbar
`.quartz-gui/`	was QuartzHub zum Projekt speichert: Frames, Snapshots, Ziele, Bücher
`public/`	die gebaute Website, nach jedem Build neu

Alles außer `.quartz-gui/` ist ein gewöhnliches Quartz-Projekt. Was dort liegt,

beschreibt 8.2.

2.3 – Ein vorhandenes Projekt öffnen

Vorhandenes Projekt öffnen nimmt einen Ordner, in dem Quartz schon eingerichtet ist, in die Projektliste auf. Der Befehl steht im Fenster **Projekte** (Ordner-Symbol in der Symbolleiste), unter den ersten Schritten des leeren Fensters und im Menü **Ablage ▸ Vorhandenes Projekt öffnen…** (⌘O). Der Ordner-Dialog beginnt im Standard-Projektverzeichnis (2.5).

Welcher Ordner

Der Ordner, in dem `quartz.config.yaml` liegt, nicht ein Ordner darüber oder darunter. Er wird aufgenommen und gleich geöffnet; das Projekt heißt in der Liste wie der Ordner.

Liegt dort keine `quartz.config.yaml`, nimmt QuartzHub den Ordner zwar in die Liste auf, öffnet ihn aber nicht, und die Zeile sagt **Kein Quartz-Projekt**. So ein Eintrag lässt sich mit **Aus der Liste entfernen…** wieder loswerden; am Ordner ändert das nichts.

Steht der Ordner schon in der Liste, entsteht kein zweiter Eintrag; das vorhandene Projekt öffnet sich.

Woher ein Projekt kommen kann

Aus einem git-Repository. Ein Projekt, das man auf einem anderen Rechner angelegt und mit **Git-Sync** (6.5) hochgeladen hat, holt man mit `git clone` und öffnet es dann hier. Einem frischen Klon fehlen die npm-Pakete, denn `node_modules/` reist nicht mit; die **Übersicht** sagt das und holt sie mit **Installieren** nach (3.1). Ebenso fehlen die heruntergeladenen Plugins; die stellt **Plugins ▸ Installiert** aus `quartz.lock.json` wieder her (4.5).

Von Hand eingerichtet. Ein Quartz, das man nach der Anleitung von Quartz selbst geklont und eingerichtet hat, ist für QuartzHub ein Projekt wie jedes andere. QuartzHub ändert beim Öffnen nichts daran. Erst wenn man etwas einstellt, schreibt es, und dann in genau die Datei, die man auch von Hand geändert hätte.

Ein Content-Ordner, der ein Link ist

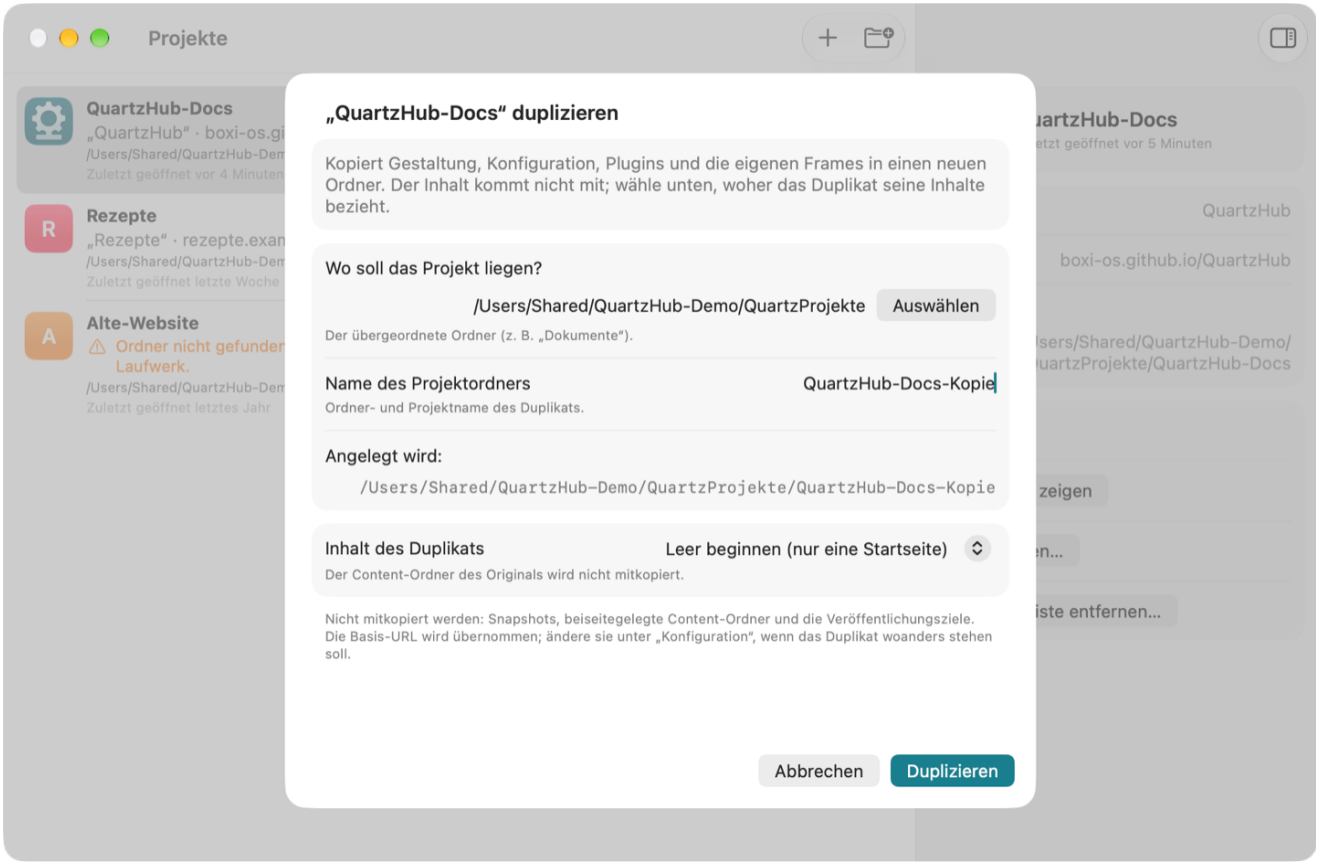
Ist `content/` ein Link auf einen Vault, folgt QuartzHub ihm. Liegt der Vault auf einem Laufwerk, das nicht eingebunden ist, oder wurde er verschoben, sagt das die Übersicht; der Content-Ordner lässt sich unter **Konfiguration ▸ Website** neu verknüpfen (4.2).

2.4 – Ein Projekt duplizieren

Ein Duplikat ist ein neues Projekt mit der Gestaltung eines vorhandenen. Wozu das gut ist:

- **eine zweite Website im selben Aussehen**, etwa für ein anderes Thema oder eine andere Gruppe;
- **ein Versuch ohne Risiko**: ein Update, ein neues Plugin, ein anderes Layout ausprobieren, ohne das Original anzufassen.

Duplizieren… steht im Inspektor des Fensters **Projekte**, wenn ein Projekt ausgewählt ist, und im Kontextmenü seiner Zeile.



Der Dialog „Duplizieren“. Unten steht, was nicht mitkopiert wird.

Ort, Name und Inhalt

- **Wo soll das Projekt liegen?** und **Name des Projektordners** wie beim Anlegen (2.2). Den Zielordner darf es noch nicht geben, und er darf nicht im Original liegen und das Original nicht in ihm.
- **Inhalt des Duplikats.** Der Content-Ordner des Originals kommt nie mit. Zur Wahl stehen:
  - **Leer beginnen (nur eine Startseite)**: ein Content-Ordner mit einer `index.md`,

die den Namen des Projekts trägt und sagt, wo man Notizen verknüpft; ohne sie zeigte die Website eine 404-Seite;
- **Mit einem Ordner verknüpfen (z. B. einem Obsidian-Vault)**: der Content-Ordner wird ein Link, wie beim Anlegen;
- **Einen Ordner hineinkopieren**: die Notizen aus dem Ordner werden einmal übernommen.

Was mitkommt und was bleibt

Kommt mit	Bleibt beim Original
Quartz, die Konfiguration samt Basis-URL, die Stylesheets	der Content-Ordner
die installierten Plugins	die Snapshots
die eigenen Frames, Theme-Voreinstellungen und Breakpoints	beiseitegelegte Content-Ordner
das git-Repository mit seiner Geschichte	die Ziele zum Veröffentlichen und was QuartzHub über frühere Uploads weiß

Die Ziele bleiben aus gutem Grund zurück. Ein Duplikat, das das Ziel des Originals geerbt hätte, würde beim ersten **Jetzt veröffentlichen** dessen Website überschreiben. Das Duplikat bekommt seine eigenen Ziele (6.7). Die Zugänge, also Passwörter und Tokens, gelten ohnehin für alle Projekte.

Die Basis-URL kommt mit. Soll das Duplikat unter einer anderen Adresse stehen, ändert man sie unter **Konfiguration ▸ Website** (4.1).

Das git-Repository kommt ohne seine Verbindung nach außen mit. Die Geschichte bleibt, damit Updates von Quartz weiter zusammengeführt werden können; ein `git push` des Duplikats landet aber nicht im Repository des Originals. Für Git-Sync verbindet man es mit einem eigenen (6.5).

Was beim Duplizieren passiert

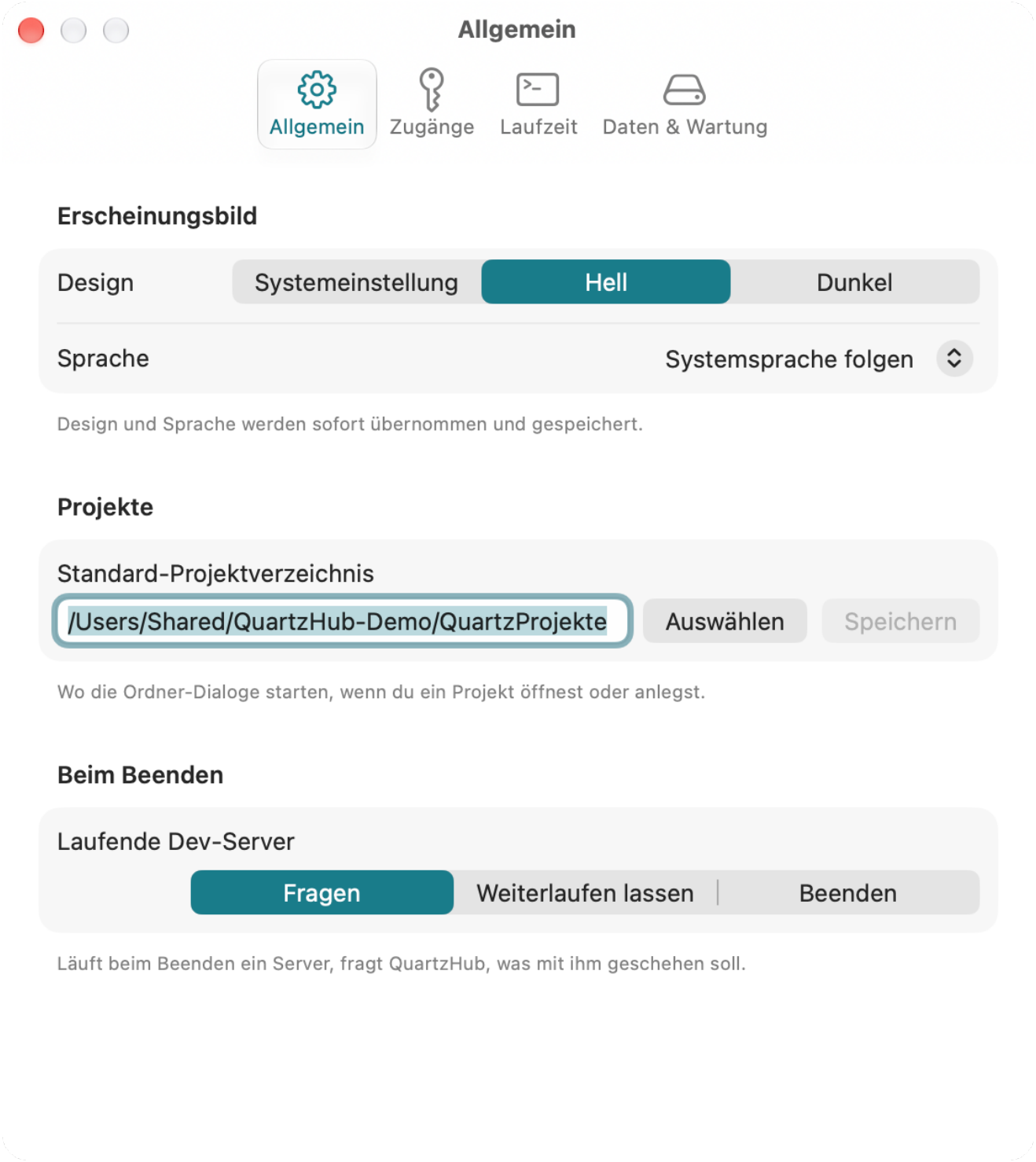
Unten im Dialog steht „Kopiere…“. QuartzHub kopiert, was mitkommt, legt den Inhalt an und installiert die npm-Pakete neu, statt `node_modules/` zu kopieren; das dauert etwas. Danach steht das Duplikat in der Liste, und sein Fenster öffnet sich. Die Website baut es bei der ersten Vorschau.

Scheitert etwas, bleibt der Dialog offen und nennt den Grund.

2.5 – Die Einstellungen

Die Einstellungen gelten für QuartzHub als Ganzes, nicht für ein Projekt. Sie öffnen sich mit **QuartzHub ▸ Einstellungen…** (⌘,) in einem eigenen Fenster mit vier Reitern. Was man dort ändert, gilt sofort; einen Knopf zum Speichern haben nur das Standard-Projektverzeichnis und die Zugänge.

Allgemein



Erscheinungsbild

- **Design**: **Systemeinstellung**, **Hell** oder **Dunkel**. Mit „Systemeinstellung“ folgt QuartzHub dem Mac.
- **Sprache**: **Systemsprache folgen**, **Deutsch** oder **English**. Die Sprache gilt für QuartzHub selbst, nicht für die Websites; deren Sprache steht in ihrer Konfiguration (4.1).

Projekte

- **Standard-Projektverzeichnis**: wo die Ordner-Dialoge beginnen, wenn man ein Projekt anlegt, öffnet oder dupliziert. **Auswählen** sucht den Ordner aus und speichert ihn gleich; wer den Pfad tippt, speichert mit **Speichern** oder der Eingabetaste.

Beim Beenden

Laufende Dev-Server sagt, was mit Vorschauen geschieht, die beim Beenden von QuartzHub noch laufen:

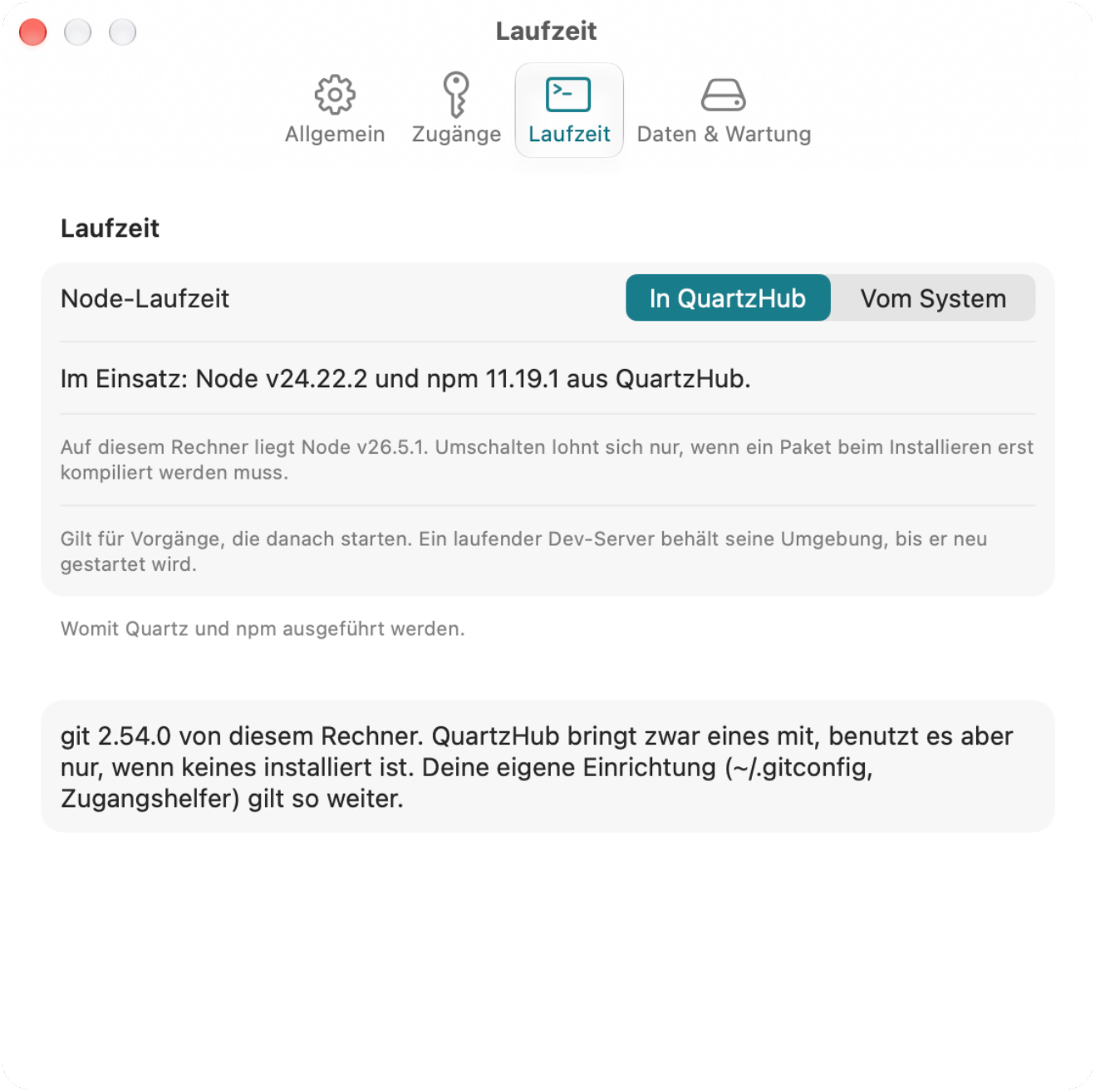
- **Fragen**: QuartzHub fragt jedes Mal. Die Frage bietet **Abbrechen**, **Weiterlaufen lassen** und **Server beenden**; mit **Nicht mehr fragen** wird die gewählte Antwort zur Einstellung.
- **Weiterlaufen lassen**: Die Server laufen weiter und bleiben im Browser erreichbar. Beim nächsten Start findet QuartzHub sie wieder und fragt, ob sie weiterlaufen oder beendet werden sollen.
- **Beenden**: Die Server werden ohne Rückfrage gestoppt.

Unabhängig davon fragt QuartzHub beim Beenden immer, wenn gerade ein Update, das Wiederherstellen eines Snapshots oder ein Veröffentlichen läuft: Ein Abbruch mittendrin kann das Projekt halb geändert zurücklassen.

Zugänge

Hier liegen der persönliche Zugriffstoken für GitHub und die Zugänge zu Servern (SFTP, FTP) und Webhooks. Sie gelten für alle Projekte; welches Projekt wohin veröffentlicht, legt man im Projekt unter **Veröffentlichen** fest. Beides beschreibt 6.7.

Laufzeit



Node-Laufzeit sagt, womit Quartz und npm laufen:

- **In QuartzHub** (Voreinstellung): das Node, das QuartzHub mitbringt. Es ist auf jedem Mac dasselbe und passt zu Quartz.
- **Vom System**: das Node, das auf dem Mac installiert ist. Das lohnt sich nur, wenn ein Paket beim Installieren erst kompiliert werden muss und dafür ein bestimmtes Node braucht. Quartz verlangt Node 22 oder neuer. Werkzeuge wie nvm, fnm, asdf oder mise wählen das Node je Ordner, die Laufzeit kann dann also je Projekt eine andere sein. Findet QuartzHub kein Node auf dem Mac, sagt es das vorher.

Eine Umstellung gilt für alles, was danach startet. Eine laufende Vorschau behält ihr Node, bis sie neu gestartet wird.

Darunter steht, welches **git** QuartzHub benutzt. Ist auf dem Mac eines installiert, nimmt QuartzHub dieses, damit die eigene Einrichtung (`~/.gitconfig`, Zugangshelfer) weiter gilt; sonst das mitgebrachte. Einen Schalter dafür gibt es nicht.

Daten & Wartung

- **Theme-Dokumentation**: Beschriftungen und Beschreibungen der Optionen von Community-Themes (5.6), einmal von GitHub geholt und aufbewahrt. **Cache leeren** lässt QuartzHub sie beim nächsten Mal neu laden, etwa wenn ein Theme seine Optionen geändert hat.
- **Speicherort**: der Ordner, in dem QuartzHub Projektliste, Einstellungen und Zugänge ablegt, mit **Im Finder zeigen**. Normalerweise ist das `~/Library/Application Support/QuartzHub`. Die Passwörter und Tokens der Zugänge liegen nicht dort, sondern im Schlüsselbund von macOS (8.2).

Konnte QuartzHub beim Start eine seiner eigenen Dateien nicht lesen, etwa weil es beim Speichern hart beendet wurde, steht das hier oben. Die Datei wurde dann zur Seite gelegt statt überschrieben; was darin stand, fehlt in QuartzHub und muss neu angelegt werden.

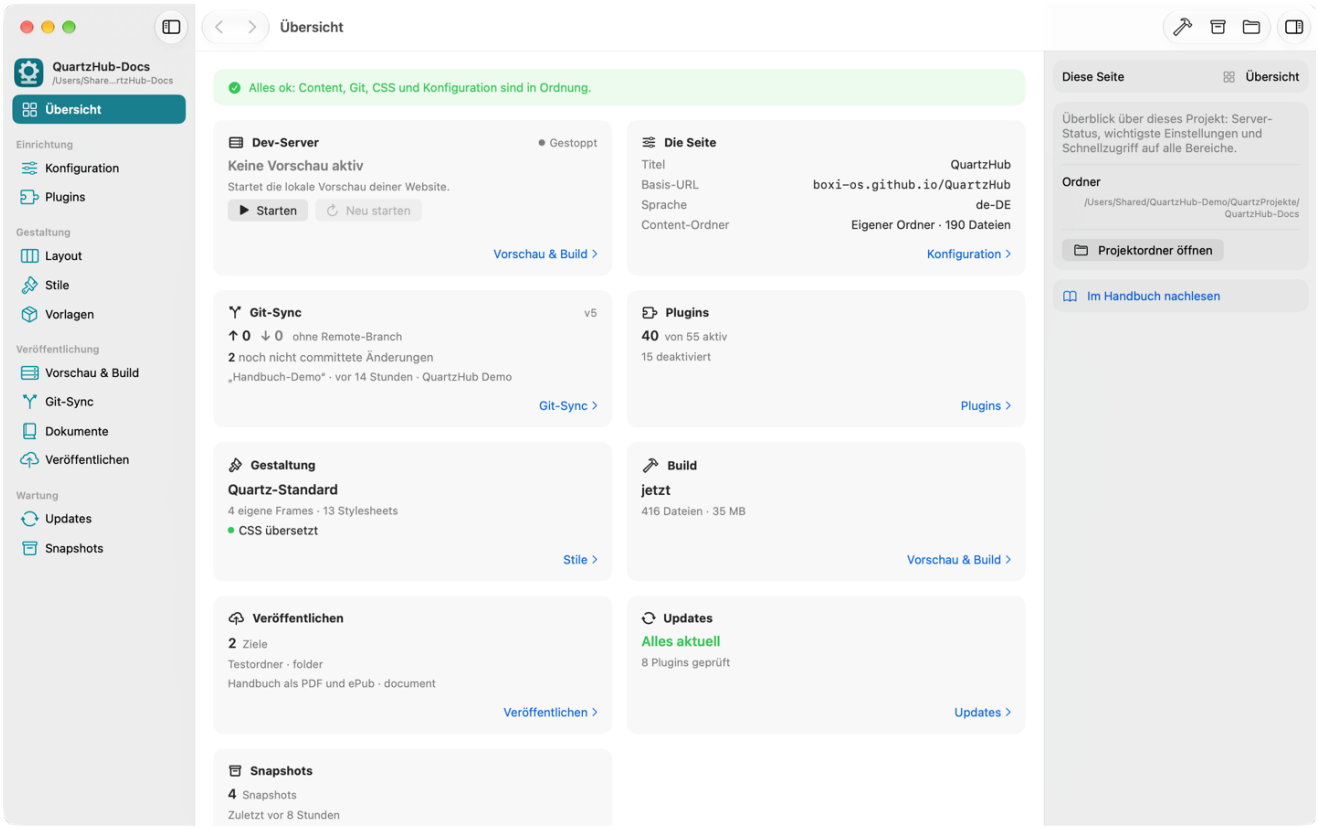
Ganz unten stehen die Version von QuartzHub und von macOS, für eine Fehlermeldung.

3 – Übersicht

Jedes Projektfenster öffnet sich auf der **Übersicht**. Sie beantwortet zwei Fragen: Muss ich etwas tun? Und wie steht das Projekt gerade da? Für alles Weitere führt sie auf die Seite, die dafür zuständig ist.

1. **Die Übersicht**: das Band „Handlungsbedarf“, die neun Karten und die drei Aktionen in der Symbolleiste.

3.1 – Die Übersicht



Die Übersicht eines Projekts ohne Handlungsbedarf, mit einem frischen Build.

Oben: Handlungsbedarf oder „Alles ok"

Über den Karten steht eines von beiden:

- **Handlungsbedarf** in Orange, mit der Zahl der Punkte. Jeder Punkt sagt, was nicht stimmt, meist auch warum, und führt mit einem Verweis auf die Seite, wo es sich beheben lässt.
- **„Alles ok: Content, Git, CSS und Konfiguration sind in Ordnung."** in Grün.

Steht keines von beiden da, ist eine Prüfung noch nicht fertig oder konnte nicht laufen, etwa weil sich das CSS nicht prüfen ließ. Kein „Alles ok" heißt dann nicht, dass etwas kaputt ist, sondern nur, dass QuartzHub es nicht weiß (1.2).

Diese Punkte kann das Band nennen:

Punkt	Was dahintersteckt
Es gibt keinen Content-Ordner	Im Projekt fehlt `content/`; ohne Notizen gibt es keine Website.

Der Content-Ordner zeigt ins Leere	`content/` ist ein Link, und sein Ziel ist weg: verschoben, umbenannt oder auf einem Laufwerk, das nicht eingebunden ist ([[4-einrichtung/02-der-content-ordner
Die Website hat keine Startseite	Im Content-Ordner fehlt die `index.md`; die Adresse der Website zeigt eine 404-Seite.
Keine Basis-URL gesetzt	Quartz baut daraus Links, den RSS-Feed und Vorschaubilder. Liefert das Plugin „Fonts“ die Schriften aus, scheitert der Build ohne sie ([[4-einrichtung/01-website
npm-Pakete fehlen	Ohne sie baut Quartz nicht, etwa bei einem frisch geklonten Projekt. **Installieren** steht direkt in der Zeile: Es führt `npm install` aus und holt die Plugins aus `quartz.lock.json`.
Ob alle npm-Pakete installiert sind, konnte nicht geprüft werden	Die `package.json` des Projekts ließ sich nicht lesen.
Eigene Frames fehlen	Eigene Frames liegen in `.quartz-gui/` und kommen nicht über git mit. Bis sie wieder da sind, vom anderen Rechner kopiert oder aus einem Snapshot geholt, baut Quartz mit dem Standard-Frame ([[5-gestaltung/04-layout-frames
Der Dev-Server ist mit einem Fehler beendet worden	Die Vorschau lief und ist abgestürzt; was Quartz dazu sagte, steht unter **Vorschau & Build** ([[6-veroeffentlichung/03-die-ausgabe-lesen
Dein CSS lässt sich nicht übersetzen	Ein Stylesheet hat einen Fehler, mit Datei und Zeile. Quartz würde daran scheitern ([[5-gestaltung/08-eigenes-css
Dateien im Konflikt	Ein Zusammenführen mit git hat Konflikte hinterlassen; bis sie gelöst sind, lässt sich nichts hochladen ([[6-veroeffentlichung/05-git-sync
Ein merge (oder rebase, cherry-pick, revert) ist angefangen und nicht abgeschlossen	git steckt mitten in einem Vorgang, der abgeschlossen oder abgebrochen werden muss.
Der Git-Stand konnte nicht gelesen werden	git hat auf die Frage nach dem Stand mit einem Fehler geantwortet.
Ein Quartz-Kern-Update ist nicht zu Ende gelaufen	Ein Update wurde abgebrochen oder `npm install` ist dabei gescheitert. Einmal **Update durchführen** löst es auf ([[7-wartung/01-updates

Die Karten

Unter dem Band stehen neun Karten, je nach Fensterbreite in einer bis drei Spalten. Jede beginnt mit der einen Zahl oder dem einen Namen, um den es geht, und endet mit einem Verweis auf die Seite, der das Thema gehört.

Dev-Server. Ob die Vorschau läuft. Läuft sie, steht ihre Adresse da; ein Klick öffnet sie im Browser. Darunter, seit wann sie läuft, dazu der WebSocket-Port, über den sie neu lädt, und die Prozessnummer (PID); dann die Knöpfe **Starten** oder **Stoppen** und **Neu starten**. Ist der Server mit einem Fehler beendet worden, steht hier, warum. → **Vorschau & Build** (6.1)

Die Seite. Titel, Basis-URL und Sprache der Website und der Content-Ordner: „Eigener Ordner“ mit der Zahl der Dateien, oder bei einem Link sein Ziel. „nicht gesetzt“ und „Ziel fehlt“ stehen farbig da. → **Konfiguration ▸ Website** (4.1)

Git-Sync. Oben rechts der Branch. Darunter, wie viele Commits noch hochzuladen (↑) und herunterzuladen (↓) sind, wie viele Änderungen noch nicht committet sind und der letzte Commit mit Zeit und Autor. „ohne Remote-Branch“ heißt: Das Projekt ist mit keinem Repository im Netz verbunden. Die Karte liest nur, was git auf dem Mac weiß; sie geht dafür nicht ins Netz. → **Git-Sync** (6.5)

Plugins. Wie viele Plugins aktiv sind und wie viele es insgesamt gibt. → **Plugins** (4.5)

Gestaltung. Das aktive Community-Theme oder „Quartz-Standard“, die Zahl der eigenen Frames und Stylesheets und ob das CSS sich übersetzen lässt: **CSS übersetzt**, **CSS-Fehler** oder **CSS ungeprüft**. → **Stile** (5.5)

Build. Wann zuletzt gebaut wurde, mit Zahl und Größe der Dateien. Die Karte schaut dafür in den Ausgabeordner; „Noch nie gebaut“ heißt, dass es ihn nicht gibt. → **Vorschau & Build** (6.2)

Veröffentlichen. Wie viele Ziele eingerichtet sind, die ersten drei mit Name und Art. → **Veröffentlichen** (6.7)

Updates. Ob es für Quartz oder die Plugins Neues gibt: **Kern-Update verfügbar**, **Update für Plugins verfügbar**, **Alles aktuell**, **Update nicht abgeschlossen** oder **Nicht prüfbar**. Darunter, wie viele Plugins geprüft wurden und wie viele Prüfungen fehlschlugen. Als einzige Karte fragt sie im Netz nach, bei GitHub; ohne Verbindung steht dort „Nicht prüfbar“, nicht „Alles aktuell“. → **Updates** (7.1)

Snapshots. Wie viele es gibt, wann der letzte entstand und warum, etwa „Vor Konfigurations-Änderung“. → **Snapshots** (7.2)

Die Symbolleiste

Rechts oben stehen drei Aktionen, die man von hier aus oft braucht:

- **Build ausführen** (Hammer): baut die Website einmal, wie **Projekt ▸ Build ausführen**

(⌘B). Ist er fertig, sagt die Karte „Build“ „Build fertig.“; scheitert er, steht dort, dass die Ausgabe unter **Vorschau & Build** zu lesen ist.
- **Snapshot anlegen**: hält den Stand des Projekts fest, bevor man etwas ausprobiert (7.2).
- **Projektordner öffnen**: zeigt den Projektordner im Finder. Der Knopf steht auch im Inspektor, unter dem vollen Pfad.

4 – Einrichtung

Die Gruppe **Einrichtung** in der Seitenleiste hat zwei Seiten. **Konfiguration** legt fest, was die Website ist: Titel, Adresse, Sprache, woher die Notizen kommen. **Plugins** legt fest, was Quartz mit den Notizen tut und was auf den Seiten erscheint.

Konfiguration

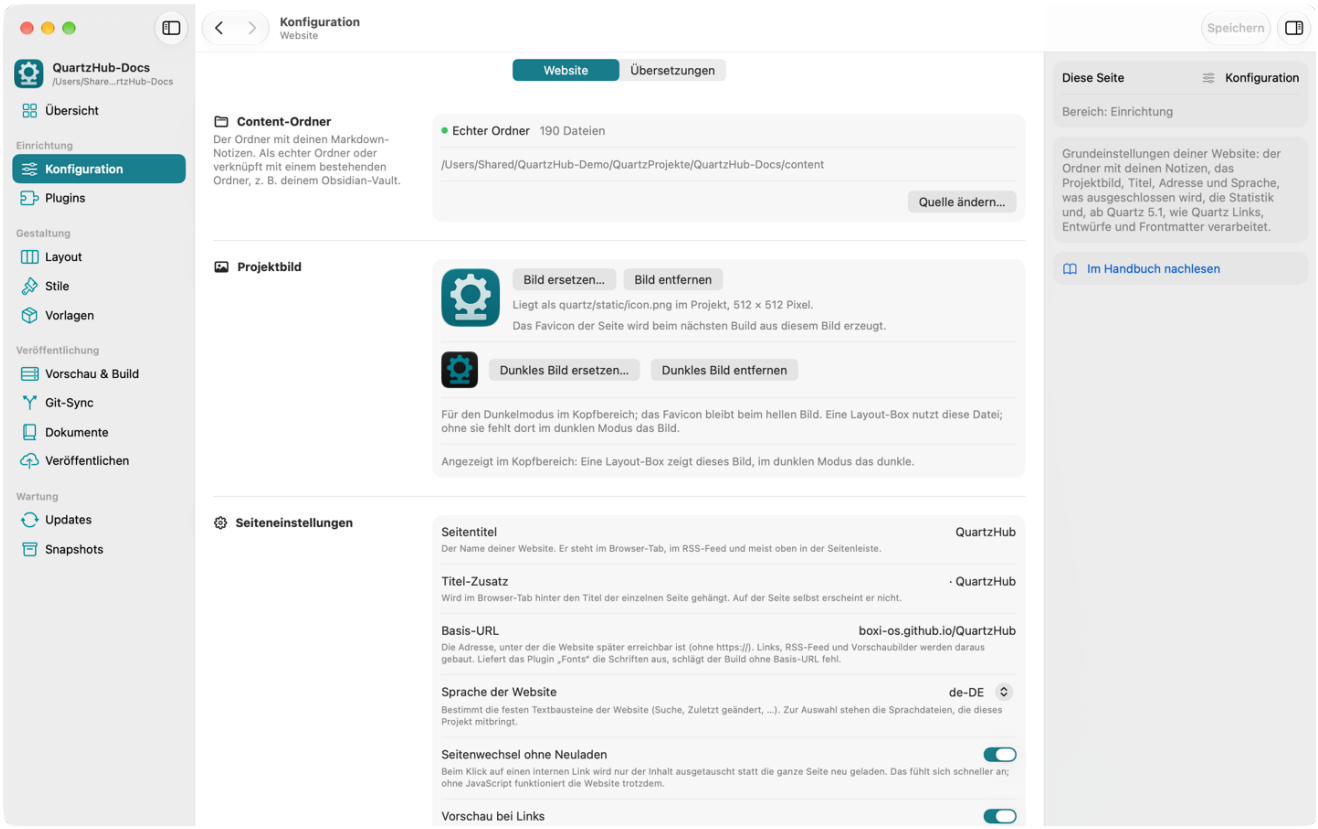
1. **Website**: Titel, Adresse, Sprache, Ausschlüsse, Statistik und die Verarbeitung von Quartz 5.1.
2. **Der Content-Ordner**: echter Ordner oder Link auf einen Vault, und wie man wechselt, ohne etwas zu verlieren.
3. **Das Projektbild**: Favicon, Bild im Kopfbereich, helle und dunkle Fassung.
4. **Übersetzungen**: die festen Texte der Website, etwa „Suche“.

Plugins

5. **Plugins**: die installierten Plugins, ihre Reihenfolge und ihre Optionen.
6. **Navigationen**: Menüs, Akkordeons, Zurück und Weiter.
7. **Zusätzliche Inhalte**: eigene Boxen mit Text, HTML oder Bildern.
8. **Mehrsprachigkeit**: eine Website in mehreren Sprachen.
9. **Formulare**: Kontaktformulare und wie ihre Nachrichten ankommen.
10. **Marktplatz**: neue Plugins finden und installieren.

4.1 – Website

Konfiguration ▸ Website bearbeitet die `quartz.config.yaml` des Projekts. Oben stehen Content-Ordner und Projektbild; die haben eigene Kapitel (4.2, 4.3), weil sie anders speichern als der Rest.



Konfiguration ▸ Website, oben Content-Ordner und Projektbild, darunter die Seiteneinstellungen.

Speichern

Alles unterhalb des Projektbilds schreibt erst **Speichern** (⌘S) in die Datei. Vorher legt QuartzHub einen Snapshot an („Vor Konfigurations-Änderung"), höchstens einen alle 15 Minuten, damit eine Reihe kleiner Änderungen nicht die Liste füllt. Die Datei wird dabei nicht neu geschrieben, sondern geändert: Kommentare und die Reihenfolge der Schlüssel bleiben.

Wer die Seite mit ungespeicherten Änderungen verlässt, wird gefragt (1.3).

Lässt sich die Datei nicht lesen, etwa weil sie von Hand kaputt bearbeitet wurde, sagt die Seite das mit dem Grund und zeigt nur den Content-Ordner. Die Datei repariert man dann in einem Editor oder holt sie aus einem Snapshot zurück (7.3).

Seiteneinstellungen

- **Seitentitel**: der Name der Website. Er steht im Browser-Tab, im RSS-Feed und meist oben in der Seitenleiste.
- **Titel-Zusatz**: wird im Browser-Tab hinter den Titel jeder Seite gehängt, etwa „ · Meine Notizen“. Auf der Seite selbst erscheint er nicht.
- **Basis-URL**: die Adresse der fertigen Website, ohne `https://`. Quartz baut daraus Links, den RSS-Feed und Vorschaubilder; liefert das Plugin „Fonts“ die Schriften aus, scheitert der Build ohne sie. Für GitHub Pages ist das meist `name.github.io/repository` (6.8).
- **Sprache der Website**: bestimmt die festen Texte der Website wie „Suche“ oder „Zuletzt geändert“. Zur Wahl stehen die Sprachdateien, die das Projekt mitbringt; die Texte selbst ändert man unter **Übersetzungen** (4.4). Für eine Website in mehreren Sprachen siehe 4.8.
- **Seitenwechsel ohne Neuladen**: Ein Klick auf einen internen Link tauscht nur den Inhalt aus. Das fühlt sich schneller an; ohne JavaScript funktioniert die Website trotzdem.
- **Vorschau bei Links**: zeigt beim Überfahren eines internen Links ein kleines Fenster mit dem Anfang der Zielseite.
- **Seiten vorab laden** (ab Quartz 5.1): lädt Zielseiten, bevor man klickt, damit sie sofort erscheinen: **Sichtbare Links**, **Beim Zeigen auf einen Link** oder **Aus**.

Notizen und Ordner ausschließen

- **Muster (eins pro Zeile)**: Dateien und Ordner im Content-Ordner, die gar nicht erst in die Website kommen, etwa `privat/**` für einen ganzen Ordner oder `*.excalidraw.md` für eine Dateiart.
- **Dateien veröffentlichen** (ab Quartz 5.1): **Alle Dateien im Content-Ordner** oder **Nur verwendete**. Mit „Nur verwendete“ kommen Bilder und andere Dateien nur auf die Website, wenn eine veröffentlichte Seite sie verwendet; die Anhänge unveröffentlichter Notizen bleiben so privat.

> ⚠ **Was im Content-Ordner liegt, ist öffentlich**
>
> Quartz veröffentlicht jede Notiz im Content-Ordner, die weder ausgeschlossen noch als Entwurf markiert ist, auch Notizen, auf die nichts verweist. Private Notizen gehören in einen ausgeschlossenen Ordner oder gar nicht erst in den Content-Ordner.

Statistik

Bindet das Zählskript eines Anbieters in jede Seite ein. Mit **Keine Statistik** wird nichts geladen und nichts gemessen.

Unter **Anbieter** stehen Plausible, Google, Umami, GoatCounter, PostHog, Tinylytics, Cabin, Clarity, Matomo, Vercel und Rybbit. Je nach Anbieter erscheinen darunter seine Felder, mit dem Namen, den Quartz für sie benutzt, etwa `websiteId` oder `host`; ein

Sternchen heißt „muss ausgefüllt sein“. Fehlt ein Pflichtfeld, sagt ein Hinweis, welches; gespeichert wird trotzdem. Wer den Anbieter wechselt, verliert die Felder des alten.

Ein Zählskript bedeutet in vielen Ländern, dass die Website darüber informieren muss. Was dafür gilt, sagt dieses Handbuch nicht.

Verarbeitung (ab Quartz 5.1)

Seit Quartz 5.1 erledigt Quartz einiges selbst, wofür es früher Plugins gab: Links, Entwürfe, Inhaltsindex und Frontmatter. Die Einstellungen dazu stehen dann in der Konfiguration. Diese Abschnitte erscheinen nur in einem Projekt mit Quartz 5.1 oder neuer.

Stehen die alten Plugin-Einträge noch in der Konfiguration, sagt der Abschnitt **Verarbeitung** das. Quartz liest sie weiter. **Konfiguration umstellen** lässt Quartz die Einstellungen in die Konfiguration verschieben und die alten Einträge entfernen; vorher legt QuartzHub einen Snapshot an. Ungespeicherte Änderungen müssen vorher gespeichert sein.

Links

- **Markdown-Links auflösen**: wie ein Link der Form `[Text](pfad)` seine Notiz findet: **Kürzester Pfad (wie Obsidian)**, **Relativ zur Notiz** oder **Ab dem Content-Ordner**.
- **Ordner aus dem Linktext entfernen**: aus „ordner/notiz“ wird „notiz“.
- **Externe Links in neuem Tab öffnen**, **Symbol hinter externen Links**, **Bilder und Videos erst beim Scrollen laden**.
- **Links auf fehlende Notizen markieren**: solche Links bekommen die CSS-Klasse `broken`.

Entwürfe

- **Entwürfe veröffentlichen**: Seiten mit `draft: true` im Frontmatter kommen sonst nicht auf die Website.

Inhaltsindex, Sitemap und RSS

- **Inhaltsindex erzeugen**: Ohne ihn fehlen Suche, Graph, Sitemap und RSS, und QuartzHub kann keine Dokumente exportieren (**6.6**).
- **Sitemap**, **RSS-Feed**, die **Einträge im RSS-Feed**, **Ganze Seite im RSS-Feed** (sonst nur die Beschreibung), **Name des Feeds** („index“ ergibt `index.xml`) und **Seiten ohne Text aufnehmen**.

Frontmatter

- **Begrenzer**: die Zeichen über und unter dem Frontmatter, meist `---`.
- **Sprache**: YAML oder TOML.

Ist ein Teil der Verarbeitung in der Konfiguration ausgeschaltet, ist sein Abschnitt grau und sagt, warum.

Ganz unten verweist die Seite auf die **Stile**: Farben und Schriften stehen dort (**5.5**).

4.2 – Der Content-Ordner

Der Content-Ordner ist `content/` im Projektordner: die Markdown-Notizen, aus denen Quartz die Website baut. Er steht ganz oben unter **Konfiguration ▸ Website**. Anders als der Rest der Seite gehört er nicht zur `quartz.config.yaml`; alles hier wirkt sofort, ohne **Speichern**.

Ordner oder Link

Der Content-Ordner ist eines von beiden:

- **Echter Ordner**: Die Notizen liegen im Projekt. Gut, wenn die Website ihre eigene Sammlung hat.
- **Symbolischer Link**: `content/` zeigt auf einen Ordner anderswo, meist einen Obsidian-Vault oder einen Ordner darin. Die Notizen bleiben, wo man sie schreibt, und jede Änderung ist sofort in der Vorschau.

Der Abschnitt zeigt, welches von beiden gerade gilt, wie viele Dateien darin liegen (versteckte wie `.obsidian` zählen nicht mit) und den Pfad, bei einem Link auch sein Ziel. Ist das Ziel weg, steht das in Rot dabei, und die Übersicht meldet „Der Content-Ordner zeigt ins Leere“.

> **Nur einen Teil des Vaults veröffentlichen**
>
> Der Link muss nicht auf den ganzen Vault zeigen. Ein Unterordner wie `Vault/Öffentlich` hält alles andere sicher draußen. Einzelne Notizen und Ordner schließt man unter **Notizen und Ordner ausschließen** aus (4.1).

Die Quelle wechseln

Quelle ändern… öffnet den Dialog „Content-Quelle ändern“:

- **Neuer Quellordner**: der Ordner mit den Notizen, getippt oder über **Auswählen**.
- **Strategie**: **Verknüpfen** macht `content/` zu einem Link auf den Ordner, **Kopieren** kopiert seinen Inhalt einmal in einen echten Ordner.

Übernehmen tut dann dreierlei:

1. Es legt einen Snapshot an („Vor Content-Wechsel“). Scheitert das, ändert sich nichts.
2. Es legt den bisherigen Content-Ordner **beiseite**, statt ihn zu löschen: ein echter Ordner wird nach `.quartz-gui/content-backups/` verschoben, von einem Link wird nur notiert, wohin er zeigte. Ein verlinkter Vault wird dabei nie angefasst.
3. Es legt den neuen Content-Ordner an, als Link oder als Kopie.

Der Quellordner darf nicht im Content-Ordner liegen und ihn nicht enthalten; der Dialog sagt es, wenn doch.

Beiseitegelegte Content-Ordner zurückholen

Beiseitegelegte Content-Ordner stehen unter **Snapshots**, in einer eigenen Liste (7.3). **Wiederherstellen** setzt einen davon wieder ein und legt dabei den aktuellen ebenfalls beiseite. Löschen lassen sie sich dort auch.

> ⚠ **Beiseitegelegt ist nicht gesichert**
>
> Ein beiseitegelegter echter Ordner ist die einzige Kopie dieser Notizen; Snapshots enthalten ihn nicht. Wer ihn löscht, kann ihn nicht zurückholen. Von einem Link bleibt nur das Ziel notiert; die Notizen liegen weiter im Vault.

Wenn die Startseite fehlt

Liegt oben im Content-Ordner keine `index.md`, zeigt die Adresse der Website eine 404-Seite. Der Abschnitt sagt das und bietet **Startseite anlegen…**: Sie bekommt den eingegebenen Titel und darunter eine Liste der Ordner und Notizen, die gerade oben im Content-Ordner liegen. Die Liste wird später nicht aktualisiert; die Seite ist ein Anfang, den man selbst weiterschreibt.

Ist der Content-Ordner ein Link, entsteht die Datei im verlinkten Ordner, also bei den eigenen Notizen; der Dialog sagt das vorher. Eine vorhandene `index.md` wird nie überschrieben.

Content-Ordner und git

Ein Link ist für git nur ein Verweis. Wer das Projekt mit **Git-Sync** auf einen anderen Rechner bringt, bringt die Notizen also nicht mit; dort muss der Content-Ordner neu verknüpft werden. Ob Snapshots den Content-Ordner mitsichern, stellt man unter **Snapshots** ein; einen verlinkten Vault sichern sie nie (7.2).

4.3 – Das Projektbild

Das Projektbild steht unter **Konfiguration ▸ Website**, gleich unter dem Content-Ordner. Es dient dreimal:

- **in QuartzHub**: in der Projektliste und oben in der Seitenleiste des Projektfensters;
- **als Favicon** der Website, wenn das Plugin „favicon“ aktiv ist; der Abschnitt sagt, ob es das ist;
- **im Kopfbereich** der Website, wenn man es dort einschaltet.

Bilder werden sofort geschrieben, ohne **Speichern** und ohne Snapshot.

Bild wählen

Bild wählen… nimmt eine PNG- oder JPEG-Datei, am besten quadratisch. QuartzHub legt sie als PNG nach `quartz/static/icon.png` und verkleinert sie dabei auf höchstens 512 Pixel an der längeren Kante. Das Bild, das Quartz dort mitgebracht hat, hebt es beim ersten Mal auf.

Bild entfernen setzt das ursprüngliche Bild wieder ein; gab es keines, ist danach keines da.

Steht dort „Die Seite zeigt ein Bild, das nicht in QuartzHub gewählt wurde“, stammt das Bild aus einer Vorlage oder wurde von Hand hineinkopiert. Ein eigenes ersetzt es.

Dunkle Fassung

Bild für den Dunkelmodus… legt ein zweites Bild nach `quartz/static/icon-dark.png`. Es gilt nur für das Bild im Kopfbereich, wenn die Website dunkel angezeigt wird; das Favicon bleibt beim hellen. **Dunkles Bild entfernen** löscht die Datei sofort.

Im Kopfbereich zeigen

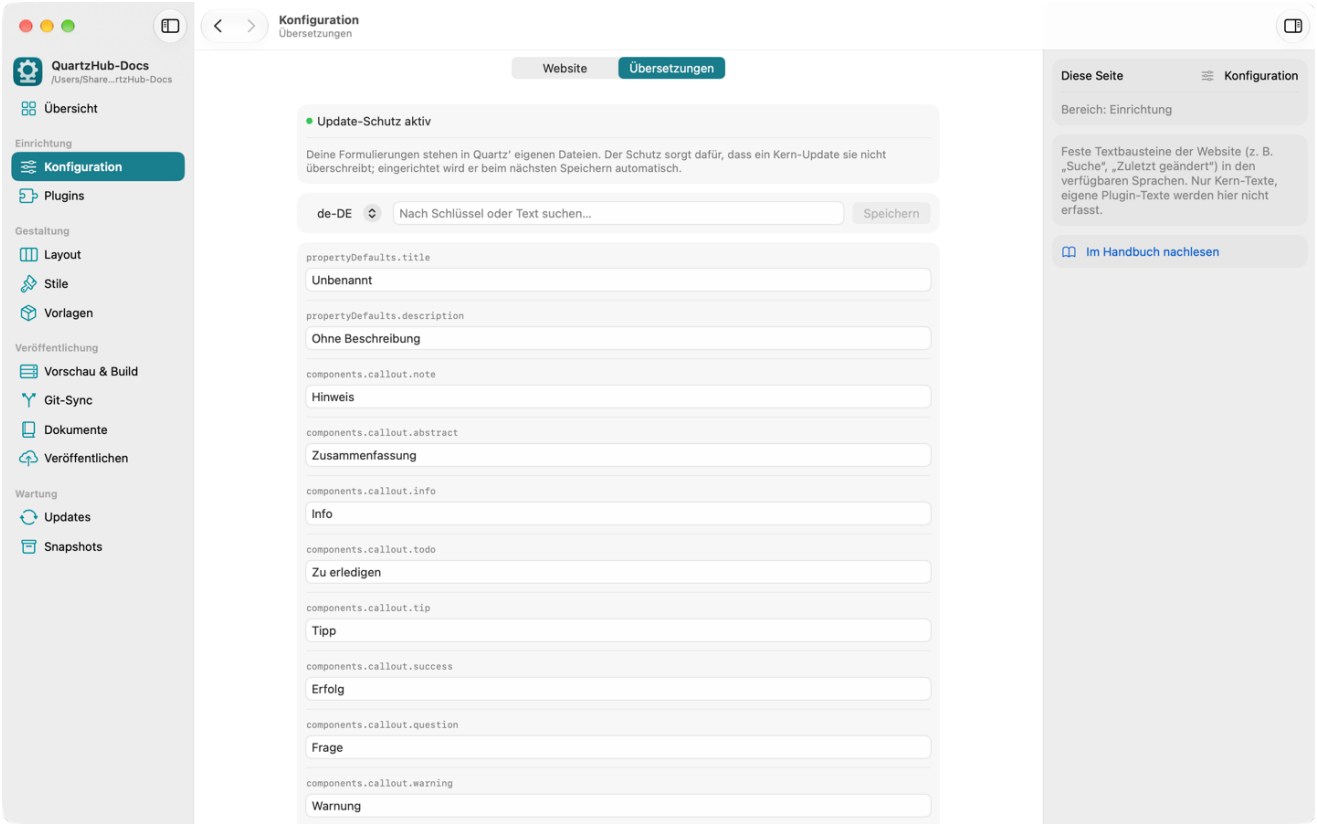
Im Kopfbereich der Website zeigen setzt das Projektbild als Link zur Startseite oben auf jede Seite, im dunklen Modus das dunkle Bild. Das geht mit dem Plugin quartz-layout-box. Fehlt es, fragt QuartzHub beim Einschalten, ob es installiert werden soll; dafür speichert es vorher die ungespeicherten Änderungen der Seite und braucht eine Verbindung ins Netz. Danach ist der Schalter an, aber erst **Speichern** schreibt ihn in die Konfiguration.

Zeigt schon eine Layout-Box einer Vorlage das Bild, etwa die des Basis-Templates, steht statt des Schalters ein Hinweis darauf.

Wo genau das Bild im Kopfbereich steht, verschiebt man auf der Bühne unter **Layout ▸ Komponenten** (5.3).

4.4 – Übersetzungen

Quartz bringt für jede Sprache eine Datei mit den festen Texten der Website mit: „Suche“, „Zuletzt geändert“, „Inhaltsverzeichnis“, die Titel der Callouts und so fort. **Konfiguration ▸ Übersetzungen** bearbeitet diese Dateien. Texte, die Plugins mitbringen, stehen nicht darin; die stellt man bei den Plugins ein, wo sie es erlauben.



Die Texte der deutschen Sprachdatei. Oben der Update-Schutz.

Texte ändern

- **Sprache**: die Sprachdatei, die bearbeitet wird. Das wählt nur die Datei aus; welche Sprache die Website spricht, steht unter **Website** (4.1).
- **Suchfeld**: findet Einträge nach Schlüssel (etwa `components.search.title`) oder Text.
- Jeder Eintrag zeigt seinen Schlüssel und darunter den Text. Einträge mit dem Zeichen **JS** sind kein einfacher Text, sondern ein Stück Programmcode, etwa wenn eine Zahl in den Text eingesetzt wird. Sie werden wörtlich als Code gespeichert; wer sie ändert, muss gültiges TypeScript schreiben.

Speichern auf diesem Reiter schreibt alle geänderten Einträge der gewählten Sprache; neben ihm steht, wie viele noch ungespeichert sind; ⌘S tut dasselbe. Ein Eintrag, der sich nicht schreiben ließ, behält die Änderung und sagt darunter, warum.

Ungespeicherte Änderungen bleiben, wenn man auf eine andere Seite geht und zurückkommt. Wer eine andere Sprache wählt, das Fenster schließt oder QuartzHub beendet, wird gefragt: speichern, verwerfen oder abbrechen.

Einen Snapshot legt das Speichern nicht an. Den Stand vor der ersten Änderung einer Sprache hebt QuartzHub aber auf, in `.quartz-gui/locale-baseline/`.

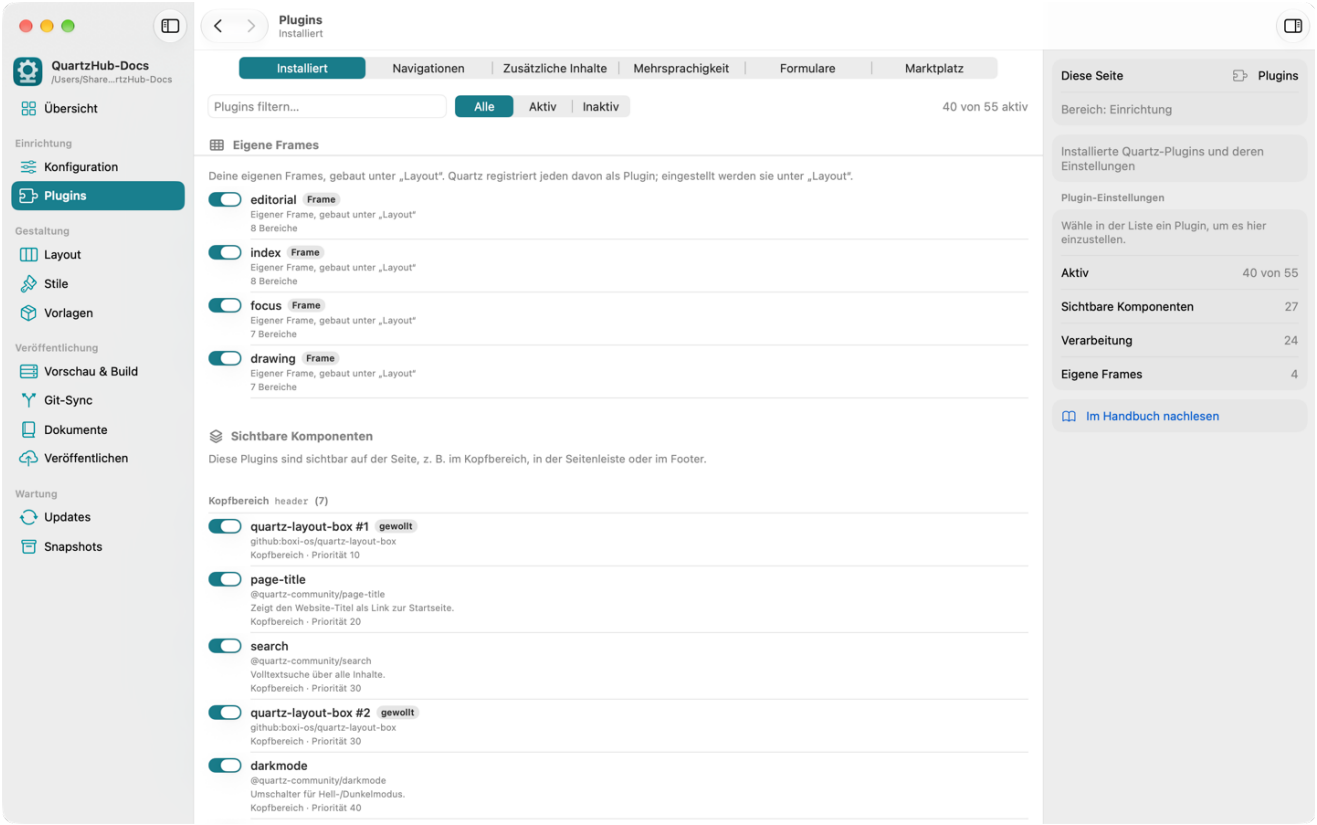
Der Update-Schutz

Die Sprachdateien gehören zu Quartz. Ein Kern-Update (**7.1**) würde die eigenen Formulierungen deshalb mit Quartz' neuen Texten überschreiben. Der **Update-Schutz** sagt git, bei diesen Dateien im Zweifel die eigene Fassung zu behalten. Er wird beim ersten Speichern von selbst eingerichtet; **Jetzt aktivieren** richtet ihn sofort ein.

Der Schutz hat eine Kehrseite: Ändert ein Update eine Sprachdatei, die man selbst geändert hat, bleibt die eigene Fassung ganz, und neue Texte des Updates kommen nicht hinein. Fehlt danach ein Text, trägt man ihn hier nach.

4.5 – Plugins

Fast alles, was Quartz tut, tut ein Plugin: Markdown lesen, Links auflösen, die Suche, die Seitenleiste, das Inhaltsverzeichnis. **Plugins ▸ Installiert** zeigt alle Plugins des Projekts. Die anderen Reiter der Seite sind Editoren für einzelne Plugins (4.6 bis 4.9) und der Marktplatz (4.10).



Plugins ▸ Installiert: oben die eigenen Frames, darunter die sichtbaren Komponenten nach Position.

Die Liste

Oben filtern ein Suchfeld und **Alle**, **Aktiv**, **Inaktiv**; daneben steht, wie viele aktiv sind. Die Plugins stehen in Gruppen:

- **Eigene Frames**: die Seitenraster aus dem Layout-Editor. Quartz führt jeden als Plugin; eingestellt werden sie unter **Layout ▸ Frames** (5.4).
- **Sichtbare Komponenten**: was auf der Seite erscheint, nach Position: **Kopfbereich**, **Linke Seitenleiste**, **Rechte Seitenleiste**, **Vor dem Inhalt**, **Nach dem Inhalt**, **Fußzeile** und **Im Inhalt**. Innerhalb einer Position nach ihrer Priorität.
- **Verarbeitung**: was die Notizen im Hintergrund verändert, etwa Formatierung, Links, Bilder, und selbst nicht sichtbar ist. Zuerst die **Seitentypen**, dann **Transformer, Filter & Emitter**, je in der Reihenfolge, in der Quartz sie anwendet.
- **Wartung**: siehe unten.

Jede Zeile zeigt den Schalter, den Namen, woher das Plugin kommt, bei bekannten Plugins eine kurze Beschreibung und seine Stelle (`position` und `priority` oder `order`). Ausgeschaltete Plugins sind blasser. Dazu können Zeichen kommen:

- **Update verfügbar**: Es gibt eine neuere Fassung (7.1).
- **In Quartz eingebaut**: Seit Quartz 5.1 erledigt Quartz das selbst; eingestellt wird es unter **Konfiguration ▸ Website** (4.1).
- **Fehler laut Quartz**, **Hinweis von Quartz** oder **gewollt**: das Ergebnis von Quartz' eigener Prüfung, siehe unten.

Ein- und ausschalten, umsortieren, entfernen

Der **Schalter** einer Zeile schreibt sofort in die Konfiguration. Die Reihenfolge ändert man durch Ziehen, mit **Anordnen ▸ Nach oben** und **Nach unten** (⌥⌘↑, ⌥⌘↓) oder im Kontextmenü, jeweils innerhalb einer Gruppe; QuartzHub nummeriert die Gruppe dabei neu (10, 20, 30 …). Solange gefiltert wird, geht das nicht.

Entfernen steht im Kontextmenü. Es fragt nach, legt einen Snapshot an und lässt dann Quartz das Plugin entfernen. Ein eigener Frame wird dabei auch von der Platte gelöscht. Im Kontextmenü steht außerdem **Repository auf GitHub öffnen**.

Die Optionen

Ist ein Plugin ausgewählt, zeigt der Inspektor seine Einstellungen. **Er speichert ohne Knopf: Schalter sofort, Textfelder beim Verlassen** oder mit der Eingabetaste. Jede Änderung legt einen Snapshot an, höchstens einen alle 15 Minuten.

- Oben Name, Herkunft, Beschreibung und **Plugin aktiv**.
- **Platzierung** (nur bei sichtbaren Komponenten): `position`, `priority`, `display` (alle Geräte, nur mobil, nur Desktop), `condition` (etwa `not-index`, `has-tags`, `has-backlinks`, `has-toc`) und `group` mit ihren Optionen. Mit der Maus geht das auch auf der Bühne: **Auf der Bühne zeigen** führt zu **Layout ▸ Komponenten** (5.3).
- **Optionen**: ein Formular aus dem, was das Plugin über seine Optionen sagt. Ja/Nein wird ein Schalter, eine Auswahl ein Menü, Listen bekommen Knöpfe zum Hinzufügen und Umsortieren. Ab acht Optionen gibt es ein Suchfeld, selten gebrauchte stehen unter **Selten gebraucht**.

Zu den Feldern:

- **geändert** heißt: Der Wert steht in der Konfiguration. Der Knopf daneben entfernt ihn wieder, und das Plugin nimmt seine Vorgabe.
- **wirkt nicht** heißt: Die Option gilt nur zusammen mit einer anderen, die gerade anders steht.
- Was QuartzHub nicht als Formular darstellen kann, bearbeitet man als **JSON**. Ungültiges JSON wird nicht gespeichert.

- Werte, die in der Konfiguration stehen, die das Plugin aber nicht beschreibt, stehen unter **Unbekannte Schlüssel**.
- Beschreibt ein Plugin seine Optionen gar nicht, kommen die Felder aus seinen Vorgaben, und **Weitere Option eintragen** nimmt einen Schlüssel mit Wert auf.

QuartzHub liest die Beschreibung der Optionen aus dem Paket des Plugins, ohne es auszuführen.

Quartz' eigene Prüfung (ab Quartz 5.1)

Ab Quartz 5.1 lässt QuartzHub nach jeder Änderung `quartz plugin doctor` laufen, Quartz' eigene Prüfung der Plugins. Was sie meldet, steht als Zeichen an der Zeile und im Inspektor unter **Quartz meldet**, in Quartz' Worten und auf Englisch. „gewollt" heißt: Quartz bemängelt etwas, das QuartzHub absichtlich so macht, etwa mehrere Einträge desselben Plugins, einen je Box oder Navigation.

Laufen GitHub-Markdown und Obsidian-Markdown in der falschen Reihenfolge, sagt ein Hinweis über der Liste das; **Reihenfolge tauschen** behebt es.

Wartung

Ganz unten stehen zwei Knöpfe, die an den heruntergeladenen Plugins in `.quartz/plugins/` arbeiten:

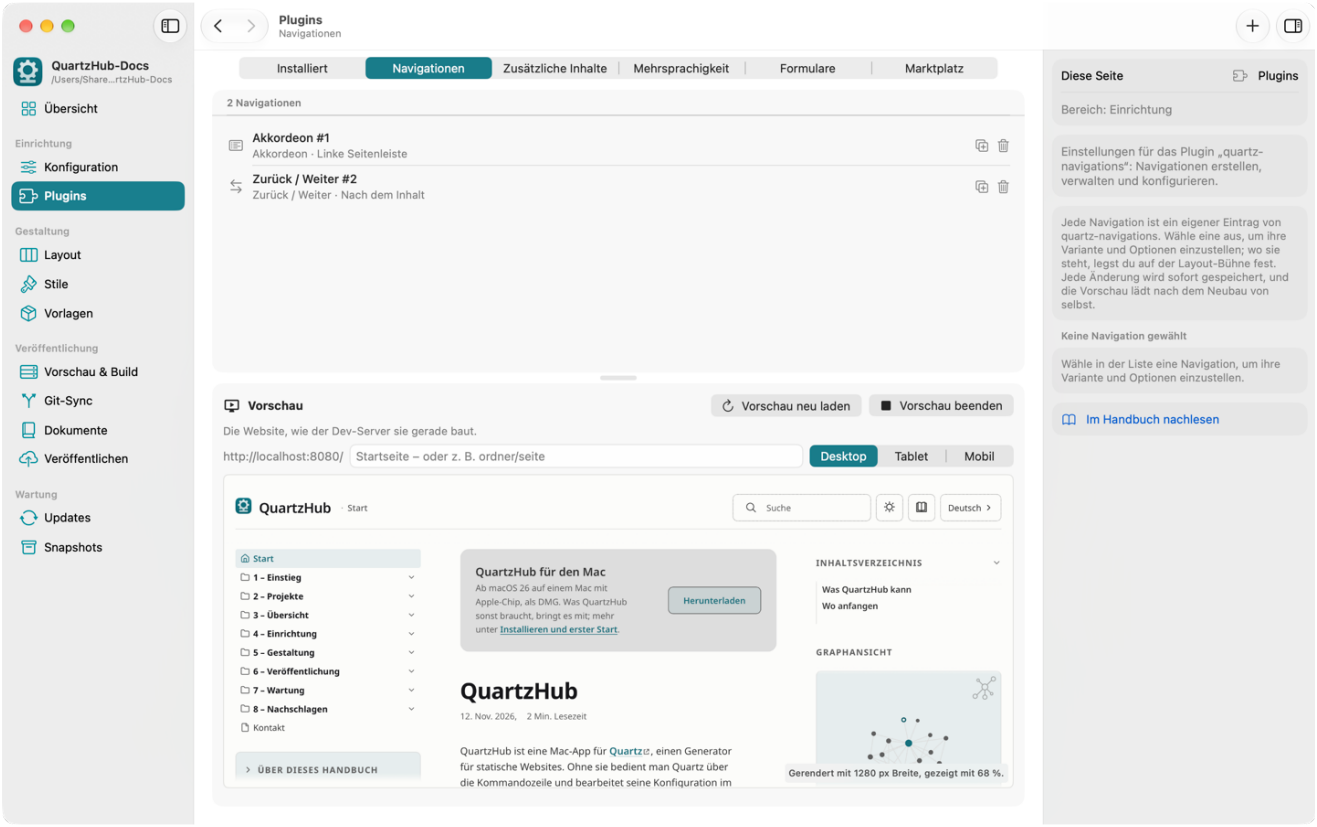
- **Aus quartz.lock.json wiederherstellen** lädt alle Plugins in der Fassung, die `quartz.lock.json` festhält. Das braucht man nach dem Klonen auf einem anderen Rechner (2.3); fehlende npm-Pakete werden dabei zuerst installiert.
- **Verwaiste Plugin-Ordner entfernen** löscht heruntergeladene Plugins, auf die die Konfiguration nicht mehr zeigt.

Beide ändern nichts an der Konfiguration.

4.6 – Navigationen

Plugins ▸ Navigationen stellt das Plugin quartz-navigations ein. Es baut aus den Ordnern und Notizen des Content-Ordners Navigationen: eine Baumliste in der Seitenleiste, ein Menü im Kopfbereich, eine Sitemap im Fuß, Zurück und Weiter unter dem Text. Jede Navigation ist ein eigener Eintrag des Plugins; eine Website kann mehrere haben.

Ist das Plugin nicht installiert, sagt der Reiter das; installiert wird es im Marktplatz über `boxi-os/quartz-navigations` (4.10).



Oben die Navigationen des Projekts, unten die Vorschau, rechts der Inspektor.

So sind die Editor-Reiter gebaut

Navigationen, Zusätzliche Inhalte, Mehrsprachigkeit und Formulare sind gleich gebaut:

- **Oben die Liste** der Einträge, **unten die Vorschau** der Website. Den Griff dazwischen kann man ziehen (1.3).
- **Rechts der Inspektor** mit den Einstellungen des gewählten Eintrags.
- **Gespeichert wird sofort**: Schalter beim Umlegen, Textfelder beim Verlassen. Jede Änderung legt einen Snapshot an, höchstens einen alle 15 Minuten. ⌘S gilt hier nicht.
- **Die Vorschau** zeigt die Website über den Dev-Server; läuft er nicht, startet **Vorschau starten** ihn. Oben wählt man die Seite und die Breite (**Desktop**, **Tablet**, **Mobil**).

Der gewählte Eintrag ist in der Vorschau markiert, und darunter steht, ob er auf dieser Seite vorkommt. **Auf Element zoomen** vergrößert ihn. Nach einer Änderung lädt die Vorschau neu, sobald Quartz neu gebaut hat.

- **Auf der Bühne zeigen** führt zu **Layout ▸ Komponenten**, wo man die Stelle auf der Seite verschiebt (5.3). **Alle Optionen** führt zum Eintrag unter **Installiert**, mit dem allgemeinen Formular (4.5).
- Kennt der Editor die installierte Fassung des Plugins nicht, sagt ein Hinweis, dass Angaben abweichen können.

Eine Navigation anlegen

➕ **Neue Navigation** in der Symbolleiste fragt nach der Variante. Die Navigation kommt dorthin, wo diese Variante üblicherweise steht; verschieben lässt sie sich danach auf der Bühne.

Variante	Was sie ist	Steht zuerst
Baum	Verschachtelte Liste, alles offen	Linke Seitenleiste
Akkordeon	Ordner klappen auf, der Pfad zur Seite ist offen	Linke Seitenleiste
Seitliche Panels	Ordner öffnen ein Panel daneben	Linke Seitenleiste
Auswahlliste	Ein Sprungmenü, Ordner als Gruppen	Linke Seitenleiste
Leiste	Erste Ebene in einer Zeile	Kopfbereich
Aufklappmenü	Zeile, Ordner öffnen ein Panel darunter	Kopfbereich
Mega-Menü	Panel mit der zweiten Ebene als Spalten	Kopfbereich
Reiter	Erste Ebene als Reiter, Kinder in einer zweiten Zeile	Kopfbereich
Spalten	Überschriften mit Kindern darunter, eine Sitemap	Fußzeile
Zurück / Weiter	Vorige und nächste Seite	Nach dem Inhalt

Die Liste

Jede Zeile zeigt Name, Variante und Position, ausgeschaltete mit „aus“. Die Knöpfe einer Zeile **duplizieren** die Navigation oder **entfernen** sie; die letzte lässt sich nicht entfernen. Entfernen löscht den Eintrag mit allen Optionen aus der Konfiguration; das Plugin bleibt installiert.

Einstellen

Der Inspektor zeigt für die gewählte Navigation:

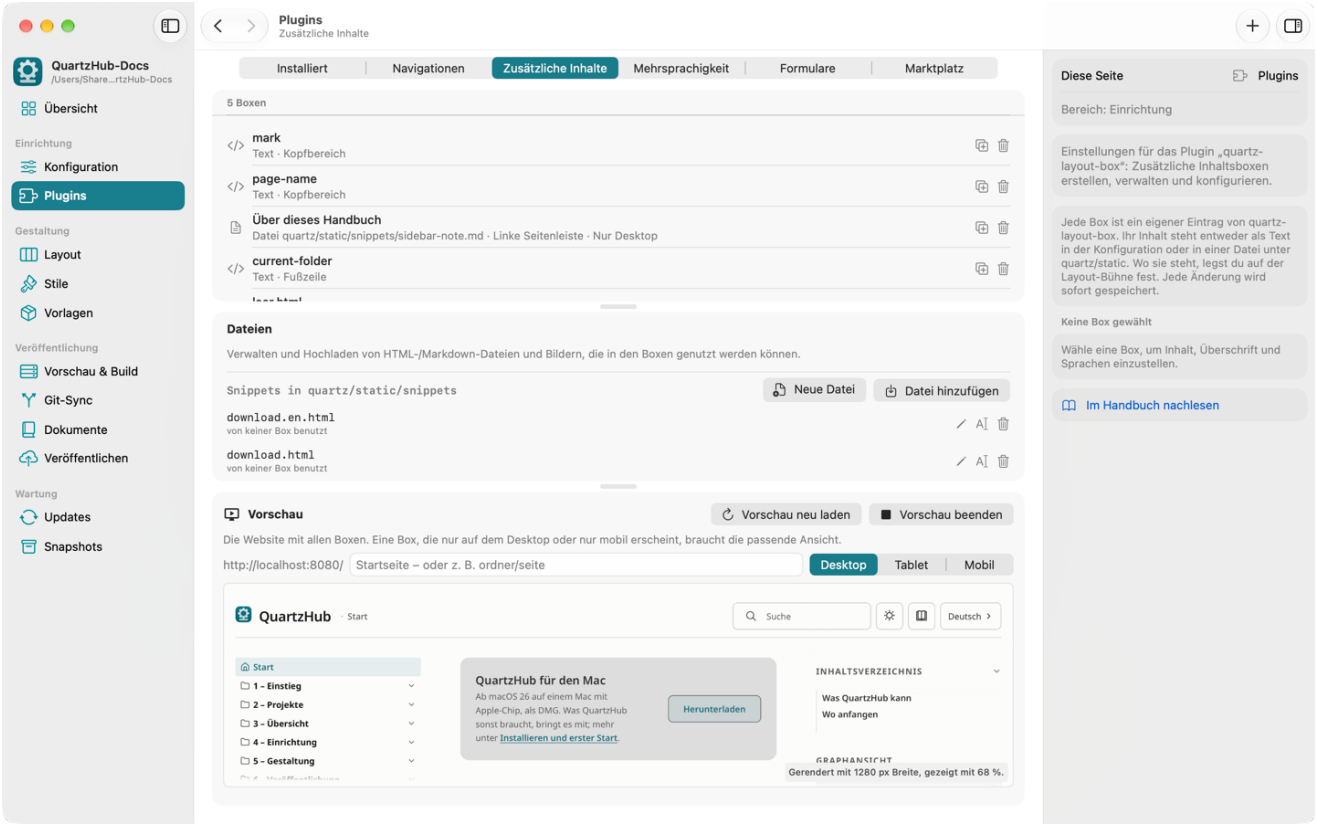
- **Navigation aktiv**: ein- oder ausschalten, ohne sie zu löschen.
- **Variante**: dieselben zehn Kacheln wie beim Anlegen; ein Klick wechselt die Variante, die übrigen Einstellungen bleiben.

- **Optionen**: zuerst die Einstellungen der gewählten Variante, dann die allgemeinen. Optionen, die bei dieser Variante nichts bewirken, sind grau und sagen, warum, etwa „Wirkt nur bei der Variante …“. Manche Varianten verhalten sich auf dem Smartphone anders; was dort gilt, stellt eine eigene Option ein.

Was die einzelnen Optionen tun, beschreibt das Plugin selbst neben jedem Feld.

4.7 – Zusätzliche Inhalte

Plugins ▸ Zusätzliche Inhalte stellt das Plugin quartz-layout-box ein. Eine Box ist ein Stück eigener Inhalt, das auf jeder Seite an derselben Stelle steht: ein Hinweis in der Seitenleiste, ein Impressum-Link in der Fußzeile, ein Logo im Kopfbereich. Jede Box ist ein eigener Eintrag des Plugins. Bedient wird der Reiter wie die anderen Editoren (4.6).



Oben die Boxen, in der Mitte die Dateien, unten die Vorschau.

Boxen anlegen und verwalten

➕ **Neue Box** legt sofort eine Box mit einem Beispieltext in der linken Seitenleiste an. Jede Zeile der Liste zeigt Name, Art des Inhalts und Position, dazu „Nur Desktop“ oder „Nur Mobil“, wo das eingestellt ist. Die Knöpfe einer Zeile duplizieren oder entfernen die Box. Entfernen löscht nur den Eintrag; eine Datei, die die Box zeigt, bleibt liegen.

Der Inhalt einer Box

Im Inspektor wählt man unter **Inhalt**, woher die Box ihren Inhalt nimmt:

- **Text in der Konfiguration**: HTML, direkt im Inspektor geschrieben. Gut für kurze Dinge.
- **Datei**: eine Markdown- oder HTML-Datei unter `quartz/static/`. Gut für Längeres, und die Datei lässt sich mit jedem Editor bearbeiten.

Wechselt man von Text zu Datei, fragt QuartzHub, ob der Text aus der Konfiguration entfernt werden soll; danach ist er weg. Eine Box kann außerdem je Sprache und Seite einen eigenen Inhalt haben; der geht dann vor.

Darunter stehen die übrigen Optionen des Plugins, etwa Überschrift, Darstellung und Sprachen.

Dateien

Der mittlere Bereich verwaltet die Dateien, die Boxen benutzen:

- **Schnipsel** in `quartz/static/snippets/`: **Neue Datei** legt eine Markdown- oder HTML-Datei an, **Datei hinzufügen** kopiert eine vorhandene hinein. Jede Datei sagt, welche Boxen sie benutzen. Eine Datei umbenennen zieht die Boxen nach, die sie nennen. Löschen legt sie in den Papierkorb.
- **Bilder** in `quartz/static/images/`: **Bild hinzufügen** nimmt PNG, JPEG, GIF, WebP, SVG und AVIF. Wird ein Bild umbenannt, das eine Box benutzt, ändert QuartzHub die Verweise im Text nicht; das muss man selbst tun.
- **Projektbild**: `icon.png` und `icon-dark.png`, nur zum Ansehen. In einem Schnipsel bindet man es mit dem Pfad ein, der dort steht (4.3).

Schnipsel bearbeiten

Der Stift an einem Schnipsel öffnet ihn im Code-Editor, mit Zeilennummern und Farben für HTML und Markdown. Der Schnipsel ist eine eigene Datei: **Snippet speichern** schreibt ihn, nicht die Optionen der Box und nicht ⌘S.

Verfügbare Platzhalter setzt beim Bauen Werte ein, im Text wie im Titel der Box:

Platzhalter	Wird zu
`{{title}}`	Titel der Seite, aus ihrem Frontmatter
`{{slug}}`	Pfad der Seite, etwa „ordner/notiz“
`{{root}}`	Weg zur Startseite, für Links und Bilder: `{{root}}/static/images/logo.png`
`{{siteTitle}}`	Titel der Website
`{{baseUrl}}`	Adresse der Website
`{{locale}}`, `{{lang}}`	Sprache der Seite, etwa „de-DE“ oder nur „de“
`{{frontmatter.author}}`	jeder Eintrag im Frontmatter der Seite

Ein Klick fügt den Platzhalter am Cursor ein. Ein unbekannter bleibt stehen, wie er

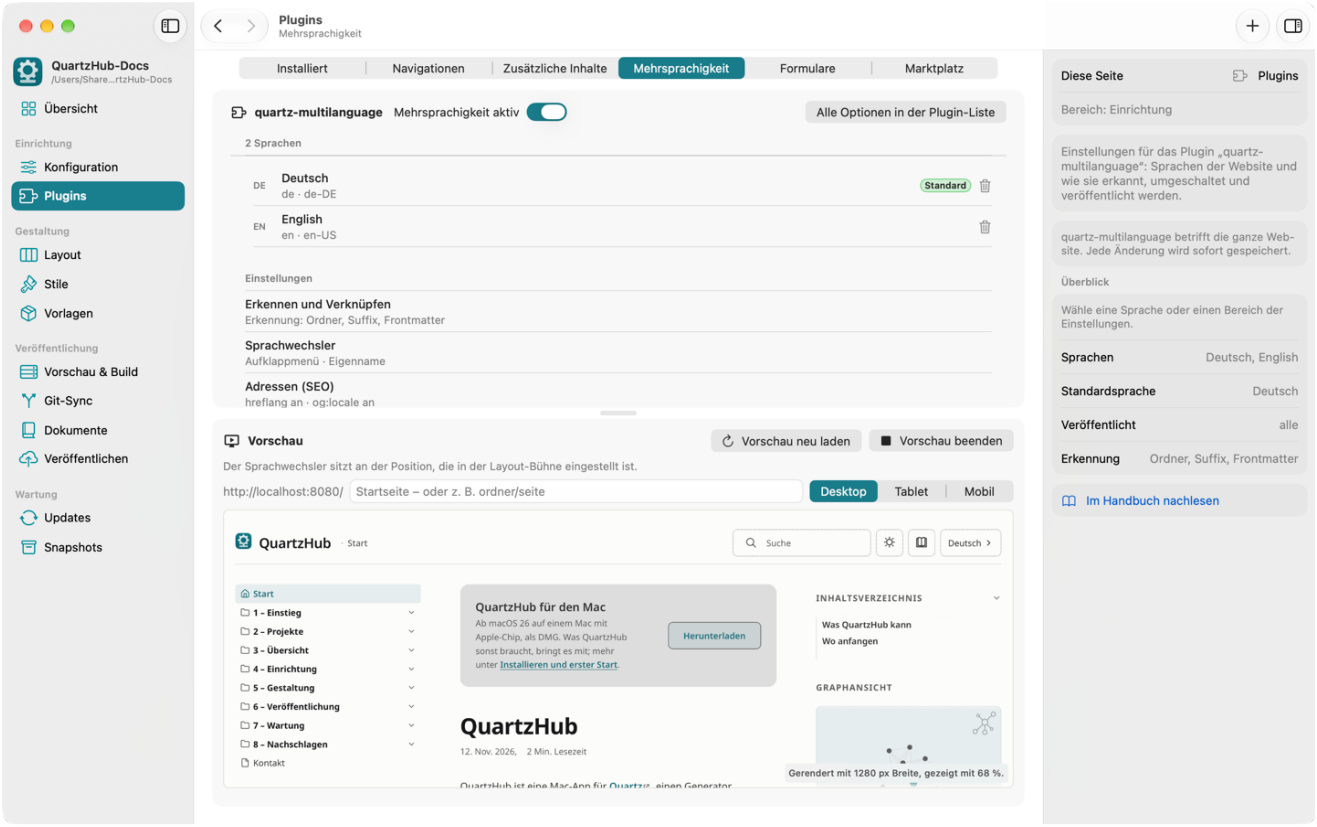
geschrieben ist.

Bild einfügen… kopiert ein Bild nach `quartz/static/images/` und setzt den Verweis ein. Eine SVG-Datei lässt sich auch als Code einfügen; QuartzHub entfernt dabei, was darin Programmcode ausführen könnte, und sagt, was es entfernt hat.

4.8 – Mehrsprachigkeit

Plugins ▸ Mehrsprachigkeit stellt das Plugin quartz-multilanguage ein. Es macht aus einem Content-Ordner eine Website in mehreren Sprachen: Es erkennt, in welcher Sprache eine Notiz geschrieben ist, verbindet Übersetzungen derselben Seite, setzt einen Sprachwechsler auf die Seite und sagt Suchmaschinen, welche Fassung für welche Sprache gilt. Das Plugin gilt für die ganze Website, nicht je Eintrag. Bedient wird der Reiter wie die anderen Editoren (4.6).

Dieses Handbuch selbst ist so gebaut: die deutschen Seiten oben im Content-Ordner, die englischen im Ordner `en/`, verbunden über einen gemeinsamen Schlüssel im Frontmatter.



Zwei Sprachen, Deutsch als Standard. Darunter die Einstellungen, unten die Vorschau.

Ein- und ausschalten

Der Schalter **Eingeschaltet** oben baut die Website mit oder ohne Spracherkennung, Sprachwechsler und Weiterleitungen. Die Einstellungen bleiben dabei erhalten.

Sprachen

➕ **Neue Sprache** fragt nach dem Code, mit dem Ordner, Dateinamen und Frontmatter die Sprache nennen, etwa `fr`. Die Liste zeigt jede Sprache mit Code, Namen und

Locale; **Standard** markiert die Standardsprache, „nicht veröffentlicht“ eine, deren Seiten nicht auf die Website kommen. In dieser Reihenfolge stehen die Sprachen im Sprachwechsler; umsortiert wird durch Ziehen, mit **Anordnen ▸ Nach oben/Nach unten** (⌥⌘↑, ⌥⌘↓) oder im Inspektor mit **Früher** und **Später**.

Für eine gewählte Sprache zeigt der Inspektor:

- **code**, **label**, **native** und **locale**. Ein neuer Code ändert keine Ordner- oder Dateinamen im Content-Ordner; Standardsprache und Veröffentlichen ziehen mit.
- **Standardsprache**: gilt für Seiten, die die Erkennung keiner Sprache zuordnet. Abgeben kann man sie nur, indem man eine andere Sprache zur Standardsprache macht.
- **Veröffentlichen**: ausgeschaltet baut die Website ohne die Seiten dieser Sprache, etwa für eine Übersetzung, die noch nicht fertig ist. Die letzte veröffentlichte Sprache bleibt an.

Eine Sprache **entfernen** nimmt sie aus der Konfiguration. Ihre Seiten bleiben liegen und gelten danach als Seiten der Standardsprache.

Einstellungen

Unter den Sprachen stehen die Bereiche der Einstellungen, jeder mit einer Zusammenfassung, etwa „Erkennung: Ordner, Suffix, Frontmatter“. Ein Klick zeigt den Bereich im Inspektor. Welche Bereiche es gibt, bestimmt das Plugin; meist sind es:

- **Erkennung**: woran das Plugin die Sprache einer Notiz erkennt, am Ordner (`en/…`), an einem Suffix im Dateinamen (`notiz.en.md`) oder am Frontmatter.
- **Sprachwechsler**: wie er aussieht und was er anzeigt. Wo er steht, legt man auf der Bühne fest (**Auf der Bühne zeigen**).
- **Adressen (SEO)**: die Angaben für Suchmaschinen, welche Fassung einer Seite für welche Sprache gilt.

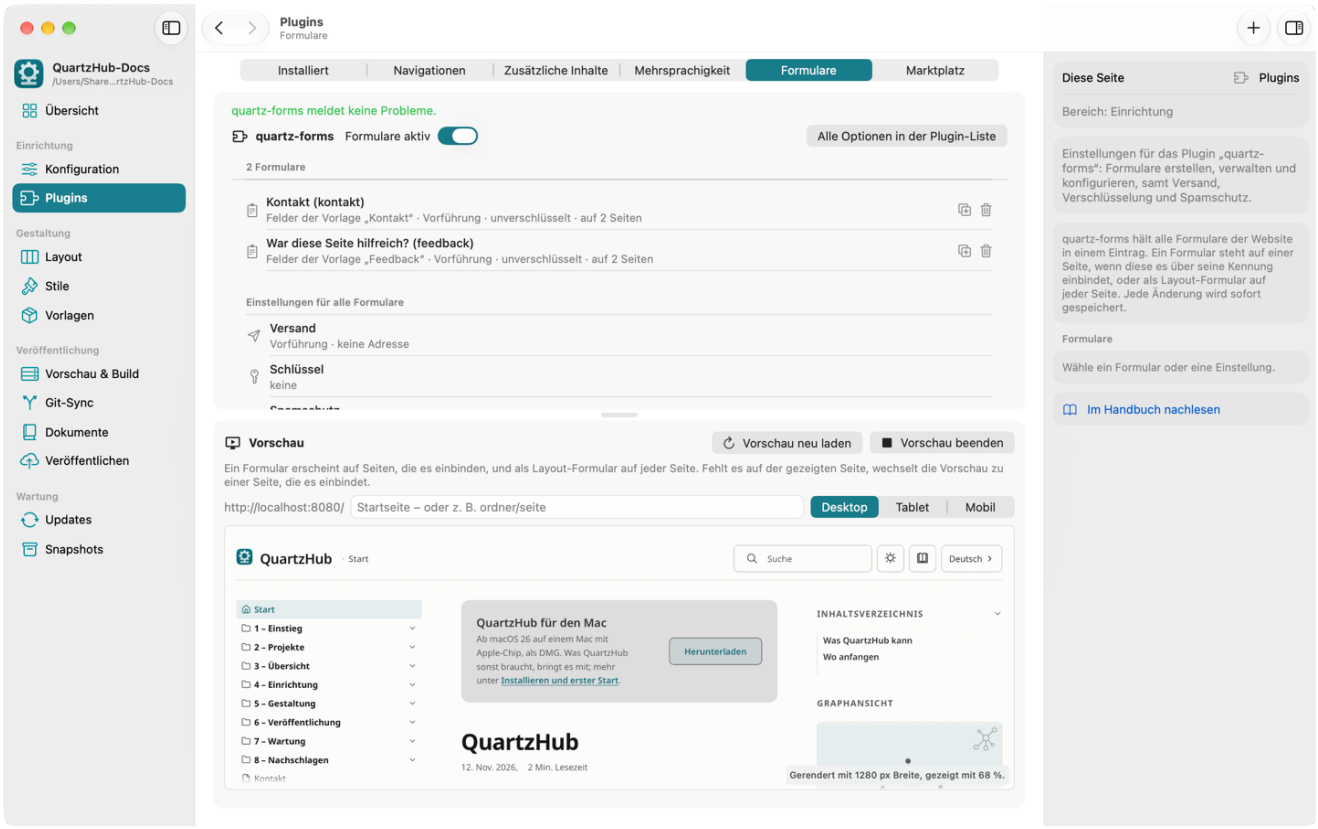
Was die einzelnen Optionen tun, beschreibt das Plugin neben jedem Feld.

> **Feste Texte je Sprache**
>
> Die festen Texte der Website wie „Suche“ kommen für jede Sprache aus Quartz’ Sprachdateien; ändern lassen sie sich unter **Übersetzungen** (4.4).

4.9 – Formulare

Eine Website aus Quartz ist eine Sammlung fester Dateien; sie kann selbst keine Nachricht annehmen. Das Plugin quartz-forms baut deshalb Formulare, die ihre Nachricht an einen Dienst schicken, der sie weitergibt: einen eigenen Endpunkt, einen Anbieter wie Formspree oder Web3Forms oder das **Relay**, ein kleines PHP-Skript auf dem eigenen Webspace. **Plugins ▸ Formulare** stellt all das ein.

Ist das Plugin nicht installiert, sagt der Reiter das; installiert wird es im Marktplatz über `boxi-os/quartz-forms` (4.10). Das Basis-Template bringt es mit, samt einem Kontaktformular, das noch nichts sendet. Bedient wird der Reiter wie die anderen Editoren (4.6).



Oben die Formulare und die Einstellungen für alle, unten die Vorschau. „Vorführung“ heißt: Das Formular sendet nichts.

Ein Formular anlegen

➕ **Neues Formular** fragt, womit es beginnt:

Vorlage	Felder
Kontakt	Name, E-Mail, Nachricht, Einwilligung
Feedback	Bewertung und Kommentar zur Seite

Anmeldung	Name, E-Mail, Termin, Personen
Umfrage	anonym: Auswahl, Skala, Kommentar
Leer	ein Nachrichtenfeld, alles Weitere selbst

Dazu kommt die **Kennung**, etwa `kontakt`: Kleinbuchstaben, Ziffern und Bindestriche. Mit ihr bindet eine Seite das Formular ein.

Ein Formular einbinden

Ein Formular erscheint auf zweierlei Weise:

- **Auf einer Seite**, die es über seine Kennung nennt, auf einem von drei Wegen:
  - im Frontmatter: `form: kontakt`, für mehrere `form: [kontakt, feedback]`;
  - als Platzhalter im Text: `{{form:kontakt}}`;
  - als Codeblock der Sprache `quartz-form` mit der Kennung als Inhalt.

  Den Frontmatter-Schlüssel und die Sprache des Codeblocks stellt man unter **Einstellungen für alle Formulare ▸ Technisches** um.
- **Als Layout-Formular** auf jeder Seite an derselben Stelle, etwa unter dem Text. Eine Seite blendet es mit `form: false` im Frontmatter aus.

Die Liste sagt bei jedem Formular, auf wie vielen Seiten es steht. Nennt eine Seite ein Formular, das es nicht gibt, steht das über der Liste, denn der Build bricht daran ab. Die Vorschau springt zu einer Seite, die das gewählte Formular einbindet.

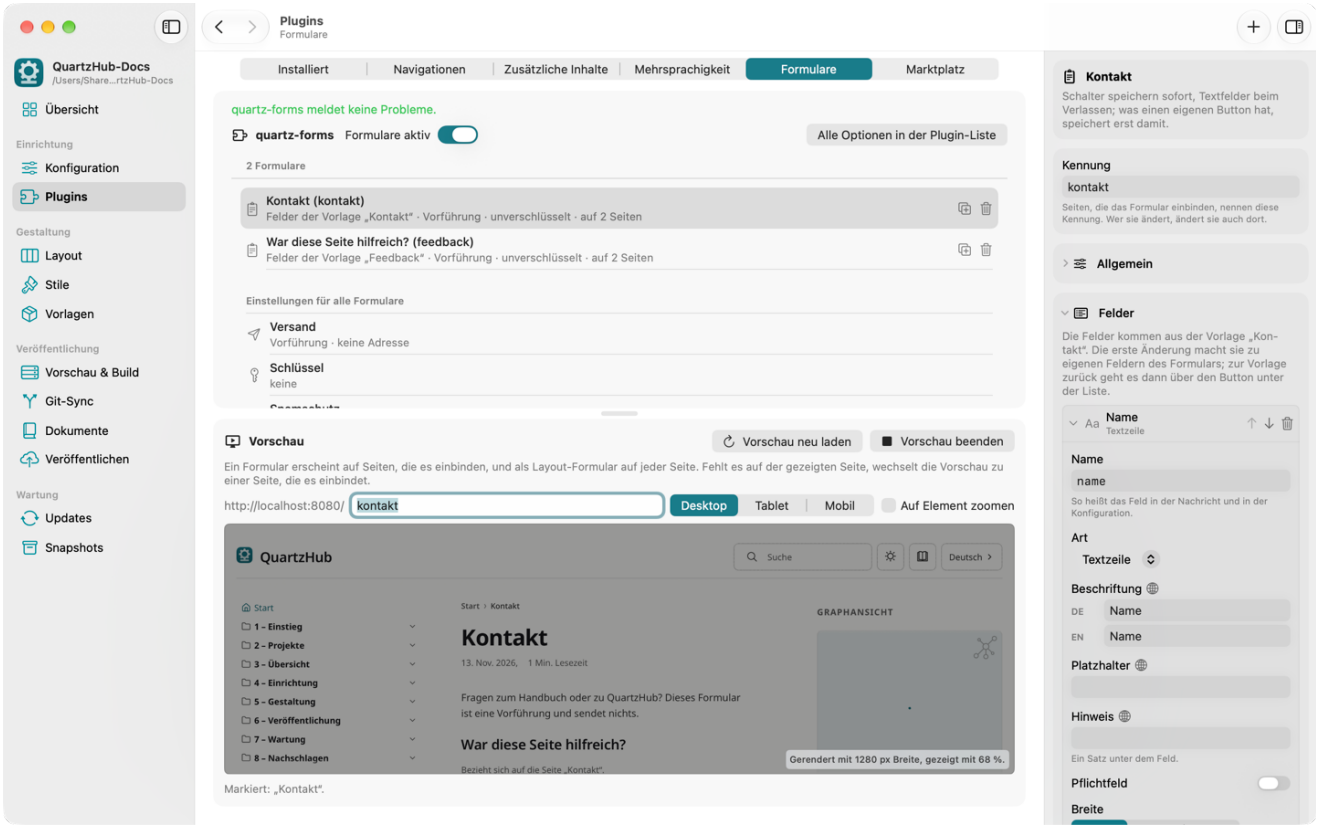
Die Liste und die Prüfung

Jedes Formular zeigt Titel und Kennung, Felder, Versandweg, Verschlüsselung und die Zahl der Seiten, dazu Zeichen wie **Layout** oder **anonym**. Kopieren legt eine Kopie mit der Kennung `…-copy` an; Entfernen sagt vorher, welche Seiten das Formular einbinden, denn die bauen danach nicht mehr.

Nach jeder Änderung lässt QuartzHub das Plugin seine Formulare prüfen. Über der Liste steht „quartz-forms meldet keine Probleme.“ oder die Zahl der Probleme; ein Problem bricht den Build ab. Die Meldungen stehen auch beim Formular und beim Bereich, den sie betreffen.

Ein Formular einstellen

Im Inspektor stehen die Teile eines Formulars zum Aufklappen, zuerst **Allgemein** und **Felder**. Ändert man die Kennung, fragt QuartzHub, wenn Seiten sie benutzen; die müssen dann angepasst werden.



Das Formular „Kontakt“ gewählt: rechts seine Felder, das erste aufgeklappt; die Vorschau zeigt eine Seite, die es einbindet.

Einstellungen für alle Formulare

Unter den Formularen stehen die gemeinsamen Bereiche. Jedes Formular kann einzelne Werte für sich ändern.

- **Versand**: wohin die Formulare senden: das **Relay**, ein eigener **Endpunkt**, **Formspree**, **Web3Forms**, das **E-Mail-Programm** der Besucher oder **Vorführung** (sendet nichts).
- **Schlüssel**: öffentliche PGP-Schlüssel, an die verschlüsselnde Formulare senden. Ein Schlüssel kommt aus einer Datei, die nach `quartz-forms/keys/` im Projekt kopiert wird, oder wird eingefügt. Private Schlüssel nimmt QuartzHub nicht an.
- **Spamschutz**: Fallen für Bots, eine Zeitfalle, eine Rechenaufgabe, ein Captcha-Anbieter und eine Grenze je Stunde. Den geheimen Schlüssel eines Captchas trägt man beim Relay ein.
- **Layout-Formular**: welches Formular auf jeder Seite steht und wo.
- **Texte**: die Wortlaute des Plugins, etwa für Fehlermeldungen, je Sprache ersetzbar. Grau steht der Wortlaut des Plugins; **Plugin-Wortlaut** setzt einen eigenen Text zurück.
- **Technisches**: woran das Plugin eine Einbindung erkennt.
- **Relay**: nur, wenn ein Formular über das Relay sendet; siehe unten.

Das Relay

Das Relay ist ein PHP-Skript (`send.php` mit seiner Konfiguration), das die Nachrichten der Formulare als Mail weitergibt. Es liegt auf dem eigenen Webspace, braucht also einen Webspace mit PHP. QuartzHub lässt es vom Plugin erzeugen und lädt es über ein Ziel des Projekts hoch.

Das braucht das Relay:

1. **Ziel**: ein Veröffentlichungsziel des Projekts vom Typ SFTP, FTP oder Ordner (6.7). Das Relay geht in einen **Unterordner** seines Pfads, etwa `forms`, und löscht dort nie etwas. Überträgt das Website-Ziel per rsync und löscht, was der Build nicht hat, bietet QuartzHub an, den Unterordner dort auszuschließen.
2. **Adresse von send.php**: wohin die Formulare senden. **Vorschlag übernehmen** setzt sie aus Basis-URL und Unterordner zusammen.
3. **Absender**: eine Adresse der Domain des Webspace, sonst weisen Mailserver die Nachrichten ab.
4. **Empfänger**: die Formulare nennen Empfänger beim Namen, etwa „kontakt“; hier bekommt jeder Name seine Adresse.
5. **Captcha-Secret**, wenn ein Formular ein Captcha benutzt.

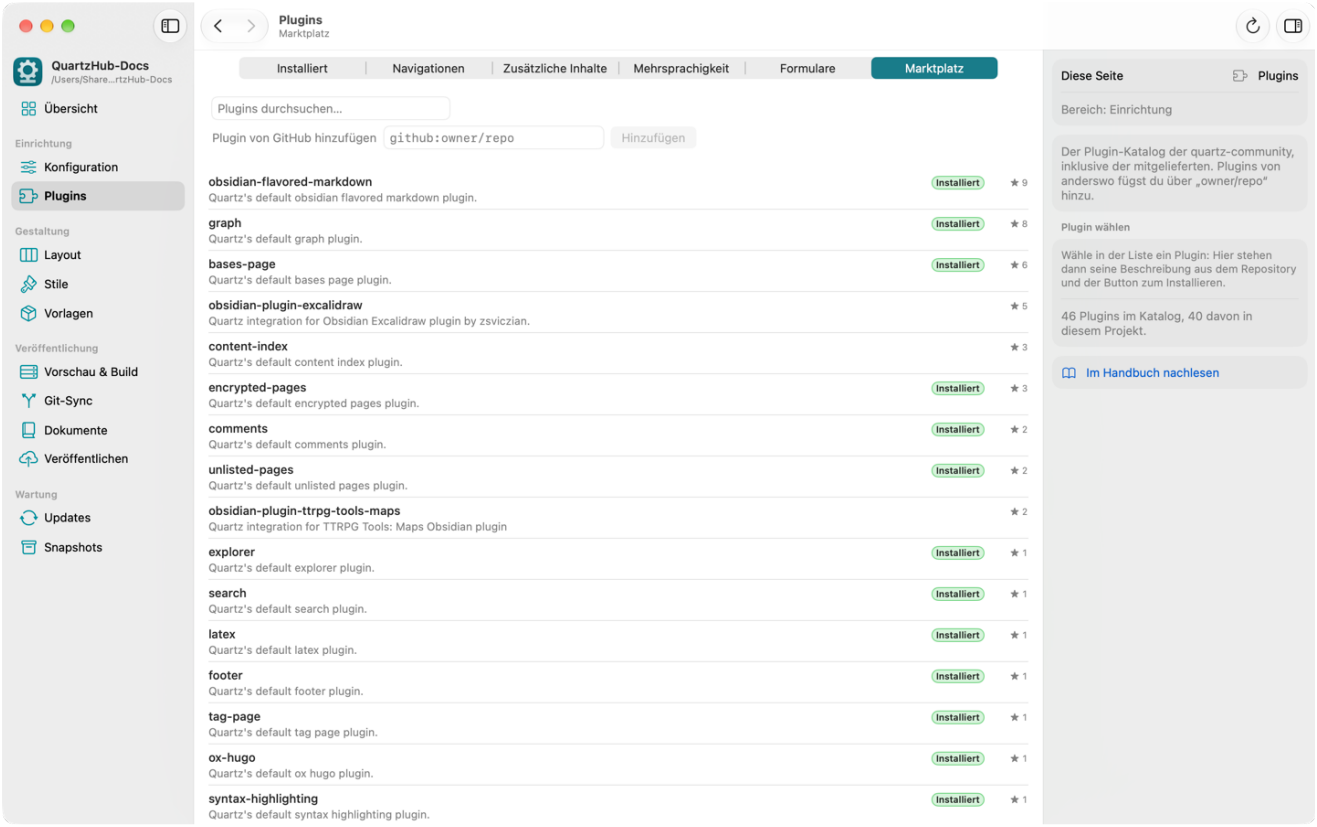
Optional: **Erlaubte Origins** (nur Anfragen von diesen Adressen nimmt das Relay an), der **Versand der Mails** über PHP `mail()` oder sendmail und unter **Selten gebraucht** Betreff-Präfix, Ordner für Zähler und ein neues Secret.

Relay hochladen lädt es hoch und fragt danach selbst nach, ob es antwortet; eine Mail schickt diese Probe nicht. Die Antwort steht darunter, etwa „Das Relay antwortet unter …“, oder was nicht stimmt: kein PHP, eine Weiterleitung, eine abgewiesene Herkunft, eine unlesbare Konfiguration. Oben im Bereich steht, ob das Relay seit dem Hochladen geändert wurde. **Als Ordner speichern …** schreibt es in einen Ordner, etwa für einen Webspace, den man anders befüllt; **Vom Server entfernen …** nimmt es wieder herunter.

> ⚠ **Wo die Adressen liegen**
>
> Empfänger-Adressen und Secrets des Relays legt QuartzHub im Schlüsselbund ab, nie im Projektordner und nie auf der Website. Auf einem anderen Mac fehlen sie deshalb; der Bereich sagt das, und man trägt sie neu ein. Wer das Projekt aus der Liste entfernt, löscht sie (2.1). In der hochgeladenen `config.php` und in einem gespeicherten Ordner stehen sie im Klartext.

4.10 – Marktplatz

Plugins ▸ Marktplatz zeigt den Katalog der **quartz-community** auf GitHub: Quartz' eigene Plugins und die anderer Leute.



Der Marktplatz, nach Sternen sortiert. Installierte Plugins sind markiert.

Der Katalog

QuartzHub lädt den Katalog beim ersten Öffnen des Reiters von GitHub und hält ihn eine Viertelstunde. **Katalog neu laden** in der Symbolleiste holt ihn sofort neu.

Jede Zeile zeigt Name, Beschreibung, Sterne und, falls zutreffend, **Installiert** oder **Archiviert**. Oben stehen die Repositories, die sich als Quartz-Plugin ausweisen, nach Sternen sortiert. Alles andere der quartz-community, etwa Quartz selbst, Bibliotheken und Vorlagen, steht eingeklappt darunter; das meiste davon ist kein Plugin. Das Suchfeld sucht in Name und Beschreibung.

GitHub beantwortet ohne Anmeldung nur eine begrenzte Zahl von Anfragen je Stunde. Ist sie erschöpft oder antwortet GitHub nicht, sagt der Reiter das, und unten steht nur ein Platzhalter. Ein persönlicher Zugriffstoken für GitHub hebt die Grenze an; QuartzHub benutzt den aus den Einstellungen (2.5).

Ein Plugin ansehen

Ist ein Plugin ausgewählt, zeigt der Inspektor seine Beschreibung aus dem Repository (die README), einen Link zu GitHub und **Installieren**. Bilder der README stehen dort nur als Text; auf GitHub sieht man sie vollständig.

Installieren

Installieren legt einen Snapshot an und lässt dann Quartz das Plugin hinzufügen: Quartz lädt es herunter, trägt es in `quartz.config.yaml` ein und hält die Fassung in `quartz.lock.json` fest. Danach steht es unter **Installiert**, mit seinen Vorgaben. Dafür braucht es eine Verbindung ins Netz.

Plugin von GitHub hinzufügen installiert ein Plugin, das nicht im Katalog steht. Man gibt `github:besitzer/repository` ein und drückt **Hinzufügen**. QuartzHub reicht die Angabe unverändert an Quartz weiter; was Quartz sonst noch annimmt, beschreibt die Quartz-Dokumentation.

> ⚠ **Plugins sind Programme**
>
> Ein Plugin läuft bei jedem Build mit den Rechten des eigenen Kontos. Installiere nur, was aus einer Quelle kommt, der du traust.

5 – Gestaltung

Die Gruppe **Gestaltung** hat drei Seiten. **Layout** legt fest, was wo auf der Seite steht. **Stile** legt fest, wie es aussieht: Farben, Schriften, CSS. **Vorlagen** packt eine fertige Gestaltung in eine Datei, die man in ein anderes Projekt übernimmt.

1. **Die Ebenen**: in welcher Reihenfolge Basis, Theme, Variablen und eigenes CSS einander überschreiben. Wer das einmal gelesen hat, sucht nicht mehr, warum eine Farbe nicht wirkt.

Layout

2. **Layout-Übersicht**: welche Komponente auf welchem Seitentyp erscheint.
3. **Komponenten**: die Bühne, auf der man Komponenten verschiebt.
4. **Frames**: eigene Seitenraster bauen.

Stile

5. **Basis**: Grundfarben, Schriften und wie sie ausgeliefert werden.
6. **Community-Themes**: fertige Farbschemata.
7. **Variablen**: einzelne CSS-Variablen gezielt ändern.
8. **Eigenes CSS**: Stylesheets selbst schreiben.

Vorlagen

9. **Vorlagen**: eine Gestaltung als Datei sichern und anwenden.

5.1 – Die Ebenen

Das Aussehen einer Quartz-Website entsteht aus mehreren Schichten, die übereinander liegen. Was weiter oben liegt, gewinnt. Die vier Reiter unter **Stile** sind diese Schichten, in dieser Reihenfolge:

Ebene	Wo	Was sie festlegt
1. Basis	**Stile ▸ Basis**	die Grundfarben für hell und dunkel und die drei Schriften, aus der Konfiguration
2. Community-Theme	**Stile ▸ Community-Themes**	ein fertiges Farbschema; überschreibt die Basis
3. Variablen	**Stile ▸ Variablen**	einzelne CSS-Variablen; gewinnen gegen das Theme
4. Eigenes CSS	**Stile ▸ Eigenes CSS**	eigene Regeln, für alles, was die anderen Ebenen nicht können

Rechts im Inspektor steht dieselbe Reihenfolge, dazu, ob ein Theme aktiv ist und wie viele Variablen überschrieben sind.

Was daraus folgt

Ist ein Community-Theme aktiv, wirken viele Basis-Farben nicht. Stile ▸ Basis zeigt das: oben steht, wie viele Farben und Schriften das Theme überschreibt, und diese Felder sind blasser. Sie gelten wieder, sobald das Theme aus ist.

Gegen ein Theme ändert man eine Farbe unter Variablen. Die Werte dort gewinnen, auch wo das Theme dieselbe Variable setzt. Ausnahme sind Farben, die ein Theme nur für eine Art von Element setzt, etwa für Callouts; die ändert man im eigenen CSS.

Eigenes CSS ist die letzte Möglichkeit, nicht die erste. Eine Farbe, die es als Variable gibt, ändert man besser unter Variablen: Sie gilt dann überall, wo die Variable benutzt wird, in hell und dunkel getrennt.

Innerhalb des eigenen CSS

Quartz bindet nur eine Datei ein, `quartz/styles/custom.scss`. Alles andere lädt diese Datei, in der Reihenfolge, die unter **Eigenes CSS** rechts unter **Ladereihenfolge** steht:

1. die eigenen Stylesheets in `quartz/styles/custom/` und `imported/`, in ihrer Reihenfolge;
2. `_quartzcontrol.scss`: die Schriften und Variablen, die QuartzHub schreibt;
3. `custom.scss` selbst, immer zuletzt.

Später Geladenes gewinnt bei gleich genauen Regeln. Eine Regel in `custom.scss` schlägt deshalb einen Wert unter **Variablen**; die Seite Variablen warnt, wenn `custom.scss` dieselbe Variable setzt. Eine Datei unter `custom/` dagegen lädt vor den Variablen und verliert gegen sie.

> **Warum die Variablen gegen das Theme gewinnen**
>
> Quartz und die Themes legen ihre Regeln in CSS-Ebenen (`@layer`). Was ein Projekt selbst schreibt, steht in keiner Ebene, und eine Regel außerhalb aller Ebenen gewinnt gegen jede Regel darin, egal wie genau diese ist.

Und das Layout?

Das Layout ist keine dieser Ebenen. Es legt fest, welche Komponenten wo stehen und wie das Raster der Seite aussieht (5.2); die Stile legen fest, wie es aussieht.

5.2 – Layout-Übersicht

Drei Begriffe

Komponenten sind die sichtbaren Teile einer Seite: Suche, Inhaltsverzeichnis, Graph, eine Navigation, eine Box. Jede ist ein Plugin-Eintrag (4.5) mit einer **Position**: Kopfbereich, linke oder rechte Seitenleiste, vor oder nach dem Inhalt, Fußzeile. Komponenten werden einmal für alle Seiten platziert.

Seitentypen sind die Arten von Seiten, die Quartz baut: Inhaltsseiten (die Notizen), Ordnerseiten, Tag-Seiten, die 404-Seite und je nach Plugins weitere wie Bases- oder Canvas-Seiten. Jeder Seitentyp kann vom Rest abweichen: Komponenten ausblenden, Bereiche leeren, einen eigenen Frame benutzen.

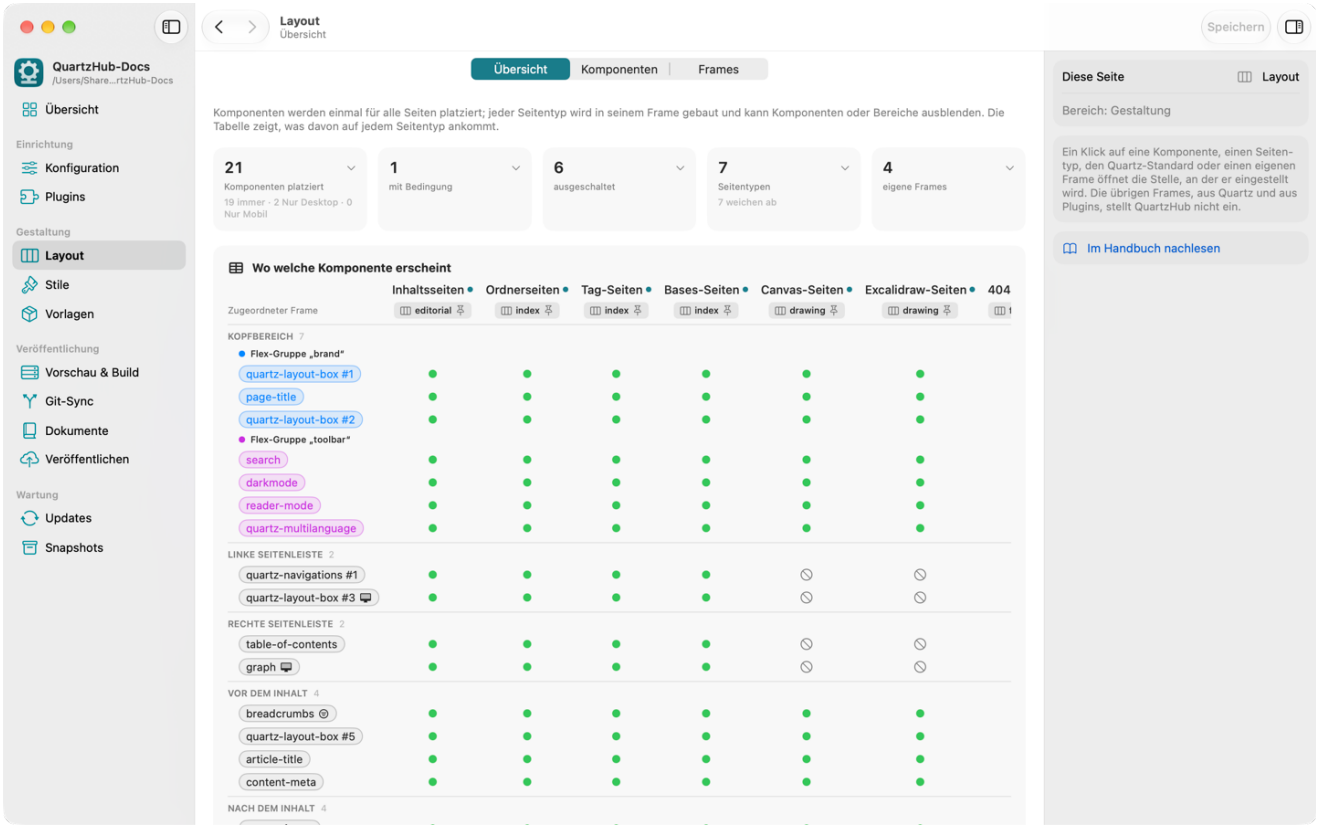
Ein **Frame** ist das Raster einer Seite: welche Bereiche es gibt, wo sie liegen und wie breit sie sind. Quartz bringt Standard, Volle Breite und Minimal mit; eigene baut man unter **Frames** (5.4).

Layout hat dafür drei Reiter: **Übersicht** zeigt alles, **Komponenten** ist die Bühne zum Verschieben, **Frames** der Editor für eigene Raster.

Speichern

Übersicht und Komponenten ändern die `quartz.config.yaml`, Frames einen Frame. Beides schreibt erst **Speichern** (⌘S), jeweils für den Reiter, der vorn ist. Vor dem Speichern der Konfiguration legt QuartzHub einen Snapshot an, höchstens einen alle 15 Minuten. Wer die Seite mit Ungespeichertem verlässt, wird gefragt.

Die Übersicht



Die Übersicht: oben die Zahlen, darunter die Tabelle.

Die Übersicht ändert nichts; sie zeigt und führt weiter. Ein Klick auf eine Komponente, einen Seitentyp oder einen eigenen Frame öffnet die Stelle, an der er eingestellt wird.

Die Zahlen oben öffnen je eine Liste darunter:

- **Komponenten platziert**, aufgeteilt nach immer, nur Desktop, nur Mobil;
- **mit Bedingung**: Komponenten, die nur auf manchen Seiten erscheinen, etwa nur mit Inhaltsverzeichnis;
- **ausgeschaltet**: Komponenten im Vorrat, die nicht gebaut werden;
- **Seitentypen**, und wie viele davon abweichen, mit dem, was jeweils ausgeblendet oder geleert ist;
- **eigene Frames**, und welche keinem Seitentyp zugeordnet sind.

Die Tabelle „Wo welche Komponente erscheint“ hat eine Spalte je Seitentyp und eine Zeile je Komponente, nach Position gruppiert. Die erste Zeile zeigt den Frame jedes Seitentyps; ein Punkt am Seitentyp heißt, dass er von „Alle Seiten“ abweicht. Jede Zelle sagt, ob die Komponente dort ankommt:

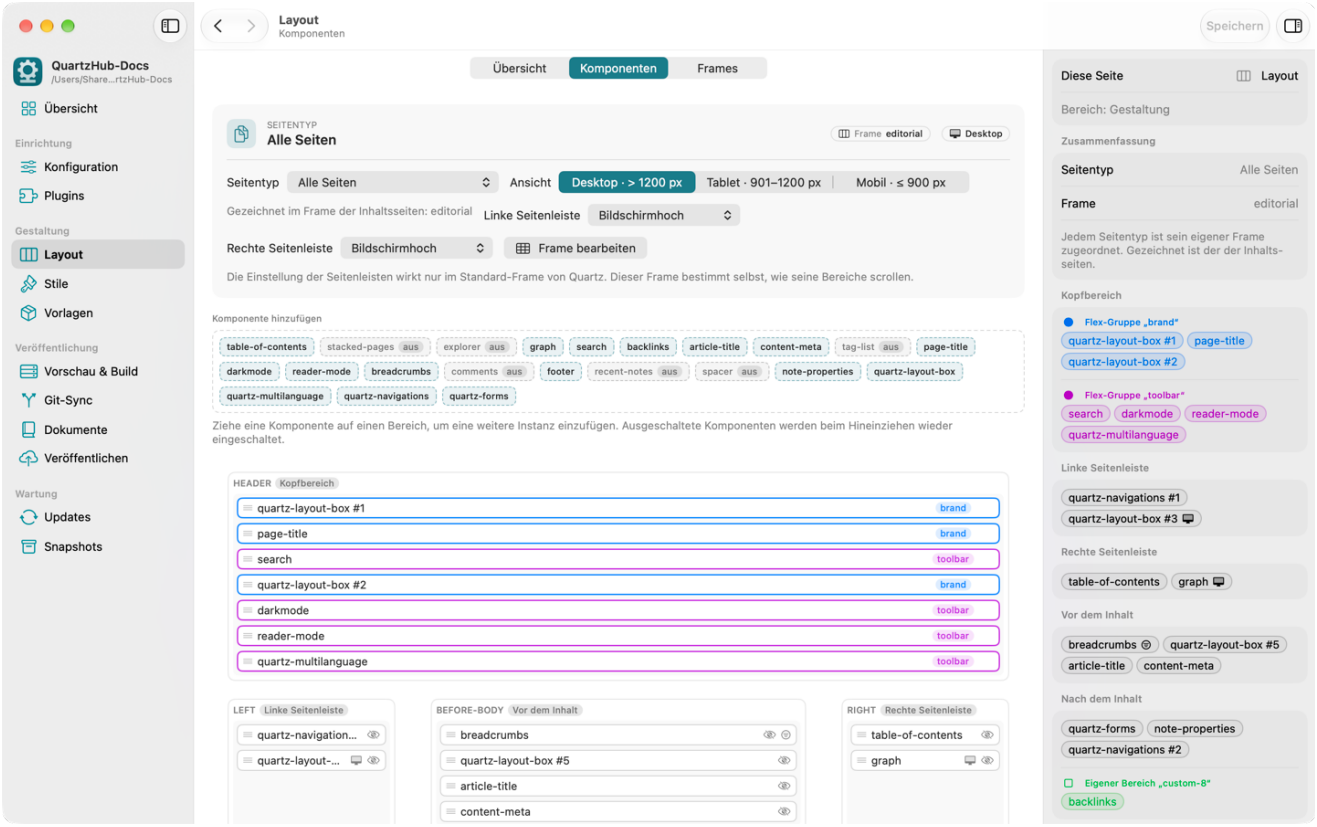
- **erscheint**, oder **nur auf** bestimmten Breiten;
- **ausgeblendet** auf diesem Seitentyp, oder ihr **Bereich geleert**;
- ihr Bereich ist **in diesem Frame nicht zugewiesen** oder auf keiner Breite gezeigt;
- **unbekannt**, wenn der Frame von einem Plugin kommt und QuartzHub seinen

Aufbau nicht kennt.

Den genauen Grund nennt der Tooltip einer Zelle. Die Legende steht unter der Tabelle.

5.3 – Komponenten

Layout ▸ Komponenten zeigt die Seite im Kleinen, gezeichnet im Frame des gewählten Seitentyps: oben der **Komponentenvorrat**, darunter die **Bühne** mit ihren Bereichen, unten die **Flex-Gruppen**. Alles hier geht in einen Entwurf; erst **Speichern** (⌘S) schreibt die Konfiguration (5.2).



Die Bühne für alle Seiten in der Desktop-Ansicht. Rechts die Zusammenfassung je Position.

Kopf

- **Seitentyp**: **Alle Seiten** oder ein bestimmter Seitentyp. Ein angepasster trägt „(angepasst)".
- **Ansicht**: **Desktop**, **Tablet** oder **Mobil**. Mit einem eigenen Frame stehen die Breiten dabei.
- **Frame** (nur bei einem Seitentyp): Standard, Volle Breite, Minimal, ein eigener Frame, oder der, den das Plugin des Seitentyps mitbringt.
- **Linke/Rechte Seitenleiste** (ab Quartz 5.1): was die Seitenleiste tut, wenn ihr Inhalt höher als der Bildschirm ist: **Bildschirmhoch** (die einzelnen Komponenten scrollen), **Scrollt selbst** oder **Scrollt mit der Seite**. Das wirkt nur im Standard-Frame von Quartz.
- **Frame bearbeiten** öffnet einen eigenen Frame im Editor. **Anpassung entfernen** setzt einen Seitentyp auf „Alle Seiten" zurück.

> **Was für alle gilt und was für einen Seitentyp**
>
> Ist ein Seitentyp gewählt, gelten nur **Ausblenden** und **Leeren** für ihn. Was man verschiebt oder im Inspektor einstellt, gilt für alle Seiten.

Verschieben

- **Ziehen** auf eine andere Komponente setzt die gezogene davor, auf den freien Rest eines Bereichs ans Ende.
- **Aus dem Vorrat ziehen** fügt eine weitere Instanz ein, etwa eine zweite Navigation. Eine ausgeschaltete Komponente wird dabei wieder eingeschaltet.
- **In den Vorrat ziehen** schaltet eine Komponente aus; ein Duplikat wird entfernt.
- Mit der **Tastatur** geht dasselbe im Inspektor: **Position**, **Früher** und **Später**.

QuartzHub nummeriert die Komponenten einer Position dabei neu (10, 20, 30 …), so dass ihre Reihenfolge in der Konfiguration der auf der Bühne entspricht. Ein Klick ins Leere oder Esc hebt die Auswahl auf.

Leeren (der Radiergummi an einem Bereich, nur bei einem Seitentyp) lässt einen Bereich auf diesem Seitentyp leer; ein zweiter Klick füllt ihn wieder.

Gestrichelte Komponenten stehen nicht auf der gezeigten Seite; der Tooltip sagt, warum.

Der Inspektor

Für eine gewählte Komponente:

- **Auf {Seitentyp} ausblenden** (nur bei einem Seitentyp). Gibt es mehrere Instanzen derselben Komponente, blendet das alle aus; Quartz kann einzelne Duplikate nicht getrennt ausschließen.
- **Position**, **Reihenfolge** mit **Früher** und **Später**.
- **Sichtbarkeit**: **Immer**, **Nur Desktop** oder **Nur Mobil**. „Nur Desktop“ gilt auch auf dem Tablet; Quartz schaltet erst an der Mobil-Grenze um.
- **Bedingung**: **Immer**, **Nicht auf der Startseite**, **Nur auf Seiten mit Tags**, **… mit Rückverweisen**, **… mit Inhaltsverzeichnis** oder eine **eigene**, die ein Plugin mitbringt.
- **Gruppe** und, in einer Gruppe, wie die Komponente darin Platz nimmt.
- **Duplizieren**, **In den Vorrat legen** oder **Duplikat entfernen**.
- **Optionen**: dasselbe Formular wie unter **Plugins ▸ Installiert**.

Ohne Auswahl zeigt der Inspektor eine **Zusammenfassung**: den Frame und je Position ihre Komponenten.

Flex-Gruppen

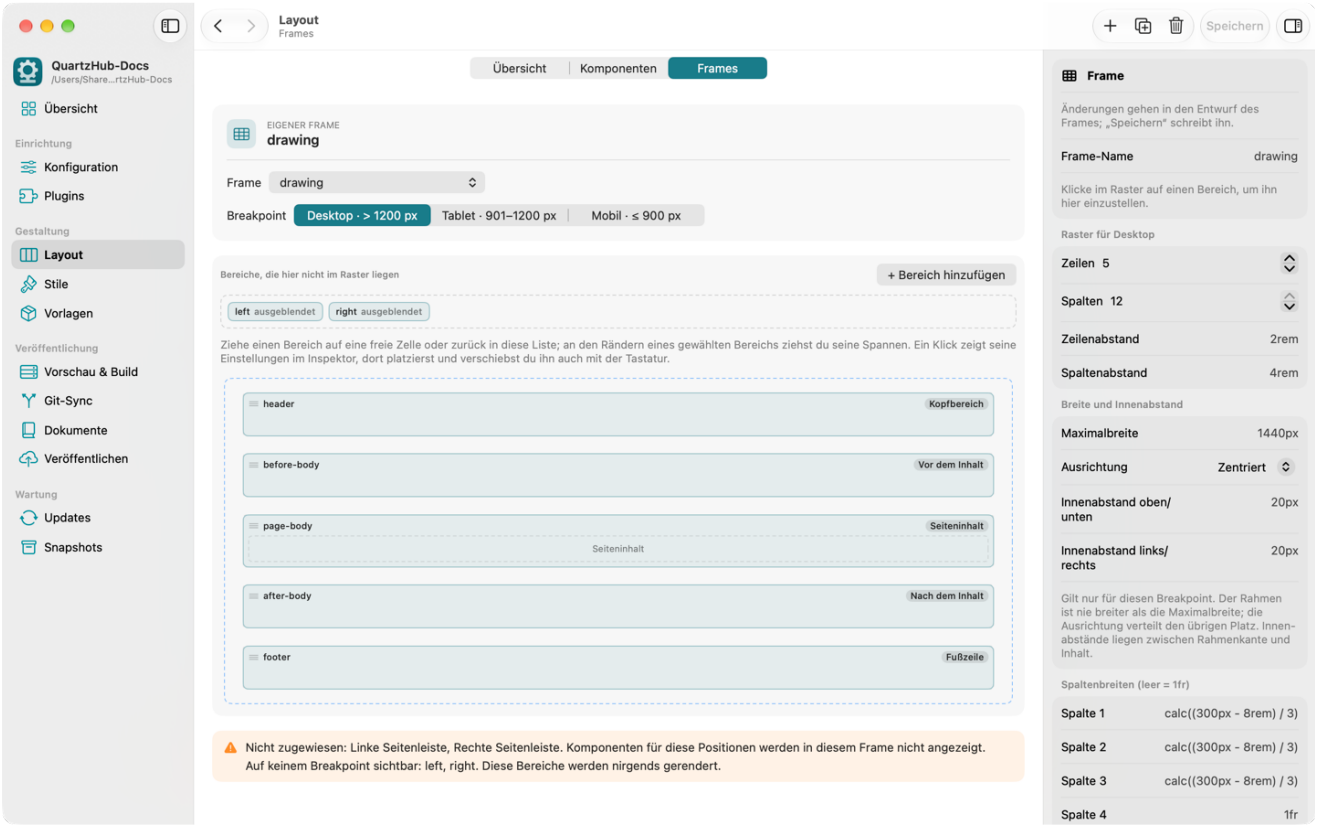
Komponenten derselben Gruppe stehen nebeneinander oder untereinander statt einzeln

untereinander, etwa Suche, Hell/Dunkel-Schalter und Sprachwechsler in einer Zeile im Kopf. Je Gruppe stellt man **Anordnung** (nebeneinander, untereinander, je auch rückwärts), **Umbruch**, **Abstand** und **Priorität** ein. Eine neue Gruppe legt man unten mit Namen und **Hinzufügen** an; Mitglied wird eine Komponente über **Gruppe** im Inspektor oder durch Ziehen in die Gruppe.

5.4 – Frames

Ein Frame ist das Raster einer Seite: welche Bereiche es gibt (Kopfbereich, Seitenleisten, Inhalt, Fußzeile), wo sie liegen und wie breit sie sind. Was in den Bereichen erscheint, legt man unter **Komponenten** fest; dort wählt man auch den Frame je Seitentyp (5.3).

Layout ▸ Frames baut eigene Frames, ohne CSS zu schreiben. Jeder Frame wird für Desktop, Tablet und Mobil getrennt eingerichtet.



Ein eigener Frame in der Desktop-Ansicht. Rechts die Einstellungen des Rasters.

Frames wählen und anlegen

Oben wählt **Frame** zwischen **Quartz-Standard** (nur ansehen) und den eigenen Frames, **Breakpoint** zwischen Desktop, Tablet und Mobil. In der Symbolleiste stehen:

- ➕ **Neuer Frame**: ein leerer Frame mit allen Bereichen, noch nicht platziert;
- **Duplizieren**: eine Kopie des gewählten Frames, „…-kopie“;
- **Löschen**: Seitentypen, die ihn benutzen, fallen auf den Standard-Frame zurück;
- **Speichern** (⌘S): schreibt den Frame.

Auf **Quartz-Standard** zeigt der Reiter, wie Quartz eine Seite ohne eigenen Frame aufteilt. **Als eigenen Frame anpassen** legt eine Kopie davon an, **Leeren Frame anlegen** einen leeren.

Das Raster

Über dem Raster liegen die **Bereiche, die hier nicht im Raster liegen**. **+ Bereich hinzufügen** legt einen neuen an.

- **Ziehen** eines Bereichs auf eine freie Zelle platziert ihn; zurück in die Liste nimmt ihn aus dem Raster, nur auf diesem Breakpoint. Wo er landen würde, zeigt ein Schatten; rot heißt, er würde einen anderen Bereich überdecken.
- An den **Rändern** eines gewählten Bereichs zieht man seine Spannen über mehrere Zeilen oder Spalten.
- Ein **Klick** zeigt seine Einstellungen im Inspektor. Ein Klick ins Leere oder Esc hebt die Auswahl auf.

Unter dem Raster stehen Warnungen, etwa wenn Bereiche übereinander liegen, über das Raster hinausreichen, eine Position keinem Bereich zugewiesen ist (ihre Komponenten erscheinen dann in diesem Frame nicht) oder ein Bereich auf keinem Breakpoint sichtbar ist.

Der Inspektor

Ohne Auswahl stellt er den Frame ein:

- **Frame-Name**.
- **Raster für {Breakpoint}**: **Zeilen** und **Spalten** (1 bis 12), **Zeilenabstand** und **Spaltenabstand**.
- **Breite und Innenabstand**: **Maximalbreite**, **Ausrichtung** und die Innenabstände. Ohne Maximalbreite nutzt der Frame die ganze Fensterbreite.
- **Spaltenbreiten** (leer = gleiche Anteile) und **Zeilenhöhen** (leer = so hoch wie der Inhalt), als CSS-Werte wie `16rem` oder `1fr`.
- **Benannte Grid-Lines**, wenn man die Linien später in eigenem CSS ansprechen will.
- **Auf {Breakpoint} kopieren**: übernimmt das ganze Raster dieses Breakpoints auf einen anderen, ohne Rückfrage.

Mit einem Bereich stellt er diesen ein:

- **Bereichsname**: wird zu einem CSS-tauglichen Namen aus Kleinbuchstaben, Ziffern und Bindestrichen; weicht er vom Getippten ab, steht darunter, als was er gespeichert wird.
- **Belegung**: welche Position der Bereich zeigt, oder keine.
- **Eigener Bereich**: Der Bereich zeigt dann nur die Komponenten, die man unter **Komponenten** hierher zieht, nicht alles, was zu seiner Position gehört. So bekommt eine Position zwei Bereiche, etwa eine Seitenleiste oben und unten.
- **Zeilen-Spanne**, **Spalten-Spanne** und **Sichtbar auf {Breakpoint}**.
- **Verschieben nach** Zeile und Spalte: dasselbe wie Ziehen, mit der Tastatur.
- **Aus Raster lösen** (nur dieser Breakpoint) und **Bereich löschen**.

Breakpoints

Unter **Breakpoints** stellt man ein, ab welcher Fensterbreite eigene Frames auf Tablet und Mobil umschalten. Vorgabe ist 1200 und 800 Pixel, wie bei Quartz. Das eigene **Speichern** dieses Abschnitts schreibt sofort und gilt für alle eigenen Frames. Quartz' eingebaute Frames und Community-Themes bringen eigene Werte mit.

Was beim Speichern passiert

Ein eigener Frame ist für Quartz ein Plugin: ein Ordner unter `.quartz-gui/authored-frames/` mit der Beschreibung des Rasters und dem erzeugten CSS. Ein neuer Frame wird beim ersten Speichern bei Quartz angemeldet, vorher legt QuartzHub einen Snapshot an. Vor jedem Build und jedem Start der Vorschau schreibt QuartzHub alle Frames neu.

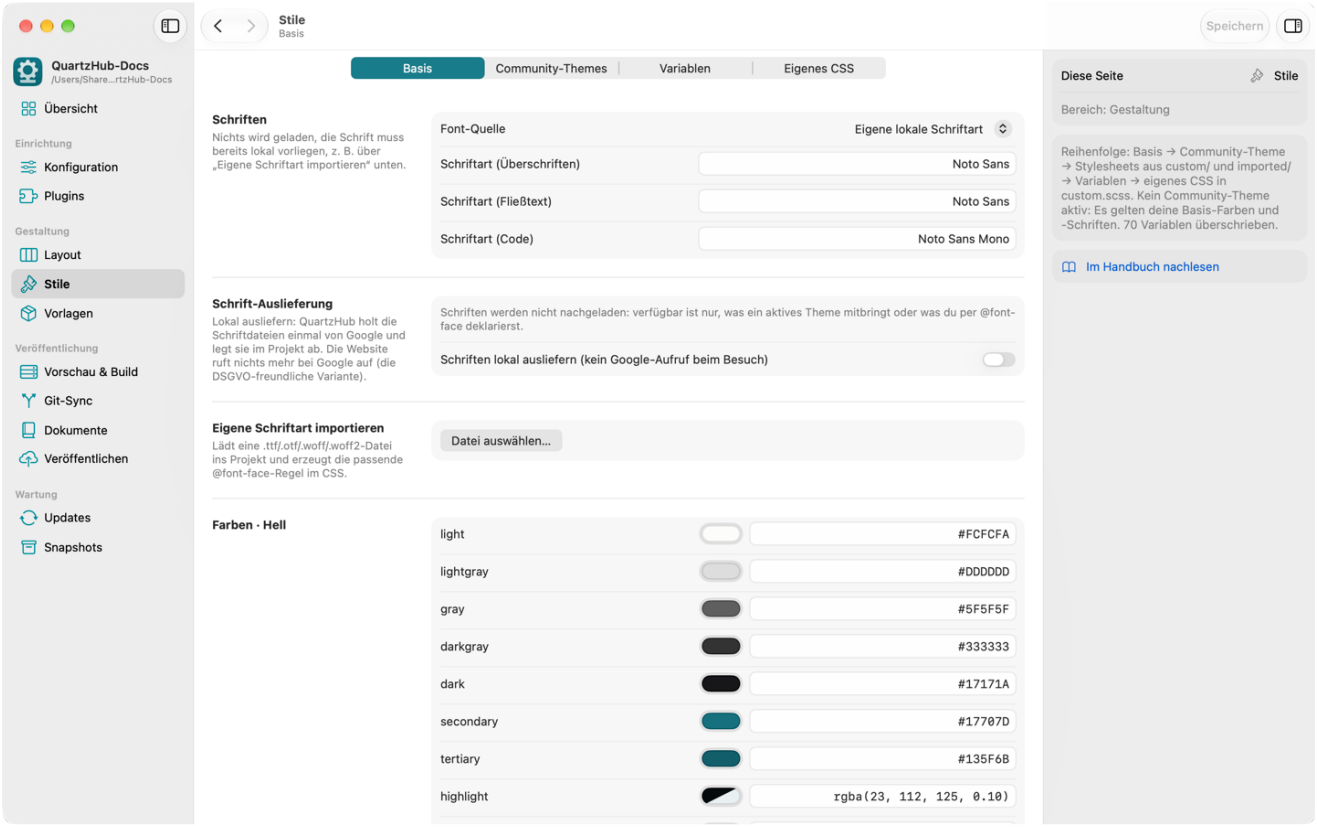
Läuft die Vorschau, sieht man eine Änderung an einem vorhandenen Frame erst nach einem Neustart; der Reiter bietet **Dev-Server neu starten** an.

Seitentypen nennen ihren Frame beim Namen. Wer einen Frame umbenennt, dem zieht QuartzHub beim Speichern die Seitentypen nach, die ihn benutzen.

Eigene Frames kommen nicht über git mit (3.1).

5.5 – Stile ▸ Basis

Stile ▸ Basis ist die unterste Ebene (5.1): die Farben und Schriften, die in der `quartz.config.yaml` stehen.



Schriften, Schrift-Auslieferung und die Farben für den hellen Modus.

Speichern

Die vier Reiter unter **Stile** teilen sich ein **Speichern** (⌘S): Was man auf einem Reiter ändert, bleibt beim Wechsel auf einen anderen erhalten und wird mit ihm gespeichert. Vor dem Schreiben der Konfiguration legt QuartzHub einen Snapshot an, höchstens einen alle 15 Minuten. Einiges wirkt sofort, ohne Speichern; das steht jeweils dabei.

Wenn ein Theme aktiv ist

Ist ein Community-Theme aktiv, sagt ein Hinweis oben, wie viele Farben und Schriften es überschreibt. Diese Felder sind blasser; ändern lassen sie sich trotzdem, sie gelten dann, sobald das Theme aus ist. Gegen das Theme ändert man Werte unter **Variablen** (5.7).

Schriften

- **Font-Quelle**: **Google Fonts** oder **Eigene lokale Schriftart**. Mit Google bieten die

Felder alle Google-Schriften zur Auswahl; beim Tippen wird gefiltert. Lokal wird nichts geladen; die Schrift muss im Projekt liegen, etwa über **Eigene Schriftart importieren**.
- **Schriftart (Überschriften)**, **(Fließtext)** und **(Code)**: die Schrift für Überschriften, Text und Code.

Unter einem Feld steht, wenn etwas nicht stimmt: Google kennt den Namen nicht oder schreibt ihn anders, das Theme überschreibt die Schrift, oder eine Variable im Projekt tut es; dann ändert das Feld auf der Website nichts.

Schrift-Auslieferung

Wo die Schriften für die Besucher herkommen, ist eine Frage des Datenschutzes: Lädt die Website sie bei Google, erfährt Google von jedem Besuch.

- **Schriften lokal ausliefern (kein Google-Aufruf beim Besuch)**: Beim Speichern holt QuartzHub die Schriftdateien einmal von Google und legt sie nach `quartz/static/fonts/`. Die Website ruft danach nichts mehr bei Google auf. Ausgeschaltet entfernt das Speichern die lokalen Kopien wieder.
- Darüber steht, wer im Projekt gerade Schriften lädt: Quartz selbst, QuartzHub, das Plugin „Fonts“ oder das Theme. Was Google direkt verlinkt, steht in Orange.
- **Jetzt ins Projekt holen** speichert die Seite sofort und holt dabei die Schriften.
- **Theme-eigene Schriften laden** (nur mit Theme): Ein Theme lädt seine Schriften von unpkg.com; lokal ausliefern kann QuartzHub sie nicht. Ausgeschaltet gelten die eigenen Schriften.

Liefert das Plugin „Fonts“ die Schriften selbst aus, braucht es eine Basis-URL (4.1).

Bekannte Konflikte

Mit dem Plugin „Fonts“ und einem Theme zusammen folgen die Überschriften dem Theme nicht. **Fix anlegen** legt sofort eine kleine Datei `custom/fix-heading-fonts.scss` an, die das behebt; sie steht danach unter **Eigenes CSS** wie jede andere.

Eigene Schriftart importieren

Datei auswählen… nimmt eine `.ttf`-, `.otf`-, `.woff`- oder `.woff2`-Datei, kopiert sie sofort nach `quartz/static/fonts/` und schreibt die passende `@font-face`-Regel. **Direkt verwenden für** setzt sie gleich als Schrift für Überschriften, Text oder Code; das gilt erst nach dem Speichern.

Ungenutzte Schriften listet importierte Schriften, die nichts benutzt. Entfernen löscht Regeln und Dateien sofort, ohne Snapshot.

Farben

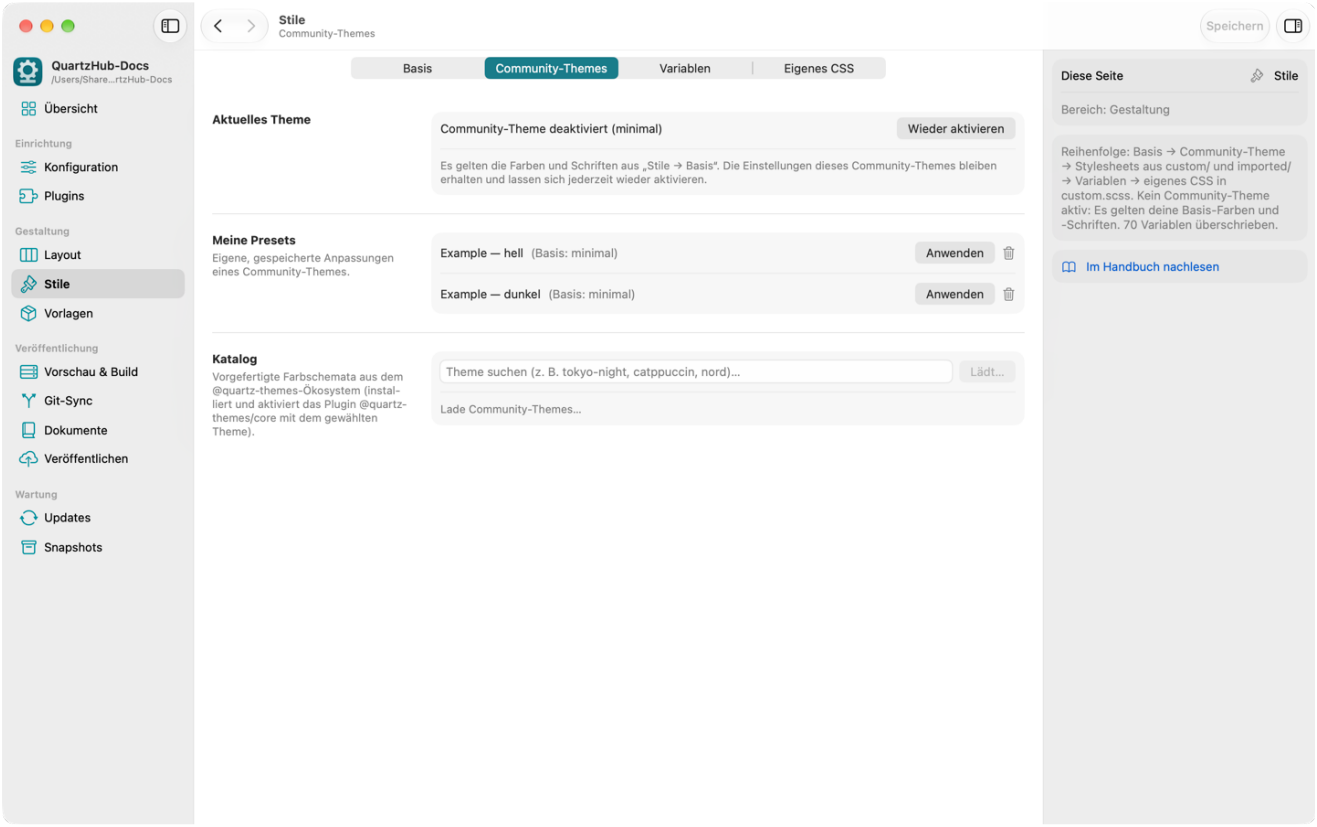
Je Modus, hell und dunkel, die Grundfarben mit ihren Namen aus Quartz:

Name	Wofür Quartz sie benutzt
`light`	Hintergrund
`lightgray`	Linien und Rahmen
`gray`	stärkere Rahmen, Verbindungen im Graph
`darkgray`	Fließtext
`dark`	Überschriften und Symbole
`secondary`	Links, die aktuelle Seite im Graph
`tertiary`	Hover, besuchte Seiten im Graph
`highlight`	Hintergrund interner Links
`textHighlight`	markierter Text (`==so==`)

Jede hat ein Farbfeld und ein Textfeld für jeden CSS-Farbwert. Was ein Theme überschreibt, trägt das Zeichen **THEME**.

5.6 – Community-Themes

Community-Themes sind fertige Farbschemata, viele davon nach Themes für Obsidian. Sie kommen über das Plugin `@quartz-themes/core` und legen sich über die Basis (5.1).



Ein ausgeschaltetes Theme, zwei Presets und der Katalog.

Ein Theme wählen

Katalog listet die Themes, nach Sternen auf GitHub; das Suchfeld filtert, etwa nach „tokyo-night", „catppuccin" oder „nord". Ein Klick zeigt ein Theme im Inspektor: Beschreibung, Modi, Varianten, ob es eigene Farben anpassen lässt. **Installieren & aktivieren** installiert es sofort und setzt es im Entwurf; erst **Speichern** schreibt es in die Konfiguration.

Die Liste kommt von npm und wird eine Viertelstunde gehalten; ohne Netz stehen nur einige bekannte Themes als Platzhalter da.

Das aktuelle Theme

Oben steht, welches Theme aktiv ist. **Community-Themes deaktivieren** schaltet es aus und behält seine Einstellungen; **Wieder aktivieren** schaltet es ein. Ohne Theme gelten die Farben und Schriften aus **Basis**.

Einstellungen des Themes

Viele Themes bringen eigene Einstellungen mit, etwa eine Akzentfarbe oder die Art der Überschriften. QuartzHub zeigt sie mit Beschriftung und Beschreibung aus dem Original-Theme, in Gruppen und mit Suchfeld; **Zurücksetzen** an einer Option nimmt sie zurück. Die Beschreibungen holt QuartzHub einmal von GitHub und hebt sie auf; **Doku neu laden** holt sie neu, **Einstellungen ▸ Daten & Wartung** leert alle (2.5). Gibt es keine Beschreibungen, stehen nur die rohen Schalter da.

Was ein Theme nicht als Einstellung anbietet, ändert man unter **Variablen**.

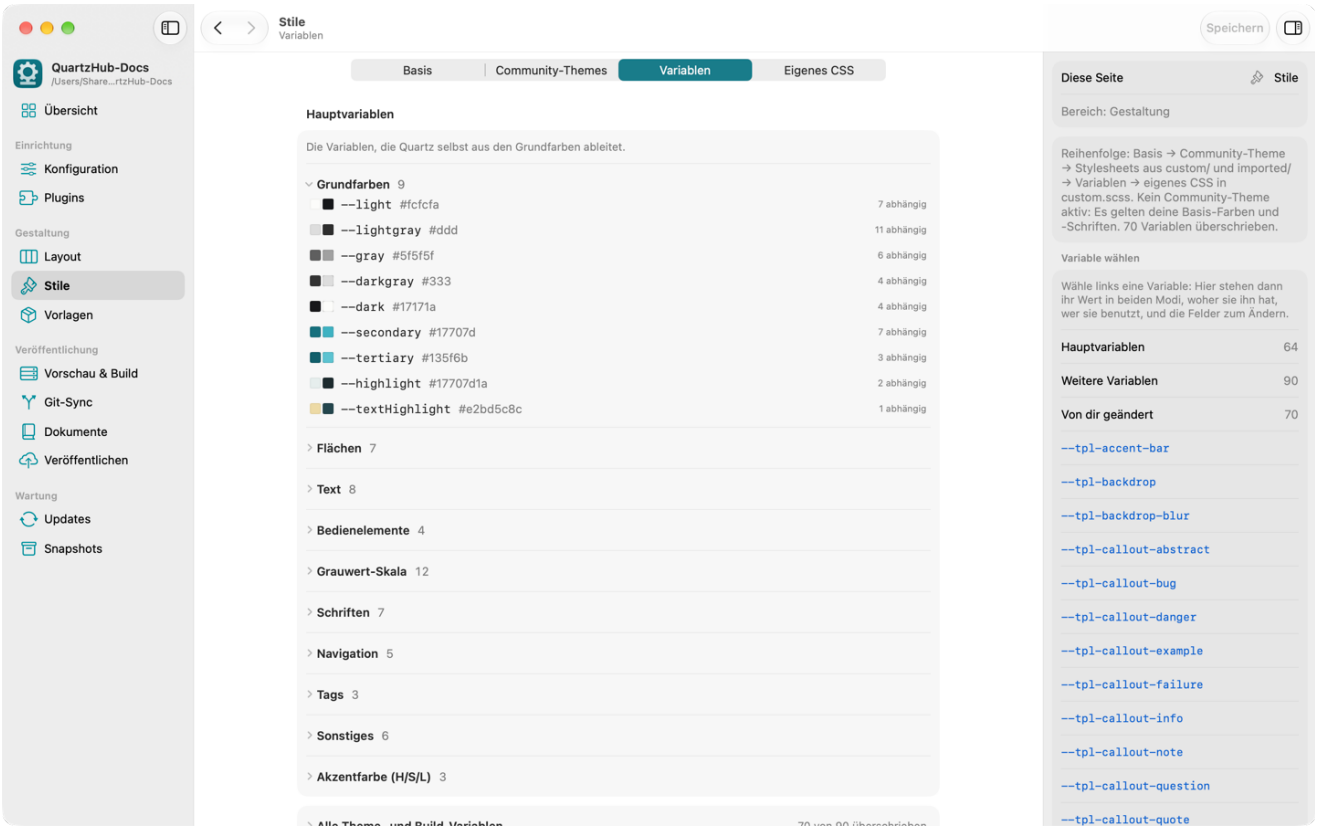
Presets

Als Preset speichern sichert das aktuelle Theme mit seinen Einstellungen unter einem Namen, sofort, in `.quartz-gui/theme-presets.json`. **Meine Presets** listet sie; **Anwenden** installiert das Theme des Presets und übernimmt seine Einstellungen in den Entwurf. Ein gelöschtes Preset ist weg.

> **Themes laden Schriften von außen**
>
> Ein Theme lädt seine Schriften von unpkg.com. Wer das nicht will, schaltet unter **Basis ▸ Schrift-Auslieferung** die Theme-eigenen Schriften aus (5.5).

5.7 – Variablen

Fast jede Farbe, Schrift und viele Abstände einer Quartz-Website kommen aus CSS-Variablen wie `--secondary` oder `--bodyFont`. **Stile ▸ Variablen** zeigt sie alle, sagt, woher jede ihren Wert hat und wer sie benutzt, und ändert sie gezielt. Werte hier gewinnen gegen das Theme (5.1).



Die Hauptvariablen, aufgeklappt die Grundfarben. Rechts, welche geändert sind.

Die Liste

- **Hauptvariablen**: die Variablen, die Quartz aus den Grundfarben ableitet, in Gruppen wie Grundfarben, Flächen, Text, Navigation, Schriften. Eine Gruppe zeigt, wie viele darin geändert sind.
- **Alle Theme- und Build-Variablen**: alles, was das Theme, Plugins oder der letzte Build mitbringen. Die Gesamtzahl kennt QuartzHub erst nach einem Build. Ein Suchfeld (etwa „callout“, „h1“, „background“), **Nur geänderte**, ein Filter nach Farben und anderen Werten und **Neu einlesen** helfen beim Finden.

Jede Zeile zeigt die Farbe für hell und dunkel, den Namen, den Wert, wie viele andere Variablen von ihr abhängen, und woher sie kommt: **geändert**, **eigene Variable** oder **Community-Theme**.

Eine Variable ansehen

Ein Klick zeigt die Variable im Inspektor:

- **Aktueller Wert**, für hell und dunkel, mit dem Farbwert.
- **Herkunft**: Quartz, das Theme, ein Stylesheet oder der letzte Build, mit der Kette, über die der Wert zustande kommt, in der Form `--a → --b → Wert`.
- **Verwendet diese Variablen** und **Wird von diesen Variablen verwendet**: ein Klick springt dorthin. Darunter steht, auf wie viele weitere Variablen sich eine Änderung auswirkt.

Eine Variable ändern

Unter **Eigener Wert** trägt man den neuen Wert ein, bei Farben auch über ein Farbfeld. **Im Dunkelmodus abweichend** trennt hell und dunkel. **Auf Originalwert zurücksetzen** nimmt die Änderung zurück. Eine Variable, die es nur im eigenen Projekt gibt, lässt sich entfernen; jede Stelle, die sie benutzt, hat danach keinen Wert mehr.

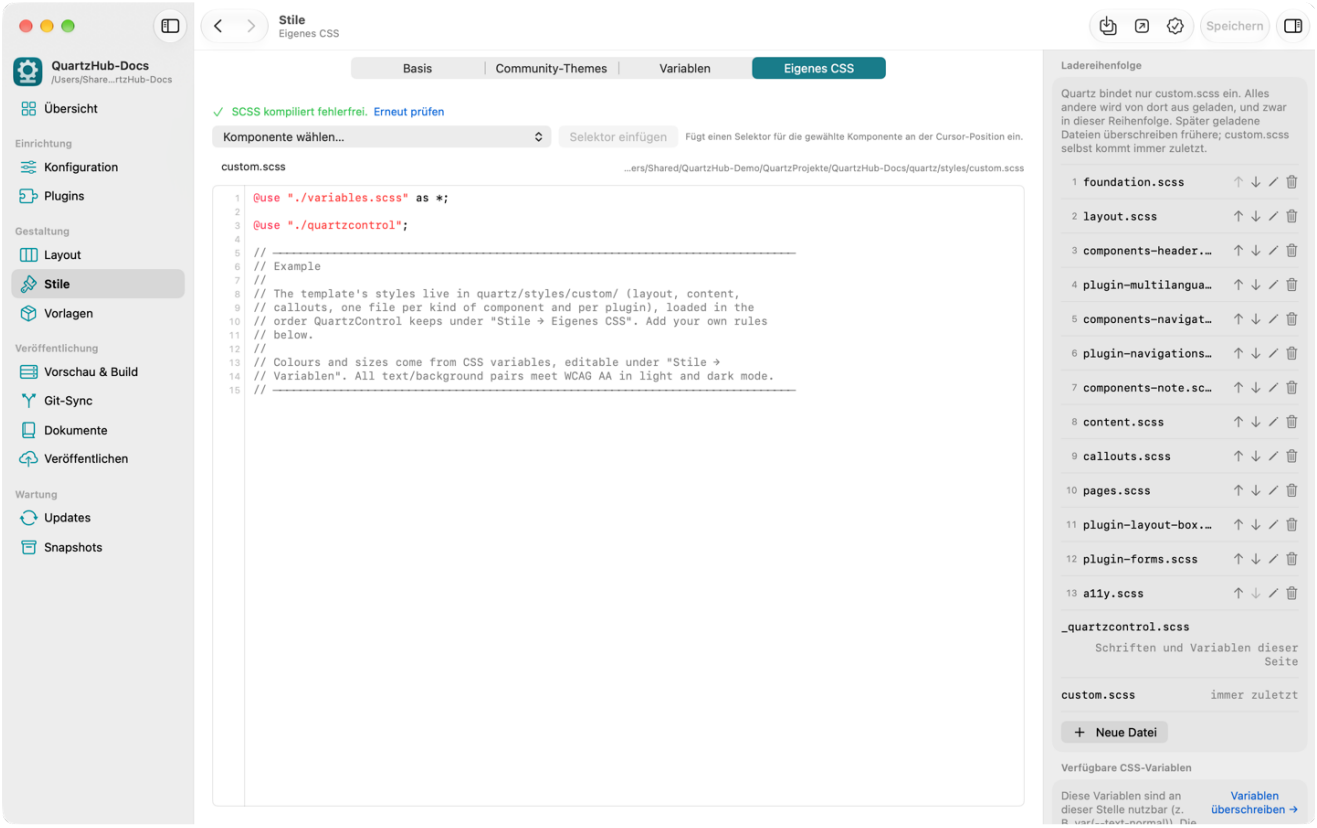
Gespeichert wird mit **Speichern** (⌘S). QuartzHub schreibt die Werte in einen eigenen Abschnitt von `quartz/styles/_quartzcontrol.scss`. Ein Wert darf kein Semikolon, keine geschweifte Klammer, keinen Backslash und keinen Kommentar enthalten; sonst speichert QuartzHub gar nichts und sagt, welcher Wert es ist.

> **Lieber die Quelle als die Ableitung**
>
> Viele Variablen leiten sich von wenigen ab. Wer `--secondary` ändert, ändert alle Links, Hervorhebungen und was davon abhängt. Die Zeile „Ein eigener Wert hier wirkt sich auf … weitere Variablen aus“ zeigt, wie weit eine Änderung reicht.

Setzt `custom.scss` dieselbe Variable selbst, gewinnt sie dort; ein Hinweis oben sagt das.

5.8 – Eigenes CSS

Stile ▸ Eigenes CSS ist die oberste Ebene (5.1): eigene Regeln in SCSS oder CSS, für alles, was Basis, Theme und Variablen nicht können.



custom.scss im Editor, rechts die Ladereihenfolge der Stylesheets.

Die Dateien

Quartz bindet nur `quartz/styles/custom.scss` ein; diese Datei lädt alle anderen. Rechts im Inspektor steht unter **Ladereihenfolge**, in welcher Reihenfolge: zuerst die eigenen Stylesheets in `quartz/styles/custom/` und `imported/`, dann `_quartzcontrol.scss`, zuletzt `custom.scss` selbst. Später geladene Dateien überschreiben frühere.

- **Neue Datei** legt ein Stylesheet unter `custom/` an und hängt es ans Ende der Reihenfolge.
- **Datei importieren…** (Symbolleiste) kopiert eine `.scss`- oder `.css`-Datei nach `imported/`. Gibt es dort schon eine gleichnamige Datei, fragt QuartzHub, ob sie ersetzt werden soll.
- Je Datei: **Nach oben**, **Nach unten**, **Umbenennen**, **Löschen**. Diese Dateiaktionen wirken sofort, ohne Snapshot.
- **Vorhanden, aber nicht eingebunden** listet Dateien, die in den Ordnern liegen, aber nicht geladen werden; **Einbinden** nimmt sie auf.

Ein Klick auf einen Namen öffnet die Datei im Editor, in einem Reiter. Änderungen am Text schreibt **Speichern** (⌘S).

> ⚠ **_quartzcontrol.scss gehört QuartzHub**
>
> Was zwischen den Markierungen in `_quartzcontrol.scss` steht, schreibt QuartzHub bei jeder Änderung an Reihenfolge, Schriften oder Variablen neu. Text außerhalb der Markierungen bleibt, eigene Regeln gehören aber in eine eigene Datei.

Der Editor

Ein Code-Editor mit Zeilennummern, Farben für SCSS und CSS und der Suchleiste von macOS. Darüber:

- **Komponente wählen…** und **Selektor einfügen**: setzt an der Cursorstelle den CSS-Selektor einer Komponente ein, etwa für die Suche oder das Inhaltsverzeichnis.
- Rechts im Inspektor unter **Verfügbare CSS-Variablen**: ein Klick fügt `var(--name)` ein, ein Klick auf die Farbe ihren Wert; mit ⌥ wird kopiert statt eingefügt.
- **Callout-Farben**: ein Klick auf einen Callout-Typ fügt ein Gerüst für seine Farben ein.
- **Aktuell geltende Farben und Schriften**: was gerade gilt, hell und dunkel, mit einer Schriftprobe.

Prüfen vor dem Build

Ein Fehler im SCSS lässt den ganzen Build scheitern. QuartzHub prüft deshalb mit dem Sass des Projekts:

- Beim Öffnen des Reiters und nach jedem Speichern prüft es alle Stylesheets zusammen. Oben steht dann „SCSS kompiliert fehlerfrei.“ oder der Fehler mit einem Verweis auf Datei und Zeile. **Erneut prüfen** prüft noch einmal.
- **Code prüfen** in der Symbolleiste prüft nur die offene Datei, auch ungespeichert.

Ist im Projekt kein Sass installiert, sagt QuartzHub, dass es nicht prüfen kann, statt Entwarnung zu geben. Ein CSS-Fehler erscheint auch auf der Übersicht (3.1).

Projekte aus älteren Fassungen

Zwei Hinweise erscheinen nur in Projekten, die mit einer frühen Fassung eingerichtet wurden:

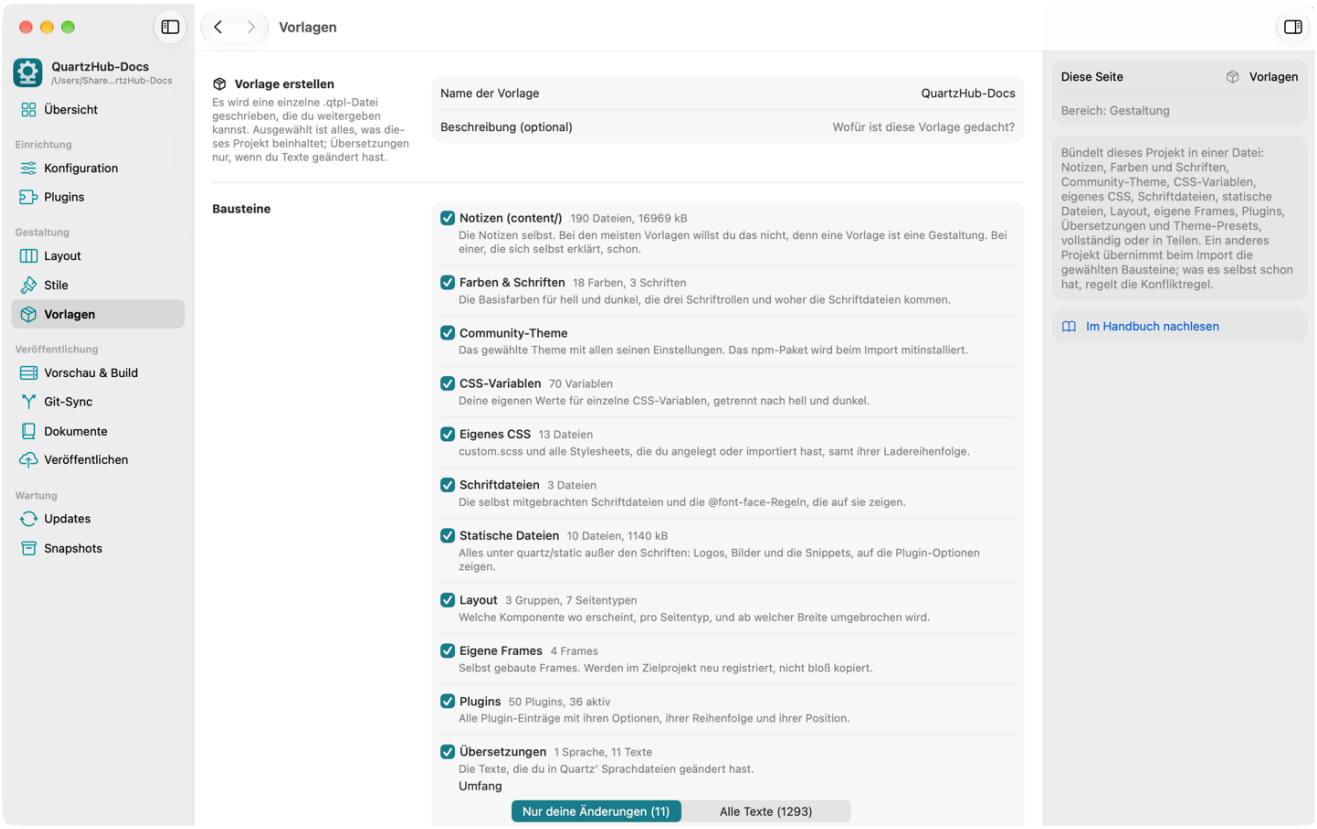
- **Jetzt umstellen** verschiebt Ladereihenfolge, Schriften und Variablen aus `custom.scss` in die eigene Datei `_quartzcontrol.scss`.
- **Zusammenlegen** fasst die Stylesheets der alten Vorlagen in die zwölf Dateien

der neuen Aufteilung zusammen und behält jede Regel.

Beide legen vorher einen Snapshot an und gehen nur ohne ungespeicherte Änderungen.

5.9 – Vorlagen

Eine Vorlage ist eine einzelne Datei mit der Endung `.qtpl`, die die Gestaltung eines Projekts trägt: Farben, Theme, CSS, Schriften, Layout, Frames, Plugins, Übersetzungen, auf Wunsch auch Inhalt. Nach dem Anwenden ist ein anderes Projekt so eingestellt wie das, aus dem sie stammt. **Vorlagen** erstellt und wendet sie an. Die Seite speichert nichts von selbst; erstellt und angewendet wird mit den Knöpfen.



Oben „Vorlage erstellen“ mit den Bausteinen, unten „Vorlage anwenden“.

Die Bausteine

Eine Vorlage besteht aus bis zu zwölf Bausteinen; beim Erstellen wie beim Anwenden wählt man, welche:

Baustein	Was er trägt
Notizen (content/)	die Dateien im Content-Ordner; für eine Gestaltung meist nicht gewollt, für eine Vorlage, die sich selbst erklärt, schon
Farben & Schriften	die Basisfarben für hell und dunkel, die drei Schriften und woher sie kommen
Communit	das gewählte Theme mit allen Einstellungen; das Paket wird beim

y-Theme	Anwenden installiert
CSS-Variablen	die eigenen Werte unter **Variablen**
Eigenes CSS	`custom.scss` und alle eigenen Stylesheets samt Ladereihenfolge
Schriftdateien	die selbst mitgebrachten Schriften und ihre Regeln
Statische Dateien	alles unter `quartz/static` außer den Schriften: Logos, Bilder, Schnipsel
Layout	welche Komponente wo erscheint, je Seitentyp, und die Breakpoints
Eigene Frames	die selbst gebauten Frames; im Zielprojekt neu angemeldet
Plugins	alle Plugin-Einträge mit Optionen, Reihenfolge und Position
Übersetzungen	die geänderten Texte aus Quartz' Sprachdateien
Theme-Presets	die gespeicherten Presets

Eine Vorlage erstellen

Unter **Vorlage erstellen**: **Name der Vorlage** (vorbelegt mit dem Projektnamen) und **Beschreibung (optional)**, dann die Bausteine mit ihren Zahlen, etwa „58 Plugins, 41 aktiv". Was das Projekt hat, ist angehakt.

Bei **Übersetzungen** wählt **Umfang** zwischen **Nur deine Änderungen** und **Alle Texte**. Was geändert ist, weiß QuartzHub erst, seit man einen Text unter **Konfiguration ▸ Übersetzungen** bearbeitet hat (4.4); für Sprachen davor nimmt man „Alle Texte".

Vorlage speichern… fragt nach dem Ort und schreibt die Datei. Statische Dateien dürfen zusammen höchstens 25 MB groß sein, die Notizen höchstens 100 MB: Eine Vorlage ist eine Gestaltung, keine Sicherung. Ein Baustein, der darüber liegt, steht nicht in der Liste; ein Hinweis darunter sagt, warum.

> ⚠ **Was eine Vorlage verrät**
>
> Eine Vorlage nennt den Ordnernamen des Projekts, aus dem sie stammt, und trägt, was die gewählten Bausteine enthalten. Wer sie weitergibt, prüft vorher, was darin steht, besonders bei **Notizen (content/)**, **Statische Dateien** und **Plugins**, deren Optionen etwa Adressen enthalten können.

Eine Vorlage anwenden

Unter **Vorlage anwenden**:

- **Basis-Template** lädt die Vorlage, die auch beim Anlegen eines Projekts angeboten wird (2.2), von GitHub. Angehakt sind nur die Bausteine der Gestaltung, nicht Inhalt, Layout, Plugins, Übersetzungen und Presets.
- **Vorlage wählen…** öffnet eine `.qtpl`-Datei. Angehakt ist alles, was sie enthält. Vorlagen im alten Format, ein Ordner statt einer Datei, tragen das Zeichen **Altes Format** und lassen sich ebenfalls anwenden.

Darunter stehen Name, Beschreibung, Datum und Herkunft der Vorlage und, nach einer Prüfung, was jeder Baustein in diesem Projekt täte: wie viel neu ist, ersetzt wird, gleich bleibt oder abgelehnt wird. Geprüft wird ohne zu schreiben.

Bei Konflikten:

- **Vorlage gewinnt**: Was das Projekt schon hat, wird durch die Vorlage ersetzt.
- **Projekt gewinnt**: Was das Projekt schon hat, bleibt; nur Fehlendes kommt dazu. Farben & Schriften und das Layout übernimmt die Vorlage dann gar nicht, auch nicht teilweise.

Muss etwas installiert werden, ein Theme oder ein Plugin, steht das darunter; dafür braucht es eine Verbindung ins Netz.

Vorlage anwenden fragt einmal nach, legt einen Snapshot an („Vor Vorlagen-Import“) und wendet die Bausteine nacheinander an. Scheitert einer, laufen die anderen weiter. Am Ende steht „Fertig: alles übernommen, nichts blieb offen.“ oder eine Liste dessen, was nicht ging oder unverändert blieb.

Ein paar Dinge tut eine Vorlage nie:

- Sie schreibt nicht in einen verlinkten Vault; ist der Content-Ordner ein Link, wird **Notizen (content/)** übersprungen.
- Sie ersetzt kein Projektbild, das man selbst gewählt hat (4.3).
- Sie schreibt nichts außerhalb des Projektordners.
- Bei quartz-forms behält das Projekt Versand, Schlüssel, Spamschutz und seine Formulare; nur fehlende Formulare kommen dazu.

Gefällt das Ergebnis nicht, führt der Snapshot zurück (7.3).

6 – Veröffentlichung

Die Gruppe **Veröffentlichung** hat vier Seiten: **Vorschau & Build** zeigt die Website und baut sie, **Git-Sync** gleicht das Projekt mit einem Repository ab, **Dokumente** macht Bücher daraus, und **Veröffentlichen** bringt das Ergebnis dorthin, wo es hin soll.

Vorschau & Build

1. **Die Vorschau**: die Website im Fenster, während man schreibt.
2. **Der Build**: die fertigen Dateien, und wohin sie kommen.
3. **Die Ausgabe lesen**: was Quartz sagt, und was davon wichtig ist.
4. **Fremde Server**: Vorschauen, die QuartzHub nicht selbst gestartet hat.

Git-Sync und Dokumente

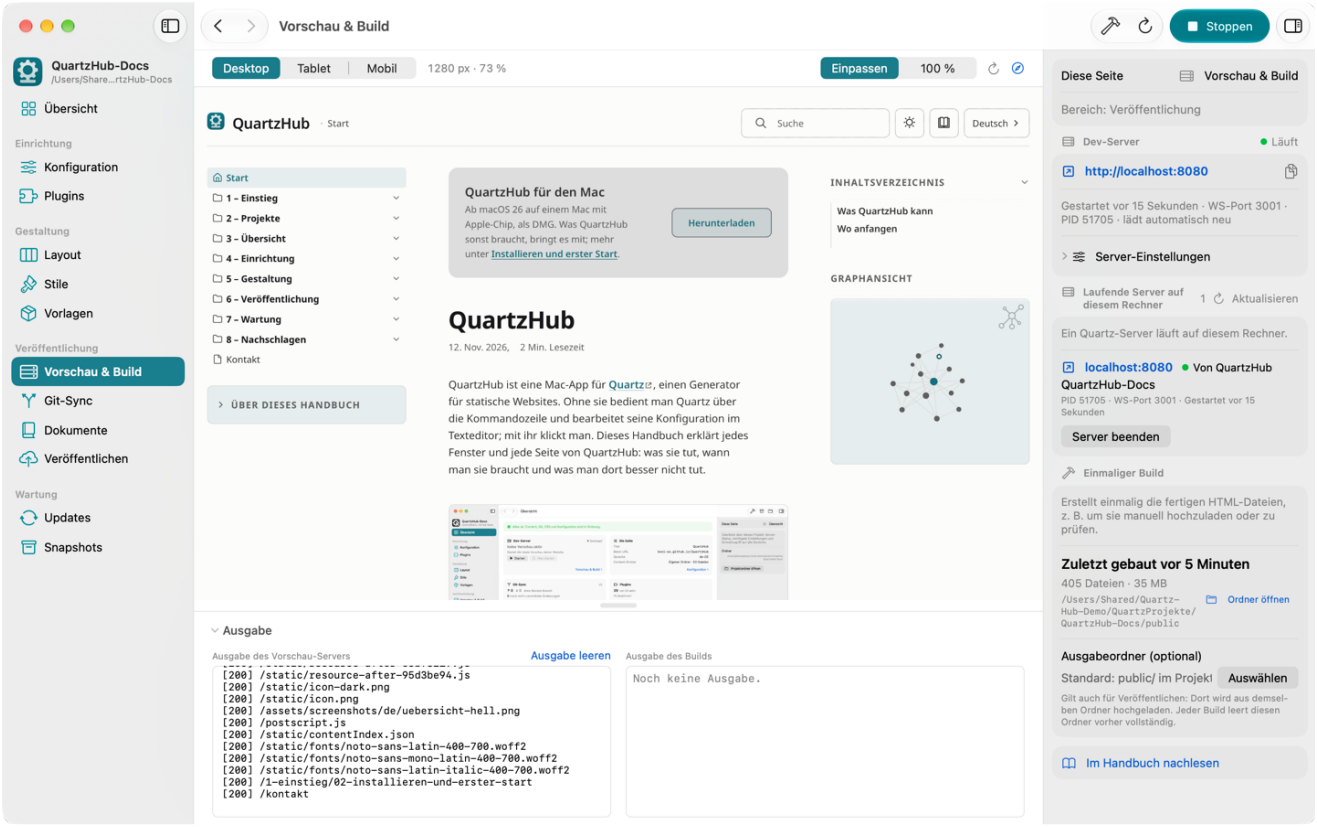
5. **Git-Sync**: das Projekt in einem Repository sichern und teilen.
6. **Dokumente**: Bücher als PDF, ePub oder HTML.

Veröffentlichen

7. **Zugänge und Ziele**: der Unterschied, und welches Ziel passt.
8. **GitHub Pages**
9. **Auf einen Webspace**: SFTP, rsync, FTP.
10. **In einen Ordner**
11. **Webhook**: Netlify, Cloudflare Pages, Vercel.
12. **Was hochgeladen wird**: die Vorschau der Änderungen und der Lauf.

6.1 – Die Vorschau

Die Vorschau ist Quartz' eigener Dev-Server: Quartz baut die Website, liefert sie auf dem Mac aus und baut bei jeder Änderung neu. **Vorschau & Build** zeigt sie in der Mitte des Fensters.



Die laufende Vorschau. Oben Breite und Maßstab, unten die Ausgabe, rechts der Server.

Starten und stoppen

Starten in der Symbolleiste oder **Projekt ▸ Server starten** (⌘R) aus jedem Fenster des Projekts. Danach steht dort **Stoppen** (⌘.), daneben **Neustarten**. Der Zustand steht im Inspektor: **Gestoppt**, **Startet…**, **Läuft**, **Stoppt…** oder **Fehler**.

Als gestartet gilt der Server, wenn er antwortet. Das dauert beim ersten Mal so lange wie ein Build, bei einer großen Website eine halbe Minute oder mehr. Danach baut Quartz nach jeder Änderung neu, an den Notizen wie an der Konfiguration, und die Seite lädt von selbst neu.

Vor jedem Start bringt QuartzHub das Projekt in Ordnung: Es schreibt die eigenen Frames neu, setzt eine fehlende Zeile in `custom.scss` wieder ein und holt fehlende Schriften. Was es dabei tut, steht in der Ausgabe.

Der Start wird verweigert, und der Grund steht oben in Rot, wenn

- die npm-Pakete des Projekts fehlen, etwa nach einem frischen Klon; **Installieren**

in der Übersicht holt sie nach;
- für dasselbe Projekt schon ein Server läuft, den QuartzHub nicht gestartet hat (6.4).

Die Vorschau im Fenster

- **Desktop**, **Tablet**, **Mobil**: Die Website wird in einer passenden Breite gebaut, Desktop mit mindestens 1280 Pixeln, und ins Fenster eingepasst. Daneben stehen Breite und Maßstab, etwa „1280 px · 73 %“.
- **Einpassen** oder **100 %**: verkleinern oder in echter Größe zeigen und seitlich scrollen.
- **Vorschau neu laden** und **Im Browser öffnen**.

Links innerhalb der Website öffnen sich in der Vorschau, Links nach außen im Browser.

Server-Einstellungen

Im Inspektor unter **Server-Einstellungen**:

- **Port** (Vorgabe 8080) und **WS-Port** (3001, für das automatische Neuladen). Ist ein Port belegt, nimmt der Server beim Start den nächsten freien und sagt das in der Ausgabe.
- **Remote-Dev-Host (optional)**: nur für eine Vorschau hinter einem Tunnel, etwa von einem anderen Rechner aus. Dann weicht der Server nicht auf andere Ports aus.

Die Einstellungen gelten, bis QuartzHub beendet wird. Ändern lassen sie sich nur bei gestopptem Server.

Was beim Beenden mit laufenden Servern geschieht, stellt man in den Einstellungen ein (2.5).

6.2 – Der Build

Ein Build erzeugt die fertige Website einmal: HTML, CSS, Bilder, alles, was auf einen Server kommt. Die Vorschau baut auch, aber nur für sich; zum Veröffentlichen braucht es einen Build.

Bauen

- **Jetzt bauen** (Hammer) in der Symbolleiste von **Vorschau & Build**,
- **Projekt ▸ Build ausführen** (⌘B) aus jedem Fenster des Projekts,
- **Build ausführen** auf der Übersicht,
- **Jetzt bauen** unter **Veröffentlichen**.

Während des Builds steht im Inspektor, seit wann er läuft und was Quartz gerade tut: vorbereiten, die Notizen lesen, die Seiten schreiben. Danach „Erfolgreich in 12,3s“ oder „Fehlgeschlagen in …“, und was Quartz dazu sagte, steht in der Ausgabe (6.3).

Unter **Einmaliger Build** steht, wann zuletzt gebaut wurde, mit Zahl und Größe der Dateien, und **Ordner öffnen**. QuartzHub liest das aus dem Ausgabeordner selbst.

Der Ausgabeordner

Vorgabe ist `public/` im Projekt. **Ausgabeordner (optional)** stellt einen anderen ein, im Projekt oder anderswo. Er gilt auch für **Veröffentlichen**: Dort wird aus demselben Ordner hochgeladen.

> ⚠ **Jeder Build leert seinen Ausgabeordner**
>
> Quartz löscht vor jedem Build alles im Ausgabeordner. QuartzHub verweigert deshalb Ordner, die das nicht vertragen: den Projektordner selbst, einen Ordner, der das Projekt enthält, den Benutzerordner und Ordner des Projekts wie `content/` oder `quartz/`. Liegen in einem Ordner Dateien, die nicht nach einem Build aussehen, fragt QuartzHub vorher.

Die Vorschau baut immer nach `public/`, unabhängig von dieser Einstellung. Läuft sie, kann ein Build nicht gleichzeitig nach `public/` schreiben; QuartzHub sagt das und baut nicht.

Alle Wege zum Build bauen in denselben, gespeicherten Ausgabeordner. Steht im Feld ein neuer Ordner, den man noch nicht mit der Eingabetaste bestätigt hat, übernimmt **Jetzt bauen** ihn zuerst; einen Ordner, den QuartzHub verweigert, baut es nicht.

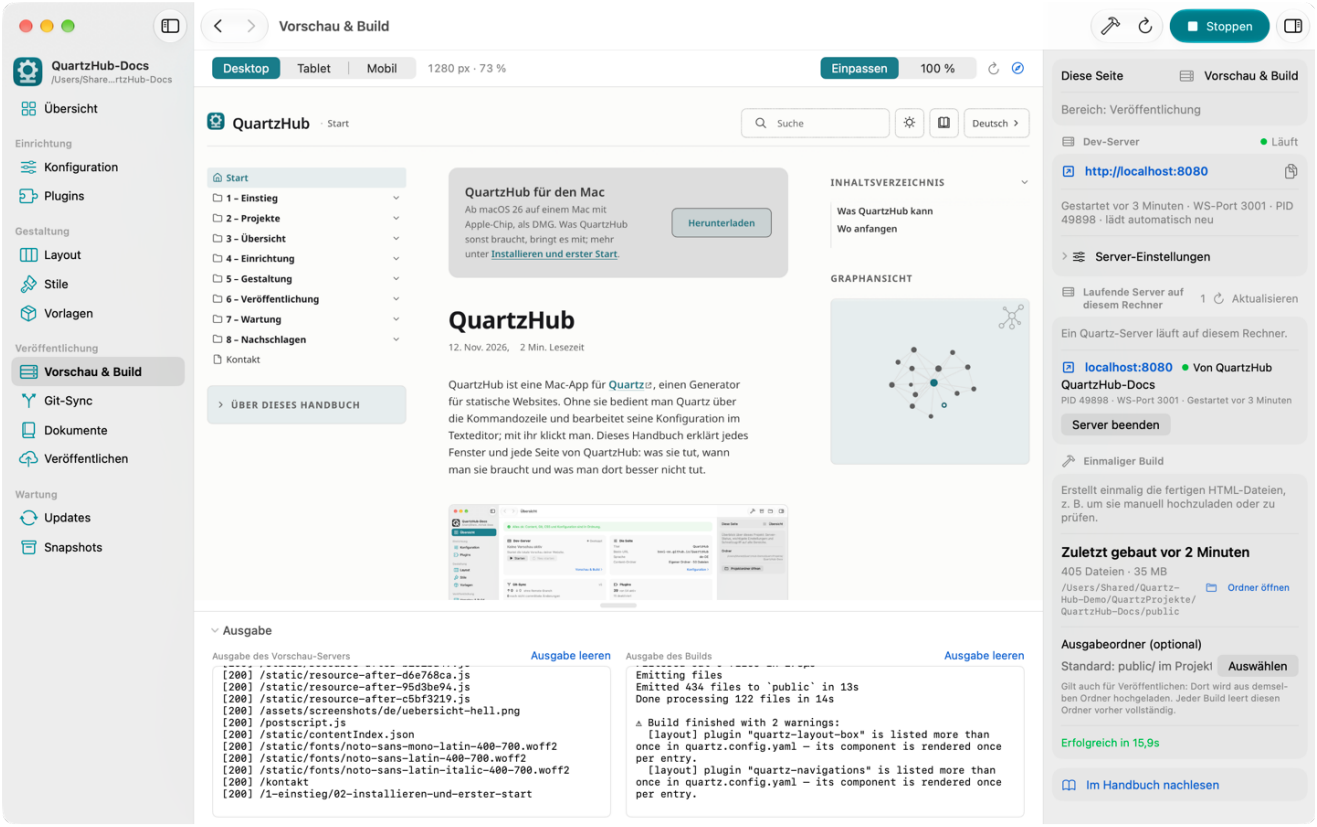
Wann gebaut werden muss

Vor jedem Veröffentlichen, das Dateien überträgt. **Veröffentlichen** baut nicht von selbst;

es lädt hoch, was im Ausgabeordner liegt, und sagt, wenn dort noch nichts ist (**6.12**).

6.3 – Die Ausgabe lesen

Unten auf **Vorschau & Build** steht die **Ausgabe**, in zwei Konsolen nebeneinander: links die des Vorschau-Servers, rechts die des letzten Builds. Den Griff darüber kann man ziehen; die Ausgabe lässt sich auch einklappen, dann bekommt die Vorschau den Platz.



Gestoppte Vorschau, rechts unten die Ausgabe eines Builds.

Farben

- **Normale Schrift**: was Quartz selbst ausgibt.
- **Orange**: ein Satz von QuartzHub über einen Lauf, der weitergeht, etwa „Port 8080 ist belegt, der Server nimmt 8081.“ oder was es vor dem Start in Ordnung gebracht hat.
- **Rot**: Fehlerausgabe von Quartz oder ein Fehler von QuartzHub, etwa ein verweigerter Build.

Ausgabe leeren leert eine Konsole. Text lässt sich markieren und mit ⌘C kopieren, etwa für eine Fehlermeldung. Jede Konsole hält die letzten rund tausend Zeilen, solange QuartzHub läuft, auch wenn man die Seite verlässt; nach einem Neustart sind sie leer.

Was Quartz sagt

Quartz’ Ausgabe ist Englisch und kommt unverändert. Ein paar Zeilen kommen oft vor:

Zeile	Bedeutung
`Parsing input files`	Quartz liest die Notizen.
`Emitting files`	Quartz schreibt die Seiten.
`Done processing … files in …`	Der Build ist fertig.
`Started a Quartz server listening at http://localhost:8080`	Die Vorschau läuft.
`Detected change, rebuilding…`	Eine Datei hat sich geändert; die Vorschau baut neu.
`[200] /pfad`	Die Vorschau hat eine Seite ausgeliefert.
`[404] /pfad`	Etwas wurde verlangt, das es nicht gibt, etwa ein Bild mit falschem Pfad.

Warnungen in eckigen Klammern nennen das Plugin, von dem sie kommen, etwa `[links]` für Links auf Seiten, die es nicht gibt. Sie halten den Build nicht auf.

Wenn der Build scheitert

Ein Build scheitert meist an einer von drei Stellen:

1. **Am CSS.** Ein Fehler in einem Stylesheet. **Stile ▸ Eigenes CSS** zeigt ihn mit Datei und Zeile, ebenso die Übersicht (5.8).
2. **An einer Notiz.** Kaputtes Frontmatter (etwa ein Doppelpunkt ohne Anführungszeichen) oder eine Einbindung, die es nicht gibt. Quartz nennt die Datei; man repariert sie im Editor.
3. **An einem Plugin.** Ein Plugin, das fehlt oder nicht zu diesem Quartz passt. Quartz nennt es; **Plugins ▸ Installiert** zeigt, was Quartz' eigene Prüfung dazu sagt (4.5).

Die wichtigste Zeile steht meist am Ende der roten Ausgabe. Die Übersicht meldet einen abgestürzten Server auch unter **Handlungsbedarf**.

6.4 – Fremde Server

Ein Quartz-Server läuft als eigener Prozess und kann QuartzHub überleben: wenn man beim Beenden „Weiterlaufen lassen“ wählt, wenn QuartzHub abstürzt oder wenn jemand Quartz im Terminal startet. QuartzHub findet solche Server.

Laufende Server auf diesem Rechner

Der Abschnitt im Inspektor von **Vorschau & Build** listet jeden Quartz-Server auf dem Mac, egal wer ihn gestartet hat. **Aktualisieren** sucht neu; sonst sucht QuartzHub, wenn das Fenster wieder nach vorn kommt und wenn ein eigener Server startet oder endet.

Jede Zeile zeigt:

- die Adresse, ein Klick öffnet sie im Browser;
- **Von QuartzHub** (in dieser Sitzung gestartet) oder **Außerhalb gestartet** (alles andere);
- das Projekt, wenn QuartzHub es kennt, sonst den Titel der Website oder den Ordner;
- Prozessnummer, WebSocket-Port und seit wann er läuft;
- **Server beenden**.

Einen Server, den QuartzHub nicht gestartet hat, beendet es nie von sich aus. **Server beenden** fragt vorher, denn wer ihn gestartet hat, arbeitet vielleicht gerade damit.

Warum das wichtig ist

Zwei Server für dasselbe Projekt würden in denselben Ordner bauen. QuartzHub startet deshalb keinen zweiten und sagt, welcher schon läuft. Ist ein Port belegt, aber von keinem erkennbaren Quartz-Server, steht das im Abschnitt; die Vorschau nimmt dann einen anderen Port.

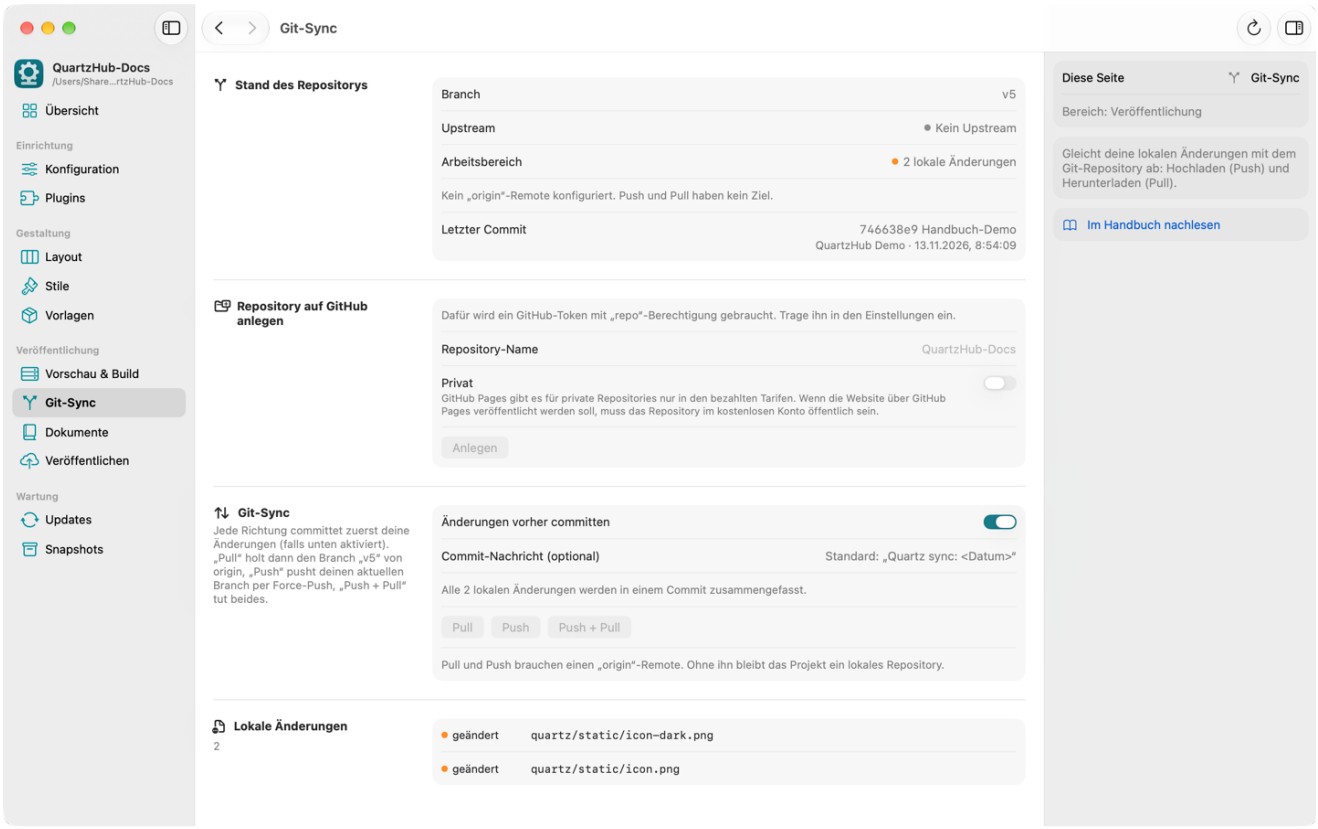
Beim Start und beim Beenden

Beim Start fragt QuartzHub nach Servern, die es selbst in einer früheren Sitzung gestartet hat und die noch laufen: **Weiterlaufen lassen** oder **Beenden**.

Beim Beenden fragt es nach den Servern dieser Sitzung, je nach Einstellung (2.5). Vorher fragt es nach ungespeicherten Änderungen und nach laufender Arbeit wie einem Update oder einem Veröffentlichen.

6.5 – Git-Sync

Ein Quartz-Projekt ist ein git-Repository. **Git-Sync** lädt es in ein Repository im Netz hoch und holt Änderungen von dort: als Sicherung, um auf einem zweiten Mac weiterzuarbeiten, oder als Grundlage für GitHub Pages (6.8). Wer nur veröffentlichen will, braucht Git-Sync nicht.



Ein Projekt ohne Repository im Netz, darunter der Abschnitt zum Anlegen.

Stand des Repositorys

- **Branch**: der ausgecheckte Branch. Quartz-Projekte stehen auf `v5`.
- **Upstream** und **Abgleich**: ob der Branch eine Entsprechung im Netz hat und wie viele Commits noch hochzuladen oder zu holen sind, „Auf dem Stand des Remotes", wenn nichts fehlt. Das gilt für den Stand beim letzten Abgleich; die Seite fragt das Netz nicht selbst.
- **Arbeitsbereich**: lokale Änderungen und Konflikte.
- **origin**: die Adresse des Repositorys im Netz, bei GitHub mit **Auf GitHub öffnen**.
- **Letzter Commit**.

Unten listet **Lokale Änderungen** die geänderten Dateien: neu, geändert, gelöscht, umbenannt, unverfolgt, Konflikt. **Aktualisieren** in der Symbolleiste liest den Stand neu.

Ein Repository auf GitHub anlegen

Ein neues Projekt hat absichtlich kein Repository im Netz, damit nichts versehentlich bei Quartz selbst landet. **Repository auf GitHub anlegen** legt eines unter dem eigenen Konto an und trägt es als `origin` ein:

- **Repository-Name**, vorbelegt mit dem Projektnamen;
- **Privat**: Bei einem kostenlosen Konto gibt es GitHub Pages nur für öffentliche Repositories.

Dafür braucht es einen persönlichen Zugriffstoken mit der Berechtigung „repo“ unter **Einstellungen ▸ Zugänge** (6.7). Wer ein Repository anderswo hat, etwa bei Codeberg, trägt es im Terminal als `origin` ein (`git remote add origin <adresse>`).

Push und Pull

- **Änderungen vorher committen** (Vorgabe an): fasst alle lokalen Änderungen in einem Commit zusammen, mit der **Commit-Nachricht** oder „Quartz sync: “.
- **Pull** holt den Branch `v5` von `origin` und führt ihn zusammen.
- **Push** lädt den aktuellen Branch hoch, per Force-Push: Was nur im Netz liegt, geht dabei verloren.
- **Push + Pull** tut beides.

QuartzHub benutzt dafür Quartz’ eigenes `quartz sync`. Darunter steht danach „Erfolgreich.“ oder „Fehlgeschlagen.“ mit der Ausgabe.

Anmelden: Bei einem Repository auf github.com über https benutzt QuartzHub den Token aus den Einstellungen. Bei ssh-Adressen und anderen Anbietern gelten die eigenen Schlüssel und Zugangshelfer von git, wie im Terminal.

> ⚠ **Force-Push heißt: dieser Mac gewinnt**
>
> Arbeiten zwei Rechner am selben Projekt, erst **Pull**, dann ändern, dann **Push**. Wer zuerst pusht, überschreibt, was der andere Rechner hochgeladen hat.

Wenn ein Pull scheitert

Haben beide Seiten dieselbe Datei geändert, bleibt ein Konflikt: Der Arbeitsbereich zeigt ihn, oben steht „Ein Merge ist angefangen und nicht abgeschlossen.“ **Merge abbrechen** setzt auf den Stand vor dem Pull zurück; was seitdem von Hand vorgemerkt wurde, ginge dabei verloren, und QuartzHub fragt dann vorher. Einen Konflikt lösen kann QuartzHub nicht; das geht in einem Editor oder im Terminal.

Was nicht mitkommt

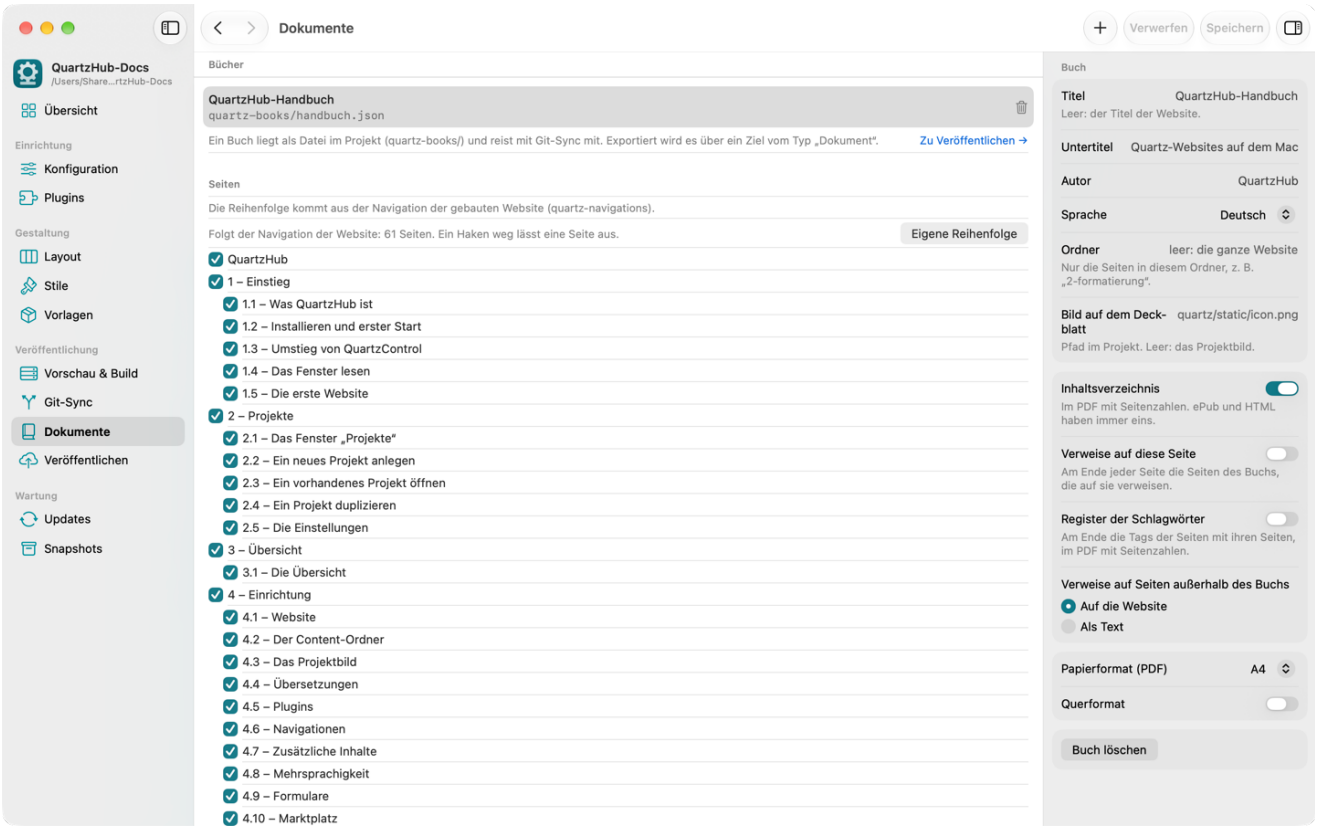
- `node_modules/` und `public/`: Sie entstehen auf dem anderen Rechner neu (2.3).

- `.quartz-gui/`, also eigene Frames, Snapshots und Ziele: Der Ordner steht in `.gitignore`. Eigene Frames kopiert man deshalb von Hand oder holt sie aus einem Snapshot (**3.1**).
- Ein verlinkter Vault: Quartz lädt dabei die Notizen als Dateien hoch; auf dem anderen Rechner muss der Content-Ordner neu verknüpft werden (**4.2**).

Bücher (`quartz-books/`) kommen mit.

6.6 – Dokumente

Ein Buch ist eine Auswahl von Seiten der Website in einer Reihenfolge, mit Deckblatt und Inhaltsverzeichnis. **Dokumente** legt Bücher an; ausgegeben werden sie über ein Ziel vom Typ „Dokument“ unter **Veröffentlichen**, als PDF, ePub oder eine einzelne HTML-Datei. Das PDF dieses Handbuchs ist so entstanden.



Ein Buch, das der Navigation der Website folgt. Rechts seine Einstellungen.

Bücher anlegen

➕ **Neues Buch** fragt nach dem Titel und legt ein Buch an, das zuerst der Navigation der Website folgt. Jedes Buch ist eine Datei unter `quartz-books/` im Projekt und kommt mit Git-Sync mit. Die Datei heißt nach diesem Titel, aus „Über Gärten“ wird `ueber-gaerten.json`, und behält den Namen, auch wenn man den Titel später ändert; die ausgegebenen Dateien heißen ebenso. Wie sie heißen wird, steht schon beim Anlegen unter dem Titel.

Änderungen schreibt **Speichern** (⌘S); **Verwerfen** nimmt sie zurück.

Die Seiten

Die Liste der Seiten liest QuartzHub aus dem letzten Build. Ohne Build sagt es das und verweist auf **Vorschau & Build**. Auch der Inhaltsindex muss an sein (4.1).

- **Der Navigation folgen** (Vorgabe): Das Buch nimmt die Reihenfolge der Navigation der Website. Ein Haken weg lässt eine Seite aus. Seiten, die die Navigation nicht zeigt, stehen darunter.
- **Eigene Reihenfolge**: kopiert die Navigation in eine eigene Liste. Seiten zieht man an ihren Platz oder verschiebt sie mit **Anordnen ▸ Nach oben/Nach unten** (⌥⌘↑, ⌥⌘↓). **+ Zwischenüberschrift** fügt eine Überschrift ein, etwa „Anhang“; **Aufnehmen** holt eine Seite dazu, die nicht in der Navigation steht. **Wieder der Navigation folgen** wirft die eigene Liste weg.

Die Startseite des gewählten Ordners steht immer zuerst.

Einstellungen im Inspektor

- **Titel** (leer: der Titel der Website), **Untertitel**, **Autor**.
- **Sprache**, wenn die Website mehrere hat (4.8).
- **Ordner**: nur die Seiten in diesem Ordner; leer die ganze Website.
- **Bild auf dem Deckblatt**: ein Pfad im Projekt; leer das Projektbild.
- **Inhaltsverzeichnis** (im PDF mit Seitenzahlen), **Verweise auf diese Seite** am Ende jeder Seite, **Register der Schlagwörter** am Ende.
- **Verweise auf Seiten außerhalb des Buchs**: **Auf die Website** (braucht eine Basis-URL) oder **Als Text**.
- **Papierformat (PDF)**: A4, A5 oder Letter, dazu **Querformat**.

Ausgeben

Unter **Veröffentlichen** ein Ziel vom Typ **Dokument** anlegen: **Zielordner**, **Buch** und die **Formate** PDF, ePub, HTML (eine Datei) (6.7). **Jetzt exportieren** schreibt die Dateien, `<buch>.pdf` und so fort; ein laufender Export lässt sich abbrechen.

Ein PDF hat ein Deckblatt, das Inhaltsverzeichnis mit Seitenzahlen, Lesezeichen nach Überschriften und eine Fußzeile mit Titel und Seitenzahl. Formeln und Diagramme werden als Bilder gezeichnet; Formulare, Videos und eingebettete Seiten werden durch einen Satz mit ihrer Adresse ersetzt.

Findet QuartzHub Fehler, etwa Bilder, die nicht laden, oder Verweise ohne Ziel, schreibt es nichts und sagt, was es gefunden hat. Für Formeln und Diagramme lädt der Export Stylesheets und Skripte aus dem Netz; ohne Verbindung fehlen sie.

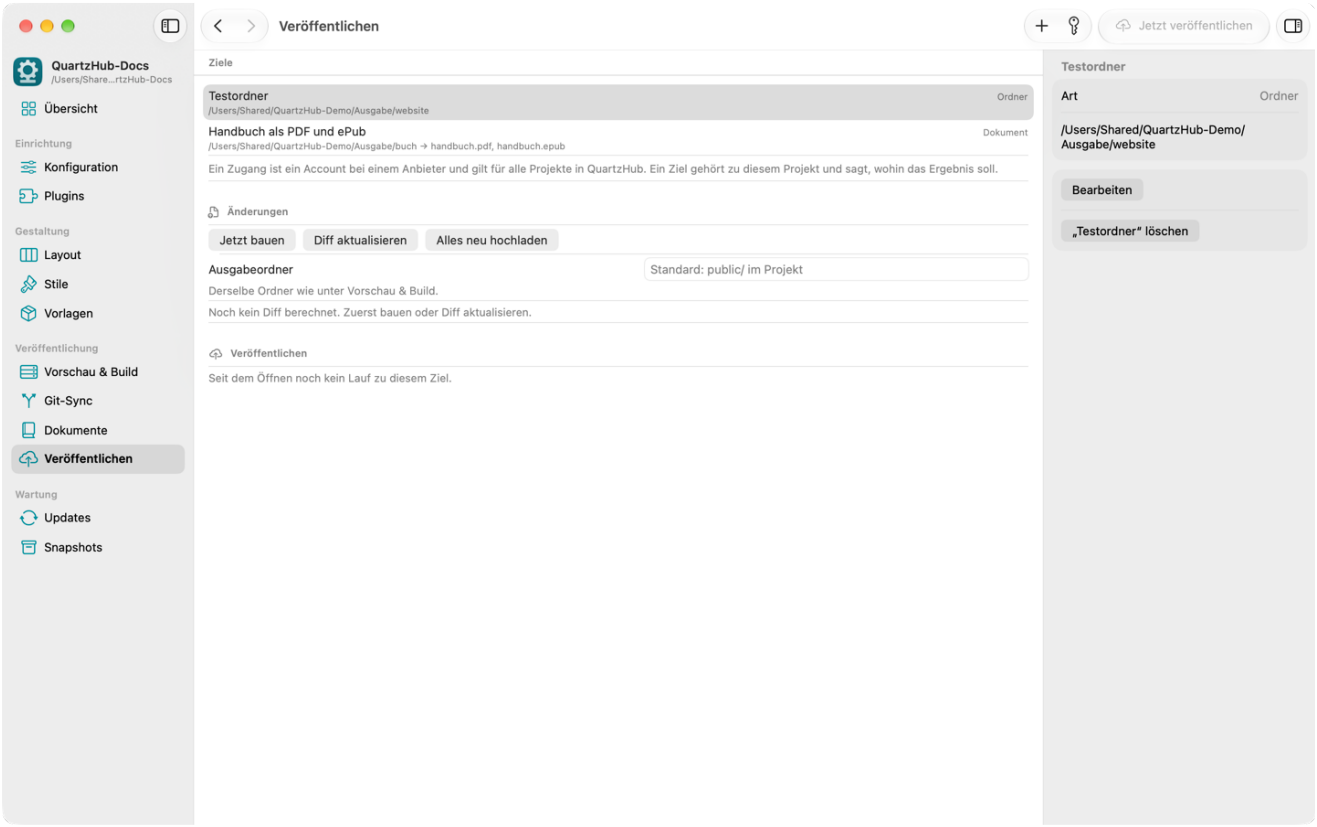
Dokumente mit hochladen bei einem anderen Ziel lädt die zuletzt exportierten Dateien mit der Website in den Ordner `downloads/` hoch; QuartzHub zeigt dazu einen Link, den man auf eine Seite setzen kann.

6.7 – Zugänge und Ziele

Zum Veröffentlichen gehören zwei Dinge:

- Ein **Zugang** ist ein Konto bei einem Anbieter: ein Server-Login, ein Build-Hook, ein Token für GitHub. Zugänge gelten für alle Projekte und stehen unter **Einstellungen ▸ Zugänge**.
- Ein **Ziel** gehört zu einem Projekt und sagt, wohin dessen Website kommt: welcher Zugang, welcher Ordner auf dem Server, welcher Branch, ob gelöscht wird. Ziele stehen unter **Veröffentlichen**.

So nutzen zwei Projekte denselben Webspace mit einem Zugang und zwei Zielen in verschiedenen Ordnern.



Zwei Ziele, ein Ordner und ein Dokument. Darunter der Abschnitt „Änderungen", noch ohne berechneten Diff.

Welches Ziel passt?

Ich habe …	Ziel	Kapitel
ein GitHub-Konto und will nichts bezahlen	**Git-Branch** mit GitHub Pages	6.8

einen Webspace bei einem Hoster	**SFTP** (oder rsync), notfalls **FTP**	6.9
einen eigenen Server oder ein NAS	**SFTP** mit rsync	6.9
Netlify, Cloudflare Pages oder Vercel, die selbst bauen	**Webhook**	6.11
einen Ordner, den etwas anderes weiterreicht (Cloud-Ordner, Freigabe)	**Ordner**	6.10
ein Buch als PDF oder ePub	**Dokument**	6.6

Codeberg Pages und GitLab Pages gehen ebenfalls über **Git-Branch**.

Zugänge anlegen

Einstellungen ▸ Zugänge (⌘, oder **Zugänge verwalten** unter **Veröffentlichen**):

- **GitHub**: ein **Persönlicher Zugriffstoken**. **Speichern & prüfen** fragt GitHub, ob er gilt, und zeigt **Gültig** mit dem Kontonamen, **Abgelehnt** oder **Nicht prüfbar**. Für den Marktplatz reicht ein Token ohne Berechtigungen; für Repositories und GitHub Pages braucht er den Scope „repo“.
- **SFTP / SSH**: **Host**, **Port** (22), **Benutzername** und die **Authentifizierung**: **Passwort**, **Privater Schlüssel** (als Datei, meist aus `~/.ssh`, oder eingefügt) oder **SSH-Agent**. Die eigene `~/.ssh/config` gilt weiter.
- **FTP**: Host, Port (21), Benutzername, Passwort und **FTPS (verschlüsselt)**, Vorgabe an. Ohne FTPS gehen Passwort und Dateien unverschlüsselt durchs Netz.
- **Webhook**: die **Webhook-URL** des Anbieters, nur https.

Passwörter, Schlüssel, Tokens und Webhook-URLs liegen im Schlüsselbund von macOS, nie im Projekt. Ein Zugang zeigt, wie viele Ziele ihn benutzen; wer ihn löscht, wird gewarnt.

Host-Key: Beim ersten Verbinden per SFTP zeigt QuartzHub den Fingerabdruck des Servers und fragt, ob er stimmt. Den Fingerabdruck nennt der Anbieter. **Verbinden und merken** speichert ihn für alle Projekte. Ändert er sich später, verweigert QuartzHub die Verbindung; erst **Host-Key vergessen** lässt neu fragen, und das nur, wenn der Server nachweislich neu aufgesetzt wurde.

Ziele anlegen

➕ **Neues Ziel** unter **Veröffentlichen**: **Name**, **Art** und je nach Art die Felder, die die folgenden Kapitel beschreiben. Für SFTP, FTP und Webhook wählt man den **Zugang**; **Neuer …-Zugang** legt einen an, ohne den Dialog zu verlassen.

Für alle Arten außer Git-Branch und Dokument gibt es:

- **Entfernte Dateien auch im Ziel löschen** (Vorgabe an): Was aus der Website

verschwunden ist, verschwindet auch im Ziel.
- **Nie veröffentlichen (ein Pfad pro Zeile)**: Pfade aus dem Build, die nie hochgeladen werden, etwa `.htaccess`, wenn der Server eine eigene hat.
- **Dokumente mit hochladen** (6.6).

Die Ziele liegen in `.quartz-gui/publish-targets.json`, ohne Passwörter. Ein Duplikat des Projekts bekommt sie nicht mit (2.4).

> ⚠ **Erst die Basis-URL**
>
> Steht die Basis-URL noch auf `localhost`, sagt **Veröffentlichen** das oben. Links, RSS-Feed und Vorschaubilder einer veröffentlichten Website zeigen sonst ins Leere (4.1).

6.8 – GitHub Pages

GitHub Pages liefert eine Website aus einem Branch eines Repositorys aus, kostenlos und mit https. QuartzHub legt den Build als einen Commit auf einen eigenen Branch, meist `gh-pages`, und stellt Pages auf diesen Branch. GitHub Actions braucht es dafür nicht.

Einmal einrichten

1. **Token**: unter **Einstellungen ▸ Zugänge** einen persönlichen Zugriffstoken mit dem Scope „repo“ eintragen (6.7).
2. **Repository**: unter **Git-Sync ▸ Repository auf GitHub anlegen**. Für ein kostenloses Konto muss es öffentlich sein (6.5).
3. **Basis-URL**: unter **Konfiguration ▸ Website** die künftige Adresse eintragen, meist `name.github.io/repository`, mit eigener Domain diese.
4. **Ziel**: unter **Veröffentlichen** ein neues Ziel der Art **Git-Branch**, **Anbieter** GitHub Pages, **Branch** `gh-pages`. Der Branch darf nicht der sein, auf dem das Projekt selbst steht.
5. **Pages einschalten**: Im Abschnitt **GitHub Pages** unter dem Ziel **Auf „gh-pages“ stellen**.

Dann bauen und **Jetzt veröffentlichen** (6.12). Ein paar Minuten später steht die Website unter der Adresse, die der Abschnitt nennt.

Der Abschnitt GitHub Pages

Er zeigt, ob Pages **Veröffentlicht**, **Wird gebaut**, **Fehlgeschlagen** oder **Noch nie gebaut** ist, die Adresse der Website und aus welchem Branch sie kommt. **Status neu laden** fragt GitHub neu.

Eigene Domain: etwa `wiki.example.org`. Beim Anbieter der Domain muss dafür ein DNS-Eintrag auf GitHub zeigen; wie, beschreibt GitHub. **Einstellungen speichern** schreibt die Domain; GitHub legt sie als Datei `CNAME` in den Branch, und QuartzHub behält sie bei jedem Veröffentlichen. **HTTPS erzwingen** geht erst, wenn GitHub das Zertifikat ausgestellt hat, einige Minuten nach dem Setzen der Domain.

Was beim Veröffentlichen passiert

Der Branch wird jedes Mal ganz durch den aktuellen Build ersetzt (Force-Push), in einem einzigen Commit ohne Vorgeschichte. Deshalb fragt QuartzHub vorher. Hat sich nichts geändert, wird nichts hochgeladen. Die Hooks des Projekts laufen dabei nicht.

Codeberg Pages liefert den Branch `pages` ebenso aus. Bei GitLab braucht es zusätzlich einen CI-Job, der die Dateien ausliefert; der Branch allein veröffentlicht dort nichts.

6.9 – Auf einen Webspace

Für einen Webspace bei einem Hoster oder einen eigenen Server gibt es drei Wege. Vorher braucht es einen Zugang unter **Einstellungen ▸ Zugänge** (6.7).

Weg	Wann
SFTP	der Normalfall; jeder Hoster mit SSH-Zugang kann es
SFTP mit rsync	schneller, vergleicht mit dem Server; braucht einen Schlüssel und rsync auf dem Server
FTP	nur, wenn der Hoster nichts anderes anbietet, und dann mit FTPS

Das Ziel

Ein neues Ziel der Art **SFTP** oder **FTP**, mit dem passenden **Zugang**, und:

- **Remote-Pfad**: der Ordner auf dem Server, in den die Website kommt, etwa `httpdocs` oder `/var/www/example.org`. Ohne führenden Schrägstrich zählt er ab dem Anmeldeordner; bei einem Webspace ist das der Normalfall. Welcher Ordner die Website ausliefert, sagt der Hoster.
- **Übertragung** (nur SFTP): **SFTP (Datei für Datei)** oder **rsync (schneller, vergleicht mit dem Server)**.
- **Entfernte Dateien auch im Ziel löschen**, **Nie veröffentlichen**.

Den Ordner der Website legt QuartzHub an, Ordner darüber nicht.

> ⚠ **Löschen und der Anmeldeordner**
>
> Mit eingeschaltetem Löschen verweigert QuartzHub den Anmeldeordner selbst (`/`, `.`, `~`) als Pfad: Er würde geleert. Die Website gehört in einen Unterordner.

SFTP oder rsync

SFTP vergleicht den Build mit einer Liste dessen, was QuartzHub zuletzt an dieses Ziel geschickt hat, und überträgt nur, was neu oder geändert ist. Was jemand direkt auf dem Server ändert oder löscht, bemerkt es nicht; dafür gibt es **Alles neu hochladen** (6.12).

rsync fragt den Server selbst, was dort liegt; der Vergleich stimmt also auch nach Änderungen von Hand. rsync braucht:

- einen Zugang mit **Privater Schlüssel** als Datei oder **SSH-Agent**, kein Passwort;
- einen bestätigten Host-Key: einmal per SFTP veröffentlichen, dann lässt sich rsync wählen;

- rsync auf dem Server.

Der Dialog sagt, was davon fehlt.

> ⚠ **rsync löscht alles, was nicht zum Build gehört**
>
> Mit eingeschaltetem Löschen entfernt rsync im Zielordner jede Datei, die der Build nicht hat, auch solche, die QuartzHub nie hochgeladen hat, etwa eine `.htaccess` oder einen Ordner mit Downloads. Was bleiben soll, gehört unter **Nie veröffentlichen**, oder die Website bekommt einen eigenen Ordner.

FTP

FTP überträgt wie SFTP Datei für Datei nach der Liste des letzten Laufs. Ohne **FTPS** gehen Passwort und Dateien im Klartext durchs Netz; QuartzHub warnt dann.

Wenn es nicht klappt

QuartzHub sagt, woran es lag, mit dem Namen des Servers: nicht erreichbar, Port abgelehnt, Anmeldung abgelehnt (mit einem eigenen Satz für Passwort, Schlüssel und Agent), keine Antwort seit 60 Sekunden. Liegt der Server im eigenen Netz, etwa ein NAS, braucht QuartzHub die Erlaubnis für das lokale Netzwerk (1.2).

6.10 – In einen Ordner

Ein Ziel der Art **Ordner** kopiert die Website in einen Ordner: auf dem Mac, auf einem Laufwerk oder einer Freigabe. Nützlich, wenn etwas anderes die Dateien weiterreicht (ein Cloud-Ordner, ein NAS, das selbst ausliefert), oder um zu sehen, was hochgeladen würde.

- **Zielordner**: ein vollständiger Pfad, über **Auswählen**. Er darf nicht im Build-Ordner liegen oder ihn enthalten und nicht der Projektordner sein. Beim ersten Mal legt QuartzHub ihn an, samt fehlender Ordner darüber.
- **Entfernte Dateien auch im Ziel löschen** und **Nie veröffentlichen** wie bei den anderen Zielen.

Übertragen wird wie bei SFTP nur, was neu oder geändert ist. Jede Datei wird erst unter einem vorläufigen Namen geschrieben und dann umbenannt, so dass ein Sync-Programm nie eine halbe Datei sieht.

Wurde das Ziel schon einmal benutzt und ist der Ordner jetzt weg oder leer, hält QuartzHub an: Meist ist dann ein Laufwerk nicht verbunden. Soll der Ordner wirklich neu entstehen, zuerst **Alles neu hochladen** wählen (6.12).

6.11 – Webhook

Manche Anbieter bauen die Website selbst, aus dem Repository: Netlify, Cloudflare Pages, Vercel oder eine eigene CI. Ein **Webhook** stößt diesen Build an. Es werden dabei keine Dateien übertragen; der Anbieter holt sich den Stand aus dem Repository.

Das heißt auch: Was nicht im Repository ist, kommt nicht an. Vor dem Webhook gehört also ein **Push** unter **Git-Sync** (6.5).

Einrichten

1. Beim Anbieter einen **Build-Hook** anlegen; er nennt dafür eine Adresse, etwa `https://api.netlify.com/build_hooks/…`.
2. Unter **Einstellungen ▸ Zugänge** einen Zugang **Webhook** mit dieser Adresse anlegen. Die Adresse ist ein Geheimnis: Wer sie kennt, kann Builds auslösen. QuartzHub legt sie im Schlüsselbund ab und zeigt nur ihren Anfang.
3. Unter **Veröffentlichen** ein Ziel der Art **Webhook** mit diesem Zugang.

Jetzt veröffentlichen fragt kurz nach und schickt dann eine Anfrage an die Adresse. Antwortet der Anbieter mit Erfolg, ist der Build bei ihm angestoßen; ob er gelingt, zeigt der Anbieter.

6.12 – Was hochgeladen wird

Bevor QuartzHub etwas überträgt, zeigt es, was sich gegenüber dem letzten Lauf an dieses Ziel geändert hat. Unter **Veröffentlichen** steht das unter dem gewählten Ziel im Abschnitt **Änderungen**.

Die Änderungen

- **Jetzt bauen** baut die Website und rechnet danach die Änderungen neu.
- **Diff aktualisieren** rechnet sie neu, ohne zu bauen. Von selbst rechnet QuartzHub sie nicht.
- **Ausgabeordner**: derselbe wie unter **Vorschau & Build** (6.2).

Darunter steht „Neu: 3 · Geändert: 12 · Entfernt: 1“ und je eine Liste. Bei SFTP, FTP, Ordner und rsync hat jede Datei einen Haken: Ohne ihn bleibt sie bei diesem Lauf aus, beim nächsten steht sie wieder da. Für immer auslassen geht über **Nie veröffentlichen** im Ziel.

Wie verglichen wird:

Ziel	Vergleich
SFTP, FTP, Ordner	mit der Liste dessen, was QuartzHub zuletzt an dieses Ziel geschickt hat, ohne Netz
SFTP mit rsync	mit dem, was auf dem Server liegt
Git-Branch	mit dem Branch im Netz
Dokument	ob die Dateien schon da sind
Webhook	gar nicht; es wird nichts übertragen

Alles neu hochladen vergisst die Liste des letzten Laufs; danach bietet die Vorschau wieder den ganzen Build an. Das braucht man, wenn auf dem Server etwas von Hand gelöscht wurde oder eine Übertragung abgebrochen ist. Auch wer ein Ziel auf einen anderen Ordner oder Zugang umstellt, lädt beim nächsten Mal alles hoch. Bei rsync gibt es den Knopf nicht: rsync vergleicht ohnehin mit dem Server.

Der Lauf

Jetzt veröffentlichen in der Symbolleiste fragt nach, mit der Zahl der Dateien, die hochgeladen und gelöscht werden; bei Git-Branch mit dem Hinweis, dass der Branch ersetzt wird. Es baut nicht selbst; hochgeladen wird, was im Ausgabeordner liegt. Ein Snapshot entsteht dabei nicht.

Während des Laufs zeigt ein Balken, welche Datei gerade dran ist. Man kann die Seite verlassen; der Lauf geht weiter, und beim Beenden fragt QuartzHub nach. Abbrechen

lässt sich nur ein Export.

Danach steht dort **Veröffentlichung erfolgreich.**, **Veröffentlichung fehlgeschlagen.** oder, wenn der Server einzelne Löschungen verweigert hat, wie viele Dateien liegen geblieben sind; die versucht QuartzHub beim nächsten Mal wieder. Die Ausgabe darunter listet jede Datei: `↑` hochgeladen, `×` gelöscht.

> **Die übliche Reihenfolge**
>
> Speichern, **Jetzt bauen**, die Liste ansehen, **Jetzt veröffentlichen**. Steht mehr in der Liste als erwartet, lohnt ein Blick, bevor man drückt.

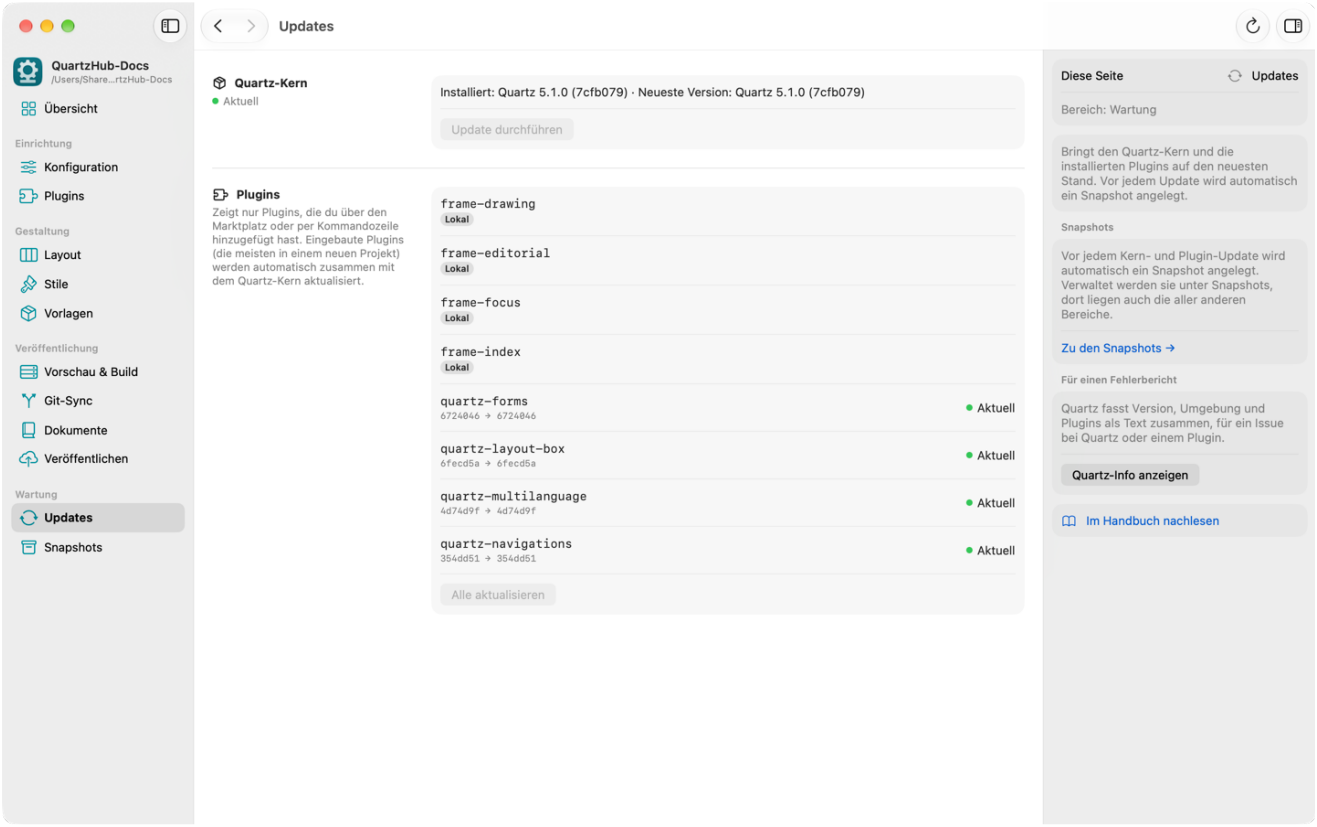
7 – Wartung

Eine Website, die läuft, braucht wenig Pflege. Ab und zu gibt es Neues von Quartz oder einem Plugin, und manchmal geht etwas schief. Dafür ist die Gruppe **Wartung** da.

1. **Updates**: Quartz und die Plugins auf den neuesten Stand bringen.
2. **Snapshots**: was sie sind, wann sie entstehen, was sie enthalten.
3. **Zurückholen**: einen Snapshot oder einzelne Dateien wiederherstellen.
4. **Aufräumen**: was sich ansammelt und wie man es loswird.

7.1 – Updates

Quartz entwickelt sich weiter, die Plugins ebenso. **Updates** sagt, ob es Neues gibt, und bringt es ins Projekt. Vor jedem Update legt QuartzHub einen Snapshot an.



Quartz ist aktuell, die Plugins aus dem Marktplatz auch.

Prüfen

Beim Öffnen der Seite fragt QuartzHub bei GitHub nach, ob es eine neuere Fassung von Quartz gibt und ob die installierten Plugins neuere haben. **Erneut prüfen** in der Symbolleiste fragt noch einmal. Ohne Netz steht dort **Nicht prüfbar**, nicht „Aktuell“.

Der Quartz-Kern

Die Zeile zeigt, welche Fassung installiert ist und welche die neueste, und wie viele Änderungen fehlen. Daneben **Aktuell**, **Update verfügbar**, **Nicht abgeschlossen** oder **Nicht prüfbar**. **Neu in Quartz …** klappt die Änderungen auf, die Quartz selbst dazu aufgeschrieben hat.

Zeigt der Remote `quartz-upstream` des Projekts nicht auf jackyzha0/quartz, etwa auf eine eigene Kopie, sagt ein Hinweis darunter, dass **Update durchführen** von dort holt, während die Zeile den Stand von jackyzha0/quartz zeigt.

Update durchführen fragt nach und tut dann:

1. einen Snapshot anlegen („Vor Kern-Update“);
2. die neue Fassung von Quartz holen und mit dem Projekt zusammenführen. Eigene Pakete in `package.json` legt QuartzHub dafür kurz beiseite und trägt sie danach wieder ein;
3. die npm-Pakete neu installieren;
4. die Website einmal bauen.

Danach steht dort „Das Kern-Update ist abgeschlossen.“, die Ausgabe zum Aufklappen und **Snapshot von vor dem Update öffnen**. Ab Quartz 5.1 lässt QuartzHub danach Quartz’ eigene Prüfung der Plugins laufen und sagt, ob Einträge umgestellt werden sollten (4.5).

> ⚠ **Erst committen, dann aktualisieren**
>
> Ein Update führt Quartz’ Änderungen mit dem Projekt zusammen. Ungespeicherte Änderungen an Dateien von Quartz können dabei im Weg stehen; QuartzHub nennt sie dann. Wer **Git-Sync** benutzt, committet vorher (6.5).

Wenn es klemmt

- **Konflikte**: Haben Quartz und das Projekt dieselbe Stelle geändert, bleibt das Zusammenführen stehen. Die Seite nennt die Dateien. **Merge abbrechen** setzt auf den Stand vor dem Update zurück; sonst löst man die Konflikte in einem Editor und führt das Update noch einmal durch. Eigene Texte in den Sprachdateien schützt der Update-Schutz (4.4).
- **Nicht abgeschlossen**: Ist `npm install` gescheitert oder fehlen eigene Pakete noch, sagt die Seite das. **Update durchführen** holt es nach.
- **Zurück**: Der Snapshot von vor dem Update führt zum alten Stand. Damit auch die Updates-Seite den alten Stand meldet, beim Wiederherstellen **Auch den Projekt-Commit zurücksetzen** einschalten (7.3).

Plugins

Hier stehen die Plugins, die über den Marktplatz oder die Kommandozeile dazukamen. Die eingebauten Plugins von Quartz kommen mit dem Kern-Update. Jede Zeile zeigt die installierte und die neueste Fassung und **Aktuell**, **Aktualisieren** oder **Nicht prüfbar**; **Lokal** heißt, das Plugin liegt im Projekt selbst (etwa ein eigener Frame) und hat keine Quelle im Netz.

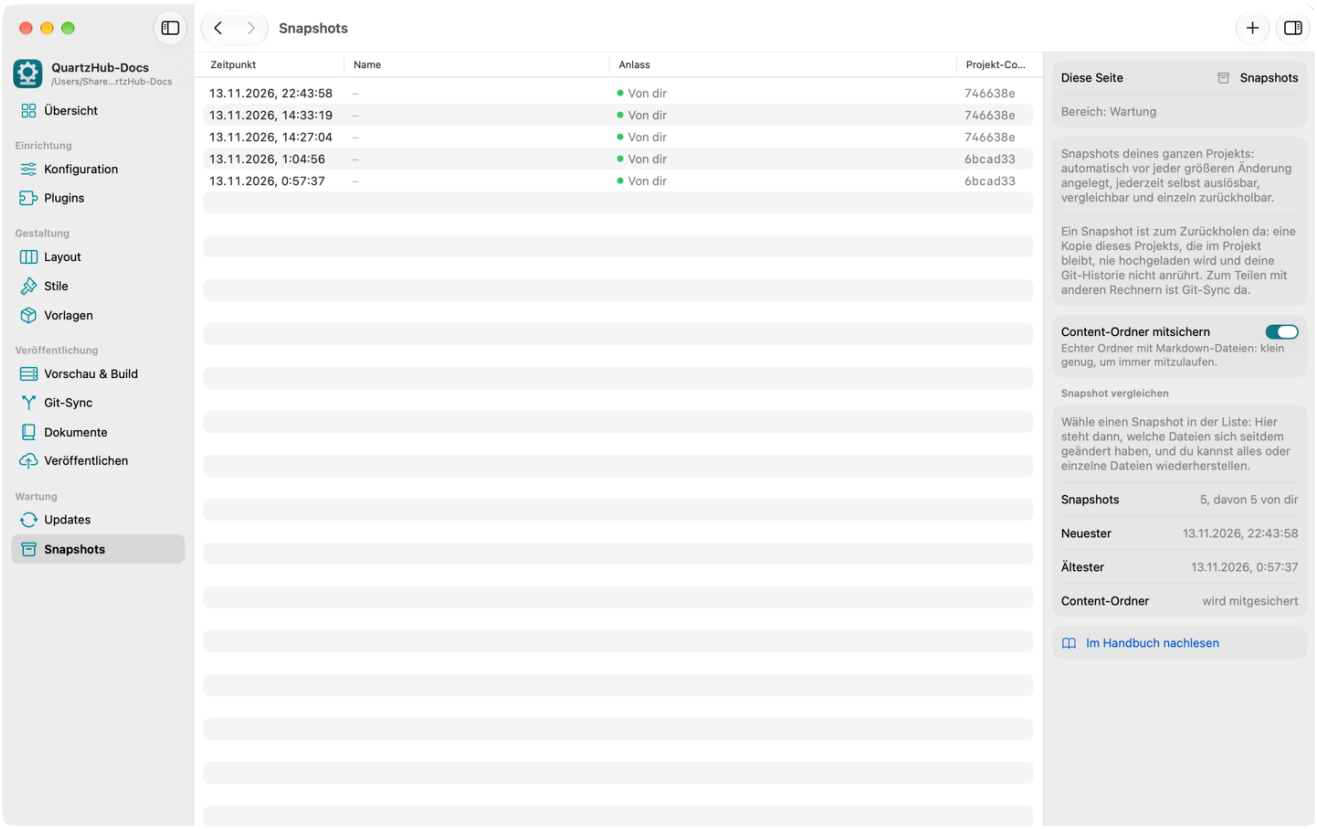
Aktualisieren und **Alle aktualisieren** fragen nach, legen einen Snapshot an und lassen Quartz die neueste Fassung holen. Die Konfiguration bleibt dabei, wie sie ist.

Für einen Fehlerbericht

Ab Quartz 5.1 steht im Inspektor **Quartz-Info anzeigen**: Quartz fasst Version, Umgebung und Plugins als Text zusammen, den man in einen Fehlerbericht an Quartz oder ein Plugin kopieren kann.

7.2 – Snapshots

Ein Snapshot ist eine Kopie des ganzen Projekts zu einem Zeitpunkt. Er bleibt im Projekt, wird nie hochgeladen und rührt die eigene git-Geschichte nicht an. Er ist zum Zurückholen da, wenn etwas schief ging; zum Teilen mit anderen Rechnern ist Git-Sync da.



Ein Snapshot „Von dir“. Rechts, ob der Content-Ordner mitgesichert wird.

Was drin ist

Alles im Projekt außer dem, was ein Build oder eine Installation neu erzeugt: `node_modules/`, `public/`, die heruntergeladenen Plugins und Zwischenspeicher. Also Konfiguration, Stylesheets, eigene Frames, Übersetzungen, Bücher, Ziele und die Notizen, wenn der Content-Ordner ein echter Ordner ist.

Content-Ordner mitsichern (Inspektor, Vorgabe an) nimmt die Notizen mit oder lässt sie aus. Ein verlinkter Vault wird nie gesichert; den sichert man mit Obsidian oder Time Machine.

Die Snapshots liegen in `.quartz-gui/snapshots.git/`, einem eigenen git-Repository.

Wann QuartzHub einen anlegt

Von selbst, vor jeder größeren Änderung. Die Spalte **Anlass** sagt, wovor:

Anlass	Wovor
Vor Konfigurations-Änderung	Speichern der Konfiguration, etwa unter Konfiguration, Layout, Stile, Plugins; höchstens einer alle 15 Minuten
Vor Plugin-Änderung	Plugin installieren, entfernen, aktualisieren, umstellen; einen neuen Frame anmelden
Vor Kern-Update	dem Update von Quartz
Vor Content-Wechsel	dem Wechsel des Content-Ordners
Vor Vorlagen-Import	dem Anwenden einer Vorlage
Vor Umstellung der Stile	dem Umstellen alter Stylesheets
Vor Wiederherstellung	jedem Zurückholen
Altes Config-Backup	Sicherungen der Konfiguration aus frühen Fassungen

Hat sich seit dem letzten Snapshot nichts geändert, legt QuartzHub keinen neuen an; vor einem Kern-Update legt es trotzdem einen an.

Selbst anlegen: ➕ **Snapshot anlegen** in der Symbolleiste, mit einem Namen wie „vor dem Theme-Wechsel“, oder **Snapshot anlegen** auf der Übersicht, ohne Namen. Diese tragen den Anlass **Von dir**.

Wie lange sie bleiben

Eigene Snapshots und die vor einem Kern-Update bleiben, bis man sie löscht. Die übrigen automatischen dünnt QuartzHub mit der Zeit aus:

- jünger als ein Tag: alle;
- bis eine Woche: einer je Tag;
- bis acht Wochen: einer je Woche;
- älter: einer je Monat.

Es bleibt jeweils der neueste. Wer einen Stand sicher behalten will, legt selbst einen Snapshot an.

Die Liste

Zeitpunkt, **Name**, **Anlass** und, im breiten Fenster, **Projekt-Commit**: der Commit des Projekts, als der Snapshot entstand (ohne git-Repository ein Strich). Im Kontextmenü **Als ZIP exportieren** und **Löschen**. Was ein Snapshot gegenüber heute ändert und wie man ihn zurückholt, beschreibt 7.3.

7.3 – Zurückholen

Vergleichen

Ein Klick auf einen Snapshot unter **Snapshots** vergleicht ihn mit dem heutigen Stand. Der Inspektor zeigt **Vergleich mit dem heutigen Stand**: wie viele Dateien sich unterscheiden und jede mit ihrem Zustand:

- **seitdem neu**: gab es damals nicht; Wiederherstellen löscht sie;
- **seitdem gelöscht**: gab es damals; Wiederherstellen legt sie wieder an;
- **geändert**: das Symbol daneben zeigt den Unterschied Zeile für Zeile.

Ist **Content-Ordner mitsichern** aus oder der Content-Ordner ein Link, bleiben die Notizen beim Vergleichen und Wiederherstellen außen vor.

Wiederherstellen

- **Alles wiederherstellen** setzt das ganze Projekt auf den Snapshot zurück. Dateien, die es damals nicht gab, werden gelöscht.
- **Auswahl wiederherstellen** setzt nur die angehakten Dateien zurück.

Beide fragen nach und legen vorher einen Snapshot vom heutigen Stand an („Vor Wiederherstellung“); ein Wiederherstellen lässt sich also selbst wieder zurücknehmen. Hat sich `package.json` geändert, installiert QuartzHub danach die npm-Pakete neu. Während es läuft, fragt QuartzHub beim Beenden nach.

Auch den Projekt-Commit zurücksetzen erscheint, wenn der Snapshot auf einem anderen git-Commit entstand als heute, etwa vor einem Kern-Update. Es setzt zusätzlich die git-Geschichte des Projekts zurück; nur dann meldet **Updates** wieder den alten Stand von Quartz.

Die Notizen folgen dabei dem Schalter **Content-Ordner mitsichern**: Ist er aus, bleiben sie, wie sie sind, und git zeigt sie danach als Änderungen gegenüber dem alten Commit. Ist er an, kommen sie aus dem Snapshot zurück; ihr Stand davor steckt im Snapshot „Vor Wiederherstellung“.

> ⚠ **Ein gepushter Commit**
>
> Wurde der Commit schon gepusht, weicht der lokale Stand nach dem Zurücksetzen vom Repository im Netz ab.

Als ZIP exportieren

Als ZIP exportieren im Kontextmenü oder im Inspektor schreibt den Stand eines Snapshots

in eine ZIP-Datei, etwa um eine einzelne Datei herauszuholen, ohne etwas zurückzusetzen.

Beiseitegelegte Content-Ordner

Wer den Content-Ordner wechselt, legt den alten beiseite (4.2). Diese Ordner stehen unter **Snapshots** in einer eigenen Liste, mit Zeitpunkt und Größe oder dem Ziel des damaligen Links. **Wiederherstellen** setzt einen wieder ein und legt den aktuellen ebenfalls beiseite.

7.4 – Aufräumen

Das meiste räumt QuartzHub selbst weg: alte automatische Snapshots dünnt es aus, Protokolle früherer Vorschauen löscht es beim nächsten Start der Vorschau, liegen gebliebene Arbeitsordner von Export, Relay und SSH beim Start der App. Für den Rest gibt es diese Stellen:

Was	Wo	Was es entfernt
Snapshots	**Snapshots**, Kontextmenü **Löschen** oder die Rückschritttaste	den Snapshot; Platz wird nach einigen Minuten frei
Beiseitegelegte Content-Ordner	**Snapshots**, Liste darunter, Papierkorb	den Ordner endgültig; er ist die einzige Kopie
Heruntergeladene Plugins, die nichts mehr benutzt	**Plugins ▸ Installiert ▸ Verwaiste Plugin-Ordner entfernen**	Ordner unter `.quartz/plugins/`, auf die die Konfiguration nicht zeigt
Ungenutzte Schriften	**Stile ▸ Basis ▸ Ungenutzte Schriften**	importierte Schriftdateien und ihre Regeln
Stylesheets	**Stile ▸ Eigenes CSS**, Ladereihenfolge	die Datei; zurück nur über git oder einen Snapshot
Beschreibungen der Themes	**Einstellungen ▸ Daten & Wartung ▸ Cache leeren**	den Zwischenspeicher; wird bei Bedarf neu geladen
Was zuletzt an ein Ziel ging	**Veröffentlichen ▸ Alles neu hochladen**	die Liste des letzten Laufs; der nächste lädt alles hoch
Ein Projekt aus der Liste	**Projekte ▸ Aus der Liste entfernen**	nur den Eintrag, nicht den Ordner (2.1)

`node_modules/` und `public/` darf man jederzeit löschen: Die Übersicht bietet danach **Installieren** an, und der nächste Build legt `public/` neu an.

8 – Nachschlagen

Dieser Teil liest sich nicht von vorn nach hinten; er ist zum Nachsehen da.

1. **Menüs und Kürzel**
2. **Wo die Daten liegen**: im Profil, im Projekt, im Schlüsselbund.
3. **Verbindungen ins Netz**: wann QuartzHub wohin Verbindung aufnimmt.
4. **Bekannte Grenzen**: was QuartzHub nicht kann oder anders macht.
5. **Glossar**

8.1 – Menüs und Kürzel

Auf einer deutschen Tastatur liegen `[` und `]` auf ⌥5 und ⌥6; macOS bietet für ⌘[und ⌘] deshalb ⌘Ö und ⌘Ä an. In Dialogen gilt immer: Eingabetaste bestätigt, Esc bricht ab.

QuartzHub

Befehl	Kürzel
Einstellungen…	⌘,

Ablage

Befehl	Kürzel	
Neues Projekt…	⌘N	2.2
Vorhandenes Projekt öffnen…	⌘O	2.3
Speichern	⌘S	speichert die Seite vorn; grau, wenn nichts zu speichern ist

Darstellung

Befehl	Kürzel
Seitenleiste ein-/ausblenden	^⌘S
Inspektor einblenden / ausblenden	^⌘I

Gehe zu

Befehl	Kürzel
Zurück	⌘[(⌘Ö)
Vorwärts	⌘] (⌘Ä)
Übersicht, Konfiguration, Plugins, Layout, Stile, Vorlagen, Vorschau & Build, Git-Sync, Dokumente, Veröffentlichen, Updates, Snapshots	–

Projekt

Wirkt auf das Projekt im vordersten Fenster.

Befehl	Kürzel	
Server starten	⌘R	6.1
Server stoppen	⌘.	
Build ausführen	⌘B	6.2
Projektordner im Finder zeigen	–	

Anordnen

Gilt in Listen, die man umsortieren kann: **Plugins ▸ Installiert**, **Mehrsprachigkeit** und die eigene Reihenfolge eines Buchs.

Befehl	Kürzel
Nach oben	⌥⌘↑
Nach unten	⌥⌘↓

Auf der Bühne unter **Layout ▸ Komponenten** verschieben **Früher** und **Später** im Inspektor.

Fenster

Befehl	Kürzel
Projekte	⇧⌘0

Darunter stehen die offenen Projektfenster, je mit Projekt und Seite.

Hilfe

Befehl	Was er tut
Handbuch	öffnet dieses Handbuch im Browser, in der Sprache von QuartzHub
Diese Seite im Handbuch	öffnet das Kapitel zur Seite und zum Reiter des vorderen Projektfensters
Rückmeldung senden…	öffnet ein neues Issue im Repository von QuartzHub auf GitHub, mit Version und macOS
Quartz-Dokumentation	öffnet **quartz.jzhao.xyz**
Plugin-Katalog	öffnet die quartz-community auf GitHub
Datenordner von QuartzHub öffnen	zeigt den Profilordner im Finder (8.2)

Lizenzen öffnen	zeigt die Lizenztexte der mitgelieferten Programme

8.2 – Wo die Daten liegen

Im Profil

`~/Library/Application Support/QuartzHub`, zu öffnen über **Hilfe ▸ Datenordner von QuartzHub öffnen** oder **Einstellungen ▸ Daten & Wartung**.

Datei	Inhalt
`projects.json`	die Projektliste: Name, Ordner, wann hinzugefügt und zuletzt geöffnet
`settings.json`	die Einstellungen: Sprache, Design, Node-Laufzeit, Verhalten beim Beenden, Standard-Projektverzeichnis
`connections.json`	die Zugänge, ohne Passwörter, Schlüssel und Tokens
`relay-secrets.json`	je Projekt, welche Empfänger und Captcha-Secrets das Formular-Relay hat, ohne die Werte
`running-servers.json`	die Vorschau-Server, die QuartzHub gestartet hat; für die Frage beim nächsten Start
`templates/`	das zuletzt geladene Basis-Template
`theme-docs/`	die Beschreibungen der Community-Themes

Eine Datei, die QuartzHub nicht lesen kann, legt es als `<name>.corrupt-<Zeit>` beiseite, statt sie zu überschreiben, und sagt es unter **Einstellungen ▸ Daten & Wartung**.

Fensterpositionen und die Sprache der Oberfläche merkt sich macOS in den Einstellungen der App.

Im Schlüsselbund

Im Anmelde-Schlüsselbund, Dienst „QuartzHub“:

- **„QuartzHub – “**: Passwort, Schlüssel, Token oder Webhook-URL eines Zugangs.
- **„QuartzHub – Relay“**: je Projekt Empfänger-Adressen und Secrets des Formular-Relays.

Ansehen und löschen lassen sich die Einträge mit der Schlüsselbundverwaltung von macOS. Beim Umzug auf einen anderen Mac kommen sie nicht mit; man trägt sie dort neu ein.

Im Projekt

Ort	Inhalt
`quartz.config.yaml`	die Konfiguration: Titel, Adresse, Plugins, Layout

`content/`	die Notizen, ein Ordner oder ein Link
`quartz/styles/custom.scss`	das eigene CSS; lädt alle anderen Stylesheets
`quartz/styles/_quartzcontrol.scss`	Ladereihenfolge, Schriften und Variablen, von QuartzHub geschrieben
`quartz/styles/custom/`, `imported/`	eigene Stylesheets
`quartz/static/`	Projektbild, Schriften, Bilder, Schnipsel
`quartz/i18n/locales/`	die Sprachdateien, die **Übersetzungen** bearbeitet
`quartz-books/`	die Bücher; kommen mit git mit
`quartz-forms/keys/`	öffentliche Schlüssel für verschlüsselnde Formulare
`.quartz-gui/`	was QuartzHub zum Projekt speichert, siehe unten; steht in `.gitignore`

In `.quartz-gui/`:

Eintrag	Inhalt
`publish-targets.json`	die Ziele zum Veröffentlichen, ohne Passwörter
`deploy-manifest-<Ziel>.json`	was zuletzt an ein Ziel geschickt wurde
`snapshots.git/`	die Snapshots, ein eigenes git-Repository
`snapshot-settings.json`	die Einstellungen der Snapshots
`content-backups/`	beiseitegelegte Content-Ordner
`authored-frames/`	die eigenen Frames
`layout-breakpoints.json`	die Breakpoints der eigenen Frames
`theme-presets.json`	die Theme-Presets
`forms-relay.json`	die Einstellungen des Formular-Relays, ohne Geheimnisse
`project-prefs.json`	etwa der Ausgabeordner
`project-icon.json`, `icon-original.png`	das gewählte Projektbild und Quartz' ursprüngliches
`locale-baseline/`	die Sprachdateien vor der ersten eigenen Änderung
`core-update.json`	ein Kern-Update, das nicht zu Ende lief
`logs/`	die Ausgabe der Vorschau-Server

QuartzHub legt `.quartz-gui/` erst an, wenn es etwas dorthin schreibt; nur lesen

legt nichts an.

8.3 – Verbindungen ins Netz

QuartzHub sendet keine Nutzungsdaten. Ins Netz geht es nur für Dinge, die ohne Netz nicht gehen, und dann zu den Stellen in dieser Tabelle. Es benutzt dabei die Proxy- und Zertifikatseinstellungen von macOS.

Von selbst, beim Öffnen einer Seite

Wann	Wohin	Was
Fenster **Projekte** öffnen	api.github.com (QuartzHubs Repository)	fragt, ob es eine neuere Fassung von QuartzHub gibt, höchstens alle sechs Stunden, ohne Anmeldung
Übersicht und **Updates** öffnen	github.com (Quartz' Repository)	fragt die neueste Fassung von Quartz ab, ohne Anmeldung
Übersicht, **Updates**, **Plugins** öffnen	die Repositories der installierten Plugins, meist github.com	fragt je Plugin die neueste Fassung ab, ohne Anmeldung

Diese Abfragen schicken nichts als die Frage selbst. Ohne Netz zeigen die Seiten „Nicht prüfbar".

Wenn man etwas tut

Tätigkeit	Wohin
Neues Projekt anlegen	github.com (Quartz), die npm-Registry
Basis-Template benutzen	raw.githubusercontent.com (einmal am Tag)
Projekt duplizieren, Plugins installieren, wiederherstellen oder aktualisieren, Kern-Update	die npm-Registry, für Plugins aus git deren Repository
Plugins ▸ Marktplatz	api.github.com, mit dem GitHub-Token, falls einer gespeichert ist
Stile ▸ Community-Themes: Katalog und Details	die npm-Registry, api.github.com, cdn.jsdelivr.net
Beschreibungen der Theme-Einstellungen	raw.githubusercontent.com (Obsidians Theme-Verzeichnis und das Theme), danach aus dem Zwischenspeicher
Google-Schriften wählen	

oder lokal ausliefern	fonts.googleapis.com, fonts.gstatic.com
Ein Buch exportieren	Stylesheets, Schriften und Bilder, die die Website von außen lädt; für Formeln cdn.jsdelivr.net, für Diagramme cdnjs.cloudflare.com
Token prüfen, Repository anlegen, GitHub Pages einrichten	api.github.com, mit dem Token
Git-Sync, Git-Branch-Ziele	das Repository des Projekts, bei github.com mit dem Token
Veröffentlichen per SFTP, rsync, FTP	der eigene Server, mit dem Zugang
Webhook	die Adresse des Anbieters
Formular-Relay prüfen	die `send.php` auf dem eigenen Webspace
Links im Hilfe-Menü und in der Vorschau	im Browser

Der GitHub-Token geht nur an GitHub, und zu git nur, wenn das Repository auf github.com liegt; er steht nie in einer Adresse oder auf der Kommandozeile.

Was ohne Netz geht

Bearbeiten, Vorschau, Build, Snapshots, Veröffentlichen in einen Ordner. Die Website selbst kann beim Bauen oder in der Vorschau Dinge von außen laden, etwa Google-Schriften; das bestimmen Quartz und seine Plugins (5.5).

Liegt ein Server im eigenen Netz, fragt macOS einmal, ob QuartzHub aufs lokale Netzwerk darf (1.2).

8.4 – Bekannte Grenzen

Rechner und Verteilung

- **macOS 26 oder neuer, nur Apple silicon.** Für Intel-Macs, Linux und Windows gibt es keine Fassung.
- **Nicht von Apple beglaubigt.** Beim ersten Start braucht es einmal „Dennoch öffnen“ (1.2).
- **Aktualisieren geht von Hand.** Gibt es eine neuere Fassung, sagt das Fenster **Projekte** es (2.1); laden und ersetzen muss man sie selbst (1.2).
- **Ein neues Projekt braucht das Netz**: Quartz kommt von GitHub, das Basis-Template ebenfalls.

Bewusst so

- Keine Vergrößerung der Oberfläche; die Textgröße folgt macOS.
- Die Desktop-Vorschau wird mit 1280 Pixeln gebaut und eingepasst.
- Das Export-Fenster für PDFs ist immer hell, Diagramme also auch.

Noch nicht da

- PDFs sind nicht für Bildschirmleser ausgezeichnet.

Worauf man achten sollte

- **Push in Git-Sync ist ein Force-Push** (6.5).
- **rsync löscht mit eingeschaltetem Löschen alles im Zielordner, was nicht zum Build gehört** (6.9).
- **Beiseitegelegte Content-Ordner sind die einzige Kopie**; Snapshots enthalten sie nicht (4.2).
- **Ältere automatische Snapshots werden ausgedünnt** (7.2).

8.5 – Glossar

Basis-URL

Die Adresse, unter der die fertige Website erreichbar ist, ohne `https://`, etwa `name.github.io/projekt`. Quartz baut daraus Links, den RSS-Feed und Vorschaubilder (4.1).

Branch

Ein Entwicklungszweig in einem git-Repository. Quartz-Projekte stehen auf dem Branch `v5`; GitHub Pages liefert die Website meist aus einem eigenen Branch `gh-pages` aus (6.8).

Breakpoint

Die Fensterbreite, ab der eine Website auf das Tablet- oder Mobil-Layout umschaltet. Für eigene Frames 1200 und 800 Pixel, einstellbar (5.4).

Build

Das Erzeugen der fertigen Website aus den Notizen: HTML, CSS, Bilder, alles im Ausgabeordner, meist `public/` (6.2).

Callout

Ein hervorgehobener Kasten in einer Notiz, in Obsidian geschrieben als `> [!note]`. Quartz stellt ihn auf der Website ebenso dar.

Commit

Ein gespeicherter Stand in einem git-Repository. **Git-Sync** fasst lokale Änderungen in einem Commit zusammen (6.5).

Community-Theme

Ein fertiges Farbschema aus dem Katalog von `@quartz-themes`, das sich über die Basis-Farben legt (5.6).

Content-Ordner

Der Ordner `content/` im Projekt mit den Notizen, aus denen die Website entsteht: ein echter Ordner oder ein Link auf einen Vault (4.2).

CSS-Variable

Ein benannter Wert im CSS, etwa `--secondary` für die Farbe der Links. Fast alle Farben und Schriften einer Quartz-Website sind Variablen (5.7).

Dev-Server

Quartz' eigener Server für die Vorschau. Er baut die Website, liefert sie auf dem Mac aus und baut bei jeder Änderung neu (6.1).

Frame

Das Raster einer Seite: welche Bereiche es gibt und wie sie liegen. Quartz bringt drei mit; eigene baut man unter **Layout ▸ Frames** (5.4).

Frontmatter

Der Block zwischen `---` am Anfang einer Notiz, mit Angaben wie `title`, `tags` oder `draft`.

Komponente

Ein sichtbarer Teil einer Seite, etwa Suche, Inhaltsverzeichnis oder Graph, mit einer Position wie Kopfbereich oder Seitenleiste (5.3).

Marktplatz

Der Katalog der Plugins der quartz-community auf GitHub, unter **Plugins ▸ Marktplatz** (4.10).

npm

Der Paketmanager von Node. Quartz und viele Plugins sind npm-Pakete; sie liegen im Projekt in `node_modules/`. QuartzHub bringt npm mit.

Plugin

Eine Erweiterung von Quartz. Fast alles, was Quartz tut, tut ein Plugin: Markdown lesen, Links auflösen, die Suche, die Seitenleiste (4.5).

Preset

Eine gespeicherte Zusammenstellung eines Community-Themes mit seinen Einstellungen (5.6).

Projekt

Ein Ordner mit einer Quartz-Website: Konfiguration, Stylesheets, Plugins und Content-Ordner (2).

Quartz

Ein Programm, das aus Markdown-Notizen eine Website baut, ein „statischer Website-Generator“. Es versteht Obsidians Wikilinks, Callouts und eingebettete Bilder. QuartzHub bedient Quartz 5 (**quartz.jzhao.xyz**).

Repository

Ein Ordner unter Versionsverwaltung mit git, mit seiner ganzen Geschichte. Jedes Quartz-Projekt ist eines; im Netz liegt es etwa bei GitHub (6.5).

Seitentyp

Eine Art von Seite, die Quartz baut: Inhaltsseiten, Ordnerseiten, Tag-Seiten, die 404-Seite und weitere je nach Plugins (5.2).

Snapshot

Ein gespeicherter Stand des ganzen Projekts, den QuartzHub vor größeren Änderungen von selbst anlegt und zu dem man zurückkehren kann (7.2).

Token

Ein Schlüssel, mit dem sich ein Programm bei einem Dienst ausweist, hier der persönliche Zugriffstoken für GitHub. Er liegt im Schlüsselbund (6.7).

Vault

Ein Ordner mit Notizen, den Obsidian verwaltet. Der Content-Ordner eines Projekts kann auf einen Vault oder einen Ordner darin verweisen (4.2).

Wikilink

Ein Link in doppelten eckigen Klammern, etwa `[[Notiz]]`, wie Obsidian ihn schreibt. Quartz versteht ihn und macht daraus einen Link auf der Website.

Ziel und Zugang

Ein **Zugang** ist ein Konto bei einem Anbieter und gilt für alle Projekte; ein **Ziel** gehört zu einem Projekt und sagt, wohin seine Website kommt (6.7).